From First Principles to Modern Large Language Models

Pablo Leyva

Contents

Block A — Foundations

Chapter 1: Sets, functions, logic, proofs

Motivation

Every object in this book—a vector, a probability distribution, a neural network, a token—is ultimately a *set with structure*. Before we can build the embedding map of Chapter 19 or even define the real numbers in Chapter 5, we need a precise grammar for membership, functions, and proof. Chapter 8 (linear maps) treats functions between vector spaces; Chapter 15 (probability) treats measures on σ-algebras of sets. Both collapse without the vocabulary below. We therefore start, with no apology, from the bottom.

Definitions

Definition 0.1 (Set, membership, subset)**.** *A* set S *is a collection of distinct objects called elements. We write* $x \in S$ *when* x *is an element of* S*, and* $x \notin S$ *otherwise. The* empty set $\emptyset$ *is the unique set with no elements. We say* A *is a* subset *of* B*, written* $A \subset B$*, iff* $\forall x\,(x \in A \Rightarrow x \in B)$*. Two sets are equal,* $A = B$*, iff* $A \subset B$ *and* $B \subset A$*.*

Definition 0.2 (Boolean operations)**.** *Given* A, B *inside a universe* U*:*

$$\begin{aligned} A \cup B &:= \{x \in U : x \in A \vee x \in B\}, \\ A \cap B &:= \{x \in U : x \in A \wedge x \in B\}, \\ A^c &:= \{x \in U : x \notin A\}, \\ \mathcal{P}(S) &:= \{T : T \subset S\}. \end{aligned}$$

Definition 0.3 (Function, image, preimage)**.** *A* function $f : A \to B$ *assigns to each* $a \in A$ *exactly one* $f(a) \in B$*. The image is* $f(A) := \{f(a) : a \in A\}$ *and the preimage of* $T \subset B$ *is* $f^{-1}(T) := \{a \in A : f(a) \in T\}$*. We call* f*:*

- injective *iff* $\forall a_1, a_2 \in A,\ f(a_1) = f(a_2) \Rightarrow a_1 = a_2$*;*
- surjective *iff* $\forall b \in B,\ \exists a \in A,\ f(a) = b$*;*
- bijective *iff both.*

Remark 0.4 (Logic). Propositional connectives: $\wedge, \vee, \neg, \Rightarrow, \Leftrightarrow$. Quantifiers: $\forall, \exists$. Standard proof techniques: *direct* (assume P, derive Q), *contrapositive* ($P \Rightarrow Q \equiv \neg Q \Rightarrow \neg P$), *contradiction* (assume $\neg P$, derive $\bot$), and *induction* (Theorem 0.7).

Theorems and proofs

Theorem 0.5 (De Morgan's laws for sets)**.** *For sets* $A, B \subset U$*,*

$$(A \cup B)^c = A^c \cap B^c \qquad \text{and} \qquad (A \cap B)^c = A^c \cup B^c.$$

Proof. We prove the first; the second is symmetric. Fix arbitrary $x \in U$. We show the biconditional $x \in (A \cup B)^c \Leftrightarrow x \in A^c \cap B^c$.

($\Rightarrow$) Assume $x \in (A \cup B)^c$. By definition, $x \notin A \cup B$, i.e. $\neg(x \in A \vee x \in B)$. By the propositional De Morgan law $\neg(P \vee Q) \equiv \neg P \wedge \neg Q$, this is $\neg(x \in A) \wedge \neg(x \in B)$, i.e. $x \notin A$ and $x \notin B$. Hence $x \in A^c$ and $x \in B^c$, so $x \in A^c \cap B^c$.

($\Leftarrow$) Assume $x \in A^c \cap B^c$. Then $x \notin A$ and $x \notin B$, so $\neg(x \in A) \wedge \neg(x \in B)$, which by propositional De Morgan equals $\neg(x \in A \vee x \in B)$, i.e. $x \notin A \cup B$, i.e. $x \in (A \cup B)^c$.

For the second identity, the same argument with $\wedge$ and $\vee$ interchanged (using $\neg(P \wedge Q) \equiv \neg P \vee \neg Q$) gives $x \in (A \cap B)^c \Leftrightarrow x \in A^c \cup B^c$. □

Theorem 0.6 (Composition of injections is injective)**.** *If $f : A \to B$ and $g : B \to C$ are injective, then $g \circ f : A \to C$ is injective.*

Proof. Let $a_1, a_2 \in A$ with $(g \circ f)(a_1) = (g \circ f)(a_2)$, i.e. $g(f(a_1)) = g(f(a_2))$. Since g is injective, $f(a_1) = f(a_2)$. Since f is injective, $a_1 = a_2$. By definition $g \circ f$ is injective. □

Theorem 0.7 (Principle of mathematical induction)**.** *Let $P(n)$ be a predicate on $\mathbb{N} = \{1, 2, 3, \ldots\}$. If*

(i) *$P(1)$ holds, and*

(ii) *$\forall n \in \mathbb{N},\ P(n) \Rightarrow P(n+1)$,*

then $\forall n \in \mathbb{N},\ P(n)$.

Proof. We assume the well-ordering principle: every nonempty subset of $\mathbb{N}$ has a least element. Let

$$S := \{n \in \mathbb{N} : \neg P(n)\}.$$

We show $S = \emptyset$ by contradiction. Suppose $S \neq \emptyset$. By well-ordering, S has a least element m. By (i), $P(1)$ holds, so $1 \notin S$, hence $m \geq 2$, so $m - 1 \in \mathbb{N}$. By minimality of m in S, $m - 1 \notin S$, i.e. $P(m-1)$ holds. By (ii) applied at $n = m - 1$, $P(m)$ holds, so $m \notin S$. This contradicts $m \in S$. Therefore $S = \emptyset$, i.e. $P(n)$ holds for every $n \in \mathbb{N}$. □

Code sketch

In the companion notebook we (a) enumerate $\mathcal{P}(\{a, b, c\})$ from scratch and verify $|\mathcal{P}(S)| = 2^{|S|}$; (b) brute-force De Morgan on $U = \{1, \ldots, 8\}$ with $A = \{1, 3, 5, 7\}$, $B = \{2, 3, 5, 7\}$; (c) implement `is_injective`, `is_surjective`, `is_bijective` as predicates over finite functions encoded as Python dictionaries; (d) verify $\sum_{k=1}^{n} k = n(n+1)/2$ both by direct summation up to $n = 20$ and by checking the induction step $S(n+1) - S(n) = n + 1$ numerically.

Connection to LLMs

A language model's *vocabulary* $\mathcal{V}$ is a finite set of tokens (typically $|\mathcal{V}| \approx 5 \times 10^4$). The tokenizer is a function $T : \text{strings} \to \mathcal{V}^*$. The *embedding map* $E : \mathcal{V} \to \mathbb{R}^d$ is a function from a finite set into a real vector space; the implementation as a lookup table $E[i]$ requires that the vocabulary index $\mathcal{V} \to \{0, 1, \ldots, |\mathcal{V}| - 1\}$ be a *bijection*. Injectivity guarantees distinct tokens get distinct rows; surjectivity guarantees every row is reachable. We revisit this in Chapter 19, where E becomes the first learnable layer of the transformer. The rest of the book is, in a strict sense, a long story about which functions between which sets one is allowed to write down.

Chapter 2: Numbers, sequences, limits, completeness

Motivation

In Chapter 1 we built logic, sets, and proof technique. We now build the arithmetic playground in which all of analysis, optimization, and ultimately neural network training takes place: the real numbers $\mathbb{R}$. The central question of this chapter is not "what are numbers?" in a metaphysical sense but rather: *what structural property of* $\mathbb{R}$ *makes it the right home for limits?* The answer is **completeness**, and it is what will let us, many chapters later, prove that gradient descent iterates $\theta_t \in \mathbb{R}^d$ actually converge.

Definitions

We accept $\mathbb{N} = \{0, 1, 2, \ldots\}$ as constructed in Chapter 1 (von Neumann ordinals). The integers $\mathbb{Z}$ extend $\mathbb{N}$ by formal additive inverses; the rationals $\mathbb{Q}$ extend $\mathbb{Z}$ by formal multiplicative inverses of nonzero elements. We then take $\mathbb{R}$ to be a complete ordered field containing $\mathbb{Q}$ (constructed via Dedekind cuts or Cauchy completion of $\mathbb{Q}$). Thus

$$\mathbb{N} \subset \mathbb{Z} \subset \mathbb{Q} \subset \mathbb{R}.$$

Definition 0.8 (Sequence)**.** *A* sequence *in a set X is a function $a : \mathbb{N} \to X$, written $(a_n)_{n\in\mathbb{N}}$.*

Definition 0.9 (Limit, ε–N)**.** *A real sequence (a_n)* converges *to $L \in \mathbb{R}$, written $a_n \to L$, iff*

$$\forall \varepsilon > 0 \;\; \exists N \in \mathbb{N} \;\; \forall n \geq N : \;\; |a_n - L| < \varepsilon.$$

Definition 0.10 (Cauchy)**.** (a_n) *is* Cauchy *iff* $\forall \varepsilon > 0 \; \exists N \; \forall m, n \geq N : |a_m - a_n| < \varepsilon$.

Definition 0.11 (Bounded, monotone)**.** (a_n) *is* bounded above *iff* $\exists M : a_n \leq M$ *for all n; bounded below symmetrically;* bounded *iff both. It is* monotone increasing *iff* $a_n \leq a_{n+1}$ *for all n.*

Definition 0.12 (Supremum, infimum)**.** *Let $S \subseteq \mathbb{R}$ be nonempty and bounded above. $u \in \mathbb{R}$ is an* upper bound *if $s \leq u$ for all $s \in S$. The* supremum $\sup S$ *is the least upper bound: an upper bound such that for every $\varepsilon > 0$ there exists $s \in S$ with $s > \sup S - \varepsilon$. The* infimum $\inf S$ *is defined dually.*

Definition 0.13 (Completeness axiom of $\mathbb{R}$)**.** *Every nonempty subset of $\mathbb{R}$ that is bounded above has a supremum in $\mathbb{R}$.*

Theorems and proofs

Theorem 0.14 (Irrationality of $\sqrt{2}$)**.** *There is no rational q with $q^2 = 2$.*

Proof. Suppose for contradiction (technique from Chapter 1) that $q = p/r$ with $p, r \in \mathbb{Z}$, $r \neq 0$, the fraction in lowest terms (so $\gcd(p, r) = 1$), and $q^2 = 2$. Then $p^2 = 2r^2$, so p^2 is even, hence p is even (since the square of an odd integer is odd). Write $p = 2k$. Substituting, $4k^2 = 2r^2$, i.e. $r^2 = 2k^2$, so r^2 is even and therefore r is even. But then $2 \mid \gcd(p, r)$, contradicting $\gcd(p, r) = 1$. □

Remark 0.15. This is the first concrete witness that $\mathbb{Q}$ is *incomplete*: the bounded-above set $S = \{q \in \mathbb{Q} : q^2 < 2\}$ has no supremum *in* $\mathbb{Q}$. The completeness axiom is exactly the patch that distinguishes $\mathbb{R}$ from $\mathbb{Q}$.

Theorem 0.16 (Bounded monotone convergence)**.** *Let (a_n) be monotone increasing and bounded above in $\mathbb{R}$. Then (a_n) converges, and* $\lim a_n = \sup_n a_n$.

Proof. Let $S = \{a_n : n \in \mathbb{N}\}$. S is nonempty (it contains a_0) and bounded above by hypothesis. By the completeness axiom, $L := \sup S$ exists in $\mathbb{R}$. Fix $\varepsilon > 0$. By the supremum's least-upper-bound property, $L - \varepsilon$ is *not* an upper bound, so there exists $N \in \mathbb{N}$ with $a_N > L - \varepsilon$. For any $n \geq N$, monotonicity gives $a_n \geq a_N > L - \varepsilon$, while $a_n \leq L$ since L is an upper bound. Hence

$$L - \varepsilon < a_n \leq L \quad \Longrightarrow \quad |a_n - L| < \varepsilon,$$

which is exactly the ε–N definition of $\lim a_n = L$. □

Lemma 0.17 (Bolzano–Weierstrass for $\mathbb{R}$)**.** *Every bounded real sequence has a convergent subsequence.*

Proof. Let $(a_n) \subseteq [c_0, d_0]$ with $c_0 < d_0$. Bisect at midpoint $m_0 = (c_0 + d_0)/2$; at least one of $[c_0, m_0]$, $[m_0, d_0]$ contains a_n for infinitely many n (else only finitely many terms total, contradicting $|\mathbb{N}| = \infty$). Call this half $[c_1, d_1]$ and pick an index n_1 with $a_{n_1} \in [c_1, d_1]$. Iterate: at stage k we have nested $[c_k, d_k]$ of length $(d_0 - c_0)/2^k$ each containing infinitely many terms, and pick $n_k > n_{k-1}$ with $a_{n_k} \in [c_k, d_k]$. The sequence (c_k) is monotone increasing and bounded above by d_0, so by Theorem 2.8 it converges to some L; similarly $(d_k) \to L$ since $d_k - c_k = (d_0 - c_0)/2^k \to 0$. Since $c_k \leq a_{n_k} \leq d_k$, the squeeze gives $a_{n_k} \to L$. □

Theorem 0.18 (Cauchy completeness of $\mathbb{R}$)**.** *Every Cauchy sequence in $\mathbb{R}$ converges.*

Proof. Let (a_n) be Cauchy.

Step 1 (bounded). Pick $\varepsilon = 1$ and N from the Cauchy condition; for $n \geq N$, $|a_n| \leq |a_N| + 1$, so (a_n) is bounded by $M = \max(|a_0|, \ldots, |a_{N-1}|, |a_N| + 1)$.

Step 2 (convergent subsequence). By the Bolzano–Weierstrass lemma, some subsequence $a_{n_k} \to L$.

Step 3 (full sequence). Fix $\varepsilon > 0$. Choose N_1 with $|a_m - a_n| < \varepsilon/2$ for all $m, n \geq N_1$ (Cauchy property), and choose K with $n_K \geq N_1$ and $|a_{n_K} - L| < \varepsilon/2$ (subsequence convergence). Then for any $n \geq N_1$,

$$|a_n - L| \leq |a_n - a_{n_K}| + |a_{n_K} - L| < \varepsilon/2 + \varepsilon/2 = \varepsilon.$$ □

Code sketch

Numerically the partial sums $S_n = \sum_{k=1}^{n} 1/k^2$ approach $\pi^2/6$. We print $|S_n - \pi^2/6|$ and locate the smallest N realizing each tolerance $\varepsilon \in \{10^{-2}, 10^{-4}, 10^{-6}\}$, contrast a Cauchy sequence $(1/n)$ with a non-Cauchy one $((-1)^n)$, run a Newton iteration to $\sqrt{2}$, and watch a bounded monotone telescoping series converge to 1.

Connection to LLMs

Training a transformer is a sequence $(\theta_t)_{t \in \mathbb{N}}$ in $\mathbb{R}^d$ produced by an optimizer (SGD, Adam). Statements like "the loss converges" or "the iterates converge to a stationary point" are *limit statements in the sense of Definition 2.2.* Convergence proofs for SGD (Chapter 13) and Adam (Chapter 14) reduce, ultimately, to bounded-monotone arguments on auxiliary scalar sequences (loss values, squared gradient norms) — and those arguments would simply be false over $\mathbb{Q}$. Completeness is not philosophical decoration; it is the load-bearing axiom under every convergence theorem in deep learning.

Chapter 3: Continuity, univariate differentiation, chain rule

Motivation

Chapter 2 built the language of limits (ε–N for sequences and ε–δ for functions). *Continuity* asserts that small input perturbations cause small output perturbations; *differentiability* strengthens this to "the perturbation is asymptotically linear." Both notions sit beneath every gradient that flows through a neural network. The chain rule, the central theorem of this chapter, is precisely the algebraic identity that backpropagation evaluates at scale (forward to Chapter 18).

Definitions

Definition 0.19 (ε–δ continuity at a point)**.** *Let $f : D \subseteq \mathbb{R} \to \mathbb{R}$ and $a \in D$. We say f is* continuous at a *iff*

$$\forall\, \varepsilon > 0\ \exists\, \delta > 0 :\ |x - a| < \delta \text{ and } x \in D \implies |f(x) - f(a)| < \varepsilon.$$

f is continuous on $S \subseteq D$ *iff it is continuous at every $a \in S$.*

Proposition 0.20 (Sequential characterization)**.** *f is continuous at a iff for every sequence $x_n \to a$ in D we have $f(x_n) \to f(a)$.*

Proof. ($\Rightarrow$) Given $\varepsilon > 0$, pick δ from continuity; since $x_n \to a$, eventually $|x_n - a| < \delta$, hence $|f(x_n) - f(a)| < \varepsilon$. ($\Leftarrow$) Suppose f is not continuous at a: some $\varepsilon_0 > 0$ admits no working δ, so for each $n \in \mathbb{N}$ pick x_n with $|x_n - a| < 1/n$ but $|f(x_n) - f(a)| \geq \varepsilon_0$. Then $x_n \to a$ and yet $f(x_n) \not\to f(a)$, contradicting the hypothesis. □

Definition 0.21 (Differentiability)**.** *For a an interior point of D, f is* differentiable at a *iff*

$$f'(a) \ := \lim_{h \to 0} \frac{f(a+h) - f(a)}{h}$$

exists in $\mathbb{R}$.

Theorems and proofs

Theorem 0.22 (Differentiable $\Rightarrow$ continuous)**.** *If f is differentiable at a, then f is continuous at a.*

Proof. For $h \neq 0$ we have the algebraic identity

$$f(a+h) - f(a) = h \cdot \frac{f(a+h) - f(a)}{h}.$$

By limit algebra (Chapter 2), the right-hand side tends to $0 \cdot f'(a) = 0$ as $h \to 0$. Hence $\lim_{h \to 0} f(a+h) = f(a)$, which is continuity at a. □

Theorem 0.23 (Sum, product, quotient rules)**.** *Let f, g be differentiable at a. Then $f + g$, fg, and (provided $g(a) \neq 0$) f/g are differentiable at a, with*

$$(f+g)'(a) = f'(a) + g'(a), \qquad (fg)'(a) = f'(a)g(a) + f(a)g'(a),$$

$$\Big(\frac{f}{g}\Big)'(a) = \frac{f'(a)g(a) - f(a)g'(a)}{g(a)^2}.$$

Proof of the product rule. Use the add-and-subtract trick:

$$\begin{aligned}\frac{(fg)(a+h)-(fg)(a)}{h} &= \frac{f(a+h)g(a+h)-f(a)g(a+h)}{h} + \frac{f(a)g(a+h)-f(a)g(a)}{h} \\ &= \frac{f(a+h)-f(a)}{h}\, g(a+h) \;+\; f(a)\,\frac{g(a+h)-g(a)}{h}.\end{aligned}$$

By Theorem 0.22, $g(a+h) \to g(a)$ as $h \to 0$. The two difference quotients tend to $f'(a)$ and $g'(a)$ respectively. Limit algebra yields $f'(a)g(a) + f(a)g'(a)$. Sum and quotient rules are analogous (use the algebraic identity $\frac{1}{g(a+h)} - \frac{1}{g(a)} = -\frac{g(a+h)-g(a)}{g(a+h)g(a)}$ and continuity of g). □

Theorem 0.24 (Chain rule, Carathéodory form)**.** *Let g be differentiable at a and f differentiable at $b := g(a)$. Then $f \circ g$ is differentiable at a and*

$$(f \circ g)'(a) = f'(g(a)) \cdot g'(a).$$

Proof. Define the *Carathéodory function*

$$\phi(y) := \begin{cases} \dfrac{f(y)-f(b)}{y-b}, & y \neq b, \\ f'(b), & y = b. \end{cases}$$

By the definition of $f'(b)$, ϕ is continuous at b. For *every* y in a neighborhood of b,

$$f(y) - f(b) = \phi(y)\,(y-b), \tag{$\star$}$$

which holds at $y = b$ trivially and by construction otherwise — a key advantage: no division by zero is ever required. Apply ($\star$) with $y = g(x)$:

$$\frac{f(g(x))-f(g(a))}{x-a} = \phi(g(x)) \cdot \frac{g(x)-g(a)}{x-a}.$$

As $x \to a$, $g(x) \to g(a) = b$ by Theorem 0.22, and ϕ is continuous at b, so $\phi(g(x)) \to \phi(b) = f'(b)$ by Proposition 0.20. The second factor tends to $g'(a)$. Limit algebra closes the proof. □

Remark 0.25. The Carathéodory device sidesteps the classical pitfall of the difference-quotient proof, where one must handle the case $g(x) = g(a)$ for x arbitrarily close to a (e.g. g constant on a sequence). Here ϕ is defined everywhere, so the identity ($\star$) requires no caveat.

Lemma 0.26 (Rolle)**.** *If f is continuous on $[a,b]$, differentiable on (a,b), and $f(a) = f(b)$, then there exists $c \in (a,b)$ with $f'(c) = 0$.*

Proof. By the extreme value theorem (Chapter 4, via compactness of $[a,b]$), f attains its maximum M and minimum m on $[a,b]$. If $M = m$ then f is constant and any $c \in (a,b)$ works. Otherwise an extremum is attained at some interior $c \in (a,b)$; Fermat's interior-extremum lemma — one-sided difference quotients have opposite signs at an interior extremum, so the two-sided limit $f'(c)$ must equal 0 — finishes the job. □

Theorem 0.27 (Mean value theorem)**.** *If f is continuous on $[a,b]$ and differentiable on (a,b), then there exists $c \in (a,b)$ with*

$$f'(c) = \frac{f(b)-f(a)}{b-a}.$$

Proof. Define the auxiliary function

$$h(x) := f(x) - \frac{f(b) - f(a)}{b - a}(x - a).$$

Then h is continuous on $[a, b]$ (sum/product of continuous functions) and differentiable on (a, b) with

$$h'(x) = f'(x) - \frac{f(b) - f(a)}{b - a}.$$

Direct computation gives $h(a) = f(a)$ and $h(b) = f(b) - (f(b) - f(a)) = f(a)$, so $h(a) = h(b)$. By Lemma 0.26 there exists $c \in (a, b)$ with $h'(c) = 0$, i.e. $f'(c) = (f(b) - f(a))/(b - a)$. □

Code sketch

The companion notebook (`cells.json`) (i) certifies ε–δ continuity for $f(x) = x^2$ at $a = 2$ by binary-search on δ for $\varepsilon \in \{10^{-1}, 10^{-2}, 10^{-3}\}$; (ii) plots the $O(h)$ error of the forward-difference approximation to $\sin'$; (iii) verifies the chain rule for $f(x) = \sin(x^2)$ at $x = 1$ to $\sim 10^{-7}$; (iv) finds an MVT witness $c \in (0, 2)$ for $f(x) = x^3 - 2x$ by bisection.

Connection to LLMs

A transformer is a composition $L_K \circ L_{K-1} \circ \cdots \circ L_1$ of differentiable layers. The gradient of the loss with respect to any parameter θ in layer i is, by iterated application of Theorem 0.242,

$$\frac{\partial \mathcal{L}}{\partial \theta} = \frac{\partial \mathcal{L}}{\partial L_K} \cdot \frac{\partial L_K}{\partial L_{K-1}} \cdots \frac{\partial L_{i+1}}{\partial L_i} \cdot \frac{\partial L_i}{\partial \theta}.$$

Backpropagation is precisely the right-to-left evaluation of this product (Chapter 18). The MVT, in turn, underwrites convergence proofs for gradient descent (Chapter 14): it bounds $|f(x) - f(y)| \le \sup |f'| \cdot |x - y|$, the Lipschitz hypothesis under which descent provably decreases the loss. Without the chain rule, deep learning does not exist.

Chapter 4: Multivariate calculus: partials, gradients, Jacobians

Motivation

A neural network is a function $F : \mathbb{R}^n \to \mathbb{R}^m$ built by composing many simple maps; training it requires the gradient of a scalar loss with respect to a high-dimensional parameter vector. Chapter 3 handled the one-dimensional theory: existence of $f'(a)$, the mean value theorem (MVT), and the single-variable chain rule through the Carathéodory factorization $f(x) - f(a) = \varphi(x)(x - a)$ with φ continuous at a. We now lift these ideas to several variables. Two pitfalls: (i) the *correct* notion of differentiability is *not* "all partials exist" but the stronger Fréchet condition, and (ii) the chain rule becomes a matrix product—the operation iterated by backpropagation in Chapter 18.

Definitions

Let $f : U \to \mathbb{R}^m$ with $U \subseteq \mathbb{R}^n$ open, $\mathbf{a} \in U$, and $\mathbf{e}_i$ the i-th standard basis vector.

Definition 0.28 (Partial derivative)**.** $\dfrac{\partial f}{\partial x_i}(\mathbf{a}) := \lim_{h \to 0} \dfrac{f(\mathbf{a} + h\mathbf{e}_i) - f(\mathbf{a})}{h}$*, when the limit exists.*

Definition 0.29 (Directional derivative)**.** *For a unit vector* $\mathbf{v}$*,* $D_{\mathbf{v}} f(\mathbf{a}) := \lim_{t\to 0} \dfrac{f(\mathbf{a}+t\mathbf{v}) - f(\mathbf{a})}{t}$.

Definition 0.30 (Fréchet differentiability)**.** f *is* differentiable *at* $\mathbf{a}$ *if there exists a linear* $L : \mathbb{R}^n \to \mathbb{R}^m$ *with*

$$\lim_{\mathbf{h}\to\mathbf{0}} \frac{\|f(\mathbf{a}+\mathbf{h}) - f(\mathbf{a}) - L\mathbf{h}\|}{\|\mathbf{h}\|} = 0,$$

i.e. the remainder is $o(\|\mathbf{h}\|)$*. The matrix of* L *in the standard basis is the* Jacobian $J_f(\mathbf{a}) \in \mathbb{R}^{m\times n}$*. When* $m = 1$*, the* gradient *is* $\nabla f(\mathbf{a}) := J_f(\mathbf{a})^\top$*.*

Remark 0.31. Differentiability is strictly stronger than existence of partials: $f(x,y) = xy/(x^2+y^2)$ (with $f(0,0) = 0$) has both partials zero at the origin, yet f is discontinuous there and so not Fréchet differentiable.

Theorems and proofs

Theorem 0.32 (Differentiable implies partials exist; $L = J_f$)**.** *If* f *is differentiable at* $\mathbf{a}$ *with derivative* L*, then* $\partial f/\partial x_j(\mathbf{a})$ *exists for every* j *and equals the* j*-th column of the matrix of* L*.*

Proof. Take $\mathbf{h} = h\mathbf{e}_j$, $h \neq 0$. Differentiability yields

$$f(\mathbf{a}+h\mathbf{e}_j) - f(\mathbf{a}) = h\,L\mathbf{e}_j + r(h), \qquad \|r(h)\|/|h| \to 0.$$

Divide by h and let $h \to 0$: the left side tends to $\partial f/\partial x_j(\mathbf{a})$ by Definition 1, the right side to $L\mathbf{e}_j$. Hence the partial exists and equals the j-th column of L. □

Theorem 0.33 (Continuous partials imply differentiability)**.** *If all* $\partial f/\partial x_j$ *exist on a neighborhood of* $\mathbf{a}$ *and are continuous at* $\mathbf{a}$*, then* f *is differentiable at* $\mathbf{a}$*.*

Sketch. Reduce to scalar codomain componentwise. Telescope along axes:

$$f(\mathbf{a}+\mathbf{h}) - f(\mathbf{a}) = \sum_{j=1}^{n} \Big[f(\mathbf{a}+\mathbf{h}_{<j} + h_j\mathbf{e}_j) - f(\mathbf{a}+\mathbf{h}_{<j}) \Big], \qquad \mathbf{h}_{<j} := \sum_{i<j} h_i\mathbf{e}_i.$$

The single-variable MVT (Chapter 3) applied to the j-th summand produces ξ_j between 0 and h_j with

$$f(\mathbf{a}+\mathbf{h}_{<j} + h_j\mathbf{e}_j) - f(\mathbf{a}+\mathbf{h}_{<j}) = h_j\,\partial_j f(\mathbf{a}+\mathbf{h}_{<j} + \xi_j\mathbf{e}_j).$$

Subtract $\sum_j h_j\,\partial_j f(\mathbf{a})$. By continuity of each $\partial_j f$, the difference $|\partial_j f(\cdot) - \partial_j f(\mathbf{a})| \to 0$ as $\mathbf{h} \to \mathbf{0}$. Cauchy–Schwarz then bounds the remainder by $\|\mathbf{h}\|\cdot\varepsilon(\mathbf{h})$ with $\varepsilon \to 0$, i.e. $o(\|\mathbf{h}\|)$. (Spivak, *Calculus on Manifolds*, Thm. 2-8.) □

Theorem 0.34 (Multivariate chain rule)**.** *If* $g : \mathbb{R}^k \to \mathbb{R}^n$ *is differentiable at* $\mathbf{a}$ *and* $f : \mathbb{R}^n \to \mathbb{R}^m$ *is differentiable at* $\mathbf{b} = g(\mathbf{a})$*, then* $f \circ g$ *is differentiable at* $\mathbf{a}$ *with*

$$J_{f\circ g}(\mathbf{a}) = J_f(\mathbf{b})\,J_g(\mathbf{a}).$$

Proof. Write the Carathéodory-style remainders $\rho_g(\mathbf{h}) := g(\mathbf{a}+\mathbf{h}) - g(\mathbf{a}) - J_g(\mathbf{a})\mathbf{h} = o(\|\mathbf{h}\|)$, and $\rho_f(\mathbf{k}) = o(\|\mathbf{k}\|)$. Let $\mathbf{k}(\mathbf{h}) := g(\mathbf{a}+\mathbf{h}) - g(\mathbf{a})$. Then

$$\begin{aligned} f(g(\mathbf{a}+\mathbf{h})) - f(g(\mathbf{a})) &= J_f(\mathbf{b})\,\mathbf{k}(\mathbf{h}) + \rho_f(\mathbf{k}(\mathbf{h})) \\ &= J_f(\mathbf{b})J_g(\mathbf{a})\mathbf{h} + J_f(\mathbf{b})\rho_g(\mathbf{h}) + \rho_f(\mathbf{k}(\mathbf{h})). \end{aligned}$$

Bound: $\|J_f(\mathbf{b})\rho_g(\mathbf{h})\| \le \|J_f(\mathbf{b})\|_{\mathrm{op}} \cdot o(\|\mathbf{h}\|) = o(\|\mathbf{h}\|)$. For the last term, $\|\mathbf{k}(\mathbf{h})\| \le (\|J_g(\mathbf{a})\|_{\mathrm{op}} + 1)\|\mathbf{h}\|$ for small $\|\mathbf{h}\|$, hence $\rho_f(\mathbf{k}(\mathbf{h})) = o(\|\mathbf{k}\|) = o(\|\mathbf{h}\|)$. The total remainder is $o(\|\mathbf{h}\|)$, so $f\circ g$ is differentiable with derivative $J_f(\mathbf{b})J_g(\mathbf{a})$. □

Theorem 0.35 (Schwarz / Clairaut)**.** *If $f : U \to \mathbb{R}$ is C^2 on U open and $\mathbf{a} \in U$, then for all i, j*

$$\frac{\partial^2 f}{\partial x_i \, \partial x_j}(\mathbf{a}) = \frac{\partial^2 f}{\partial x_j \, \partial x_i}(\mathbf{a}).$$

Proof. Fix $i \neq j$ and restrict to the two-variable slice in coordinates x_i, x_j (other coordinates frozen at the values of $\mathbf{a}$). Define the second difference

$$\Delta(h, k) := f(\mathbf{a} + h\mathbf{e}_i + k\mathbf{e}_j) - f(\mathbf{a} + h\mathbf{e}_i) - f(\mathbf{a} + k\mathbf{e}_j) + f(\mathbf{a}).$$

Let $\varphi(s) := f(\mathbf{a} + s\mathbf{e}_i + k\mathbf{e}_j) - f(\mathbf{a} + s\mathbf{e}_i)$, so $\Delta(h, k) = \varphi(h) - \varphi(0)$. By MVT in s: $\Delta = h\,\varphi'(\xi)$ for some $\xi \in (0, h)$, with

$$\varphi'(\xi) = \partial_i f(\mathbf{a} + \xi\mathbf{e}_i + k\mathbf{e}_j) - \partial_i f(\mathbf{a} + \xi\mathbf{e}_i).$$

Apply MVT again, this time in k: $\varphi'(\xi) = k\,\partial_j \partial_i f(\mathbf{a} + \xi\mathbf{e}_i + \eta\mathbf{e}_j)$ for some $\eta \in (0, k)$. Hence

$$\Delta(h, k) = hk\,\partial_j \partial_i f(\mathbf{a} + \xi\mathbf{e}_i + \eta\mathbf{e}_j).$$

By the symmetric argument starting with $\psi(t) := f(\mathbf{a}+h\mathbf{e}_i+t\mathbf{e}_j)-f(\mathbf{a}+t\mathbf{e}_j)$, $\Delta(h, k) = hk\,\partial_i \partial_j f(\mathbf{a}+\xi'\mathbf{e}_i + \eta'\mathbf{e}_j)$. Divide by hk and let $(h, k) \to (0, 0)$. Continuity of the second partials (C^2) makes both sides converge to $\partial_j \partial_i f(\mathbf{a})$ and $\partial_i \partial_j f(\mathbf{a})$ respectively, which are therefore equal. □

Code sketch

The accompanying notebook verifies every theorem numerically: gradient of $f(x, y) = x^2 y + \sin(x + y)$ via central differences; Jacobian of $\mathbf{f}(x, y, z) = (xyz,\ x^2 + \sin y + e^z)$ column-by-column; chain rule on $f \circ g$ with $g(t) = (\cos t, \sin t)$, $f(x, y) = x^2 + y^2$ (composite identically 1, so the chain-rule product must vanish); equality of mixed partials for $f(x, y) = x^3 y^2 + \sin(xy)$.

Connection to LLMs

A transformer is a composition $F = F_L \circ \cdots \circ F_1$ of differentiable layers, scored by a scalar loss $\mathcal{L} = \ell \circ F$. Theorem 4.7 states

$$\nabla_{\theta_\ell} \mathcal{L} = \left(J_\ell(F(x))\, J_{F_L} \cdots J_{F_{\ell+1}}\right) \frac{\partial F_\ell}{\partial \theta_\ell}.$$

Backpropagation never materializes any J_{F_k}. It propagates the row vector $\mathbf{v}^\top := J_\ell\, J_{F_L} \cdots J_{F_{\ell+1}}$ right-to-left via *vector–Jacobian products* (VJPs)—one per layer, each at O(forward cost). Reverse-mode autodiff is precisely Theorem 4.7 with associativity exploited; we derive the algorithm formally in Chapter 18.

Chapter 5: Linear algebra I: vector spaces, basis, linear maps

Motivation

Every transformer layer is, between its bias terms and nonlinearities, a *linear map between finite-dimensional real vector spaces.* The "hidden dimension" d is just $\dim \mathbb{R}^d$; a "weight matrix" $W \in \mathbb{R}^{m \times n}$ is the matrix representation of a linear map $\mathbb{R}^n \to \mathbb{R}^m$; "residual connections" are addition in a vector space; "low-rank adapters" are statements about the image of a linear map. Before attention (Chapter 21) or embeddings (Chapter 19), we need the grammar of vector spaces, bases, dimension, kernels, images, and the rank–nullity theorem. We use Chapter 1 (sets, functions, logic) throughout.

Definitions

Definition 0.36 (Field)**.** *A field $\mathbb{F}$ is a set with operations $+, \cdot$ such that $(\mathbb{F}, +)$ is an abelian group with identity 0, $(\mathbb{F} \setminus \{0\}, \cdot)$ is an abelian group with identity 1, and multiplication distributes over addition. The canonical example is $\mathbb{F} = \mathbb{R}$.*

Definition 0.37 (Vector space, eight axioms)**.** *A* vector space *V over $\mathbb{F}$ is a set with addition $+\colon V \times V \to V$ and scalar multiplication $\cdot\colon \mathbb{F} \times V \to V$ satisfying, for all $u, v, w \in V$ and $a, b \in \mathbb{F}$:*

1. *$(u + v) + w = u + (v + w)$;*
2. *$u + v = v + u$;*
3. *$\exists\, 0 \in V$ with $v + 0 = v$ for all v;*
4. *$\forall v\, \exists (-v)$ with $v + (-v) = 0$;*
5. *$a \cdot (u + v) = a \cdot u + a \cdot v$;*
6. *$(a + b) \cdot v = a \cdot v + b \cdot v$;*
7. *$(ab) \cdot v = a \cdot (b \cdot v)$;*
8. *$1 \cdot v = v$.*

Definition 0.38 (Subspace, span, independence, basis, dimension)**.** *A* subspace *$U \subset V$ is a subset closed under $+$ and scalar multiplication that contains 0. A* linear combination *of $v_1, \dots, v_k$ is $\sum a_i v_i$ with $a_i \in \mathbb{F}$. The* span *of $S \subset V$ is $\operatorname{span}(S) := \{\sum a_i v_i : v_i \in S, a_i \in \mathbb{F}\}$, the smallest subspace containing S. A finite set $\{v_1, \dots, v_k\}$ is* linearly independent *iff $\sum a_i v_i = 0 \Rightarrow a_1 = \cdots = a_k = 0$. A* basis *of V is a linearly independent spanning set; the* dimension *$\dim V$ is its cardinality (well-defined by Theorem 0.192 below).*

Definition 0.39 (Linear map, kernel, image, matrix representation)**.** *A map $T\colon V \to W$ between vector spaces over the same $\mathbb{F}$ is* linear *iff $T(au + bv) = aT(u) + bT(v)$. Define $\ker T := \{v \in V : Tv = 0\}$ and $\operatorname{im} T := \{Tv : v \in V\}$; both are subspaces. Given bases (e_j) of $\mathbb{R}^n$ and (f_i) of $\mathbb{R}^m$, the* matrix representation *of T is $A \in \mathbb{R}^{m \times n}$ with $T(e_j) = \sum_i A_{ij} f_i$.*

Theorems and proofs

Lemma 0.40 (Steinitz exchange)**.** *Let V be a vector space over $\mathbb{F}$. If $\{v_1, \dots, v_m\}$ is linearly independent and $\{w_1, \dots, w_n\}$ spans V, then $m \le n$, and (after reindexing the w_j) we may replace m of them by $v_1, \dots, v_m$ so that the resulting set still spans V.*

Proof. By induction on m. The case $m = 0$ is trivial. Assume the claim for $m-1$: after reindexing, $\{v_1, \dots, v_{m-1}, w_m, \dots, w_n\}$ spans V. In particular $m - 1 \le n$. Then

$$v_m = \sum_{i<m} a_i v_i + \sum_{j \ge m} b_j w_j$$

for some scalars. If all $b_j = 0$, then $v_m \in \operatorname{span}(v_1, \dots, v_{m-1})$, contradicting linear independence of $\{v_1, \dots, v_m\}$. Hence some $b_{j_0} \neq 0$ for $j_0 \in \{m, \dots, n\}$, forcing $n \ge m$. Reindex so $j_0 = m$ and solve:

$$w_m = b_m^{-1}\Big(v_m - \sum_{i<m} a_i v_i - \sum_{j>m} b_j w_j\Big) \in \operatorname{span}(v_1, \dots, v_m, w_{m+1}, \dots, w_n).$$

Therefore $\operatorname{span}(v_1, \dots, v_m, w_{m+1}, \dots, w_n)$ contains $\{v_1, \dots, v_{m-1}, w_m, \dots, w_n\}$, which spans V, so the new set spans V. □

Theorem 0.41 (Spanning sets contain bases; independent sets extend to bases)**.** *In a finite-dimensional V: (a) any finite spanning set contains a basis; (b) any linearly independent set extends to a basis.*

Proof. (a) If a spanning set S is dependent, some $w \in S$ lies in $\operatorname{span}(S \setminus \{w\})$, so $S \setminus \{w\}$ still spans. Iterate; finiteness terminates the process at an independent spanning set.
(b) Given an independent set L and a finite spanning set S, apply Lemma 0.40 to replace $|L|$ of the w's by L, producing a spanning set containing L. Then apply (a), removing only S-side elements. □

Theorem 0.42 (Invariance of dimension)**.** *Any two bases of a finite-dimensional V have the same cardinality.*

Proof. Let $\mathcal{B}_1, \mathcal{B}_2$ be bases with $|\mathcal{B}_1| = m$, $|\mathcal{B}_2| = n$. Since $\mathcal{B}_1$ is independent and $\mathcal{B}_2$ spans, Lemma 0.40 gives $m \le n$. Swap roles: $n \le m$. Hence $m = n$. □

Theorem 0.43 (Rank–nullity)**.** *Let $T\colon V \to W$ be linear with $\dim V = n < \infty$. Then*

$$\dim \ker T + \dim \operatorname{im} T \;=\; n.$$

Proof. Let $(u_1, \dots, u_k)$ be a basis of $\ker T$. By Theorem 0.41(b), extend to a basis $(u_1, \dots, u_k, v_1, \dots, v_{n-k})$ of V. We claim $(Tv_1, \dots, Tv_{n-k})$ is a basis of $\operatorname{im} T$.

Spanning. Any $w \in \operatorname{im} T$ has $w = T(\sum a_i u_i + \sum b_j v_j) = \sum b_j T v_j$ since $Tu_i = 0$.

Independence. If $\sum c_j T v_j = 0$, then $T(\sum c_j v_j) = 0$, so $\sum c_j v_j \in \ker T = \operatorname{span}(u_i)$. Write $\sum c_j v_j = \sum d_i u_i$, i.e. $\sum c_j v_j - \sum d_i u_i = 0$. Independence of the full basis forces all $c_j = 0$.

Hence $\dim \operatorname{im} T = n - k = n - \dim \ker T$. □

Code sketch

We will (a) implement `is_linearly_independent` and `dim_span` via `numpy.linalg.matrix_rank`; (b) build a 4×6 random integer matrix, compute rank and a basis of $\ker A$ via the right singular vectors of A corresponding to zero singular values, verifying $Av = 0$; (c) verify the change-of-basis formula $Av = P(P^{-1}AP)(P^{-1}v)$ for a random invertible P; (d) confirm $\operatorname{rank}(A) + \dim \ker A = 6$.

Connection to LLMs

A transformer with hidden dimension d operates on the vector space $\mathbb{R}^d$. The query/key/value projections in attention and the up/down projections in the MLP are linear maps $\mathbb{R}^d \to \mathbb{R}^{d_k}$ or $\mathbb{R}^d \to \mathbb{R}^{d_{\text{ff}}}$ (Chapter 21). The token embedding (Chapter 19) is a linear map $\mathbb{R}^{|\mathcal{V}|} \to \mathbb{R}^d$ applied to a one-hot input, i.e. a row lookup. The "rank" of an attention matrix and the "intrinsic dimension" of activations are claims about dim im. LoRA fine-tuning constrains weight updates to a low-dimensional subspace — a direct use of $\dim \operatorname{im} T \le \min(m, n)$. Rank–nullity returns when we count free parameters and degrees of freedom.

Chapter 6: Linear algebra II: inner products, norms, eigenvalues, SVD

Motivation

Chapter 5 built vector spaces and linear maps as raw algebraic objects. To do *geometry* — length, angle, orthogonality, best approximation — we add a single piece of structure: an *inner product*.

From it we will derive the Cauchy–Schwarz inequality, the triangle inequality, the spectral theorem for symmetric matrices, the singular value decomposition (SVD), and the Eckart–Young low-rank approximation theorem. These are the load-bearing tools behind PCA, attention, and LoRA fine-tuning of LLMs.

Definitions

Definition 0.44 (Inner product)**.** *A function $\langle \cdot, \cdot \rangle : V \times V \to \mathbb{R}$ on a real vector space V is an* inner product *if for all $\mathbf{x}, \mathbf{y}, \mathbf{z} \in V$ and $a, b \in \mathbb{R}$:*

1. *Symmetry: $\langle \mathbf{x}, \mathbf{y} \rangle = \langle \mathbf{y}, \mathbf{x} \rangle$.*
2. *Linearity in the first argument: $\langle a\mathbf{x} + b\mathbf{y}, \mathbf{z} \rangle = a\langle \mathbf{x}, \mathbf{z} \rangle + b\langle \mathbf{y}, \mathbf{z} \rangle$.*
3. *Positive-definiteness: $\langle \mathbf{x}, \mathbf{x} \rangle \geq 0$ with equality iff $\mathbf{x} = \mathbf{0}$.*

The canonical example on $\mathbb{R}^n$ is the dot product $\langle \mathbf{x}, \mathbf{y} \rangle = \sum_i x_i y_i = \mathbf{x}^\top \mathbf{y}$.

Definition 0.45 (Induced norm)**.** $\|\mathbf{x}\| := \sqrt{\langle \mathbf{x}, \mathbf{x} \rangle}$.

Definition 0.46 (Orthogonality)**.** $\mathbf{x}, \mathbf{y}$ *are* orthogonal *if $\langle \mathbf{x}, \mathbf{y} \rangle = 0$. A set $\{\mathbf{q}_i\}$ is* orthonormal *if $\langle \mathbf{q}_i, \mathbf{q}_j \rangle = \delta_{ij}$. An* orthonormal basis *is an orthonormal spanning set.*

Definition 0.47 (Eigenvalue, eigenvector, characteristic polynomial)**.** *For $A \in \mathbb{R}^{n \times n}$, $(\lambda, \mathbf{v})$ with $\mathbf{v} \neq \mathbf{0}$ is an* eigenpair *if $A\mathbf{v} = \lambda \mathbf{v}$. The* characteristic polynomial *is $p_A(\lambda) := \det(A - \lambda I)$; its roots are the eigenvalues.*

Definition 0.48 (Symmetric, positive (semi)definite)**.** *A is* symmetric *if $A^\top = A$;* positive semidefinite *(PSD) if symmetric with $\mathbf{x}^\top A \mathbf{x} \geq 0$ for all $\mathbf{x}$;* positive definite *if strict for $\mathbf{x} \neq \mathbf{0}$.*

Definition 0.49 (Singular value decomposition)**.** *A* singular value decomposition *of $A \in \mathbb{R}^{m \times n}$ is a factorization $A = U \Sigma V^\top$ where $U \in \mathbb{R}^{m \times m}$, $V \in \mathbb{R}^{n \times n}$ are orthogonal ($U^\top U = I$, $V^\top V = I$) and $\Sigma \in \mathbb{R}^{m \times n}$ is "diagonal" with nonnegative entries $\sigma_1 \geq \sigma_2 \geq \cdots \geq 0$.*

Theorems and Proofs

Theorem 0.50 (Cauchy–Schwarz)**.** *For all $\mathbf{x}, \mathbf{y} \in V$, $|\langle \mathbf{x}, \mathbf{y} \rangle| \leq \|\mathbf{x}\| \, \|\mathbf{y}\|$.*

Proof. If $\mathbf{y} = \mathbf{0}$, both sides vanish. Otherwise, for every $t \in \mathbb{R}$,

$$0 \leq \|\mathbf{x} - t\mathbf{y}\|^2 = \langle \mathbf{x} - t\mathbf{y}, \mathbf{x} - t\mathbf{y} \rangle = \|\mathbf{x}\|^2 - 2t\langle \mathbf{x}, \mathbf{y} \rangle + t^2 \|\mathbf{y}\|^2.$$

This is a quadratic in t that is nonnegative everywhere, so its discriminant satisfies

$$(2\langle \mathbf{x}, \mathbf{y} \rangle)^2 - 4\|\mathbf{x}\|^2 \|\mathbf{y}\|^2 \leq 0,$$

i.e. $\langle \mathbf{x}, \mathbf{y} \rangle^2 \leq \|\mathbf{x}\|^2 \|\mathbf{y}\|^2$. Taking square roots completes the proof. □

Corollary 0.51 (Triangle inequality)**.** $\|\mathbf{x} + \mathbf{y}\| \leq \|\mathbf{x}\| + \|\mathbf{y}\|$.

Proof. Expand and apply Cauchy–Schwarz:

$$\|\mathbf{x} + \mathbf{y}\|^2 = \|\mathbf{x}\|^2 + 2\langle \mathbf{x}, \mathbf{y} \rangle + \|\mathbf{y}\|^2 \leq \|\mathbf{x}\|^2 + 2\|\mathbf{x}\|\|\mathbf{y}\| + \|\mathbf{y}\|^2 = (\|\mathbf{x}\| + \|\mathbf{y}\|)^2.$$ □

Theorem 0.52 (Spectral theorem, real symmetric case)**.** *Every symmetric $A \in \mathbb{R}^{n\times n}$ admits a factorization $A = Q\Lambda Q^\top$ with Q orthogonal and Λ real diagonal.*

Proof. Induct on n. For $n = 1$, trivial. Assume the result for $n-1$. The Rayleigh quotient $R(\mathbf{x}) = \mathbf{x}^\top A\mathbf{x}$ is continuous on the unit sphere $S^{n-1} = \{\mathbf{x} : \|\mathbf{x}\| = 1\}$, which is compact, so R attains a maximum at some $\mathbf{v}_1 \in S^{n-1}$ with value λ_1. By Lagrange multipliers (or by directly differentiating $R(\mathbf{v}_1 + t\mathbf{w})$ along any tangent $\mathbf{w} \perp \mathbf{v}_1$ at $t = 0$), $A\mathbf{v}_1 = \lambda_1\mathbf{v}_1$. The eigenvalue is real because $\lambda_1 = \mathbf{v}_1^\top A\mathbf{v}_1 \in \mathbb{R}$ and $\mathbf{v}_1 \in \mathbb{R}^n$. (Alternatively, if $A\mathbf{z} = \mu\mathbf{z}$ over $\mathbb{C}$ with $\mathbf{z} \neq \mathbf{0}$, then $\mu\,\overline{\mathbf{z}}^\top\mathbf{z} = \overline{\mathbf{z}}^\top A\mathbf{z} = (A\overline{\mathbf{z}})^\top\mathbf{z} = \overline{\mu}\,\overline{\mathbf{z}}^\top\mathbf{z}$ since A is real symmetric, so $\mu = \overline{\mu}$.)

Let $W = \{\mathbf{v}_1\}^\perp$, an $(n-1)$-dimensional subspace. For $\mathbf{w} \in W$,

$$\langle A\mathbf{w}, \mathbf{v}_1\rangle = \langle \mathbf{w}, A\mathbf{v}_1\rangle = \lambda_1\langle \mathbf{w}, \mathbf{v}_1\rangle = 0,$$

so A maps $W \to W$. The restriction $A|_W$ is symmetric with respect to the inherited inner product. By induction there is an orthonormal eigenbasis $\mathbf{v}_2, \dots, \mathbf{v}_n$ of W with eigenvalues $\lambda_2, \dots, \lambda_n$. Then $\mathbf{v}_1, \dots, \mathbf{v}_n$ is an orthonormal eigenbasis of A. Setting $Q = [\mathbf{v}_1 \ \cdots \ \mathbf{v}_n]$ and $\Lambda = \operatorname{diag}(\lambda_1, \dots, \lambda_n)$ gives $A = Q\Lambda Q^\top$. □

Theorem 0.53 (Existence of SVD)**.** *Every $A \in \mathbb{R}^{m\times n}$ admits an SVD $A = U\Sigma V^\top$.*

Sketch. The matrix $A^\top A \in \mathbb{R}^{n\times n}$ is symmetric and PSD, since $\mathbf{x}^\top A^\top A\mathbf{x} = \|A\mathbf{x}\|^2 \geq 0$. By the spectral theorem $A^\top A = V\Lambda V^\top$ with V orthogonal and $\Lambda = \operatorname{diag}(\lambda_1, \dots, \lambda_n)$, $\lambda_i \geq 0$, sorted decreasingly. Set $\sigma_i := \sqrt{\lambda_i}$. For each i with $\sigma_i > 0$ define $\mathbf{u}_i := A\mathbf{v}_i/\sigma_i \in \mathbb{R}^m$; then

$$\langle \mathbf{u}_i, \mathbf{u}_j\rangle = \frac{\mathbf{v}_i^\top A^\top A\mathbf{v}_j}{\sigma_i\sigma_j} = \frac{\lambda_j\delta_{ij}}{\sigma_i\sigma_j} = \delta_{ij},$$

so the $\mathbf{u}_i$ are orthonormal in $\mathbb{R}^m$. Extend to an orthonormal basis to form U. By construction $AV = U\Sigma$, hence $A = U\Sigma V^\top$. □

Theorem 0.54 (Eckart–Young)**.** *Let $A = U\Sigma V^\top$ have singular values $\sigma_1 \geq \cdots \geq \sigma_r > 0$. The truncated SVD $A_k := \sum_{i=1}^{k} \sigma_i\mathbf{u}_i\mathbf{v}_i^\top$ minimises $\|A - B\|_F$ (and $\|A - B\|_2$) over rank-k matrices B, with $\|A - A_k\|_F^2 = \sum_{i>k}\sigma_i^2$.*

Sketch. The Frobenius norm is unitarily invariant: $\|A - B\|_F = \|U^\top(A - B)V\|_F$. The problem reduces to: among rank-k matrices $C \in \mathbb{R}^{m\times n}$, minimise

$$\|\Sigma - C\|_F^2 = \sum_i(\sigma_i - c_{ii})^2 + \sum_{i\neq j} c_{ij}^2.$$

Off-diagonal entries should be zero, and the diagonal of C should equal σ_i for $i \leq k$ and 0 for $i > k$ (matching the largest gaps), recovering A_k. The operator-norm version uses the Courant–Fischer minimax characterisation of singular values. □

Code Sketch

The accompanying notebook (`cells.json`) (i) verifies Cauchy–Schwarz on 50 random pairs in $\mathbb{R}^{10}$, (ii) computes a symmetric eigendecomposition via `numpy.linalg.eigh` and checks $A\mathbf{v}_i = \lambda_i\mathbf{v}_i$ plus orthonormality, (iii) computes an SVD via `numpy.linalg.svd` and reconstructs A, and (iv) shows that Frobenius errors of rank-1 and rank-2 approximations decrease monotonically, consistent with Eckart–Young.

Connection to LLMs

Inner products and SVD are the geometric backbone of transformers.

- **Attention** (Chapters 21–22). The score matrix $QK^\top$ is the matrix of pairwise inner products of query and key embeddings. Linear-attention and Performer-style methods exploit the empirical low-rank-ness of $QK^\top$, which by Eckart–Young is best captured by truncated SVD.
- **PCA / interpretability.** SVD of an embedding matrix yields principal components; leading singular vectors expose semantic axes used in probing studies.
- **LoRA fine-tuning.** The update $W_0 + BA$ with $B \in \mathbb{R}^{m\times r}$, $A \in \mathbb{R}^{r\times n}$, $r \ll \min(m,n)$, is literally a rank-r correction; Eckart–Young guarantees its optimality at that rank in Frobenius norm.
- **Spectral norm regularisation.** Bounding $\sigma_1(W)$ controls Lipschitz constants and stabilises training.

Every later geometric statement in this book — projections, least squares, gradient flow on weight matrices, contractive maps — descends from the inequalities and decompositions proven in this chapter.

Chapter 7: Convexity and optimization; gradient descent convergence

Motivation

Chapters 3 and 4 built derivatives, the chain rule, and gradients; Chapter 6 equipped $\mathbb{R}^n$ with a norm and the Cauchy–Schwarz inequality. With those in hand we can finally ask the optimization question that drives every line of training code in this book: given $f : \mathbb{R}^n \to \mathbb{R}$, what does the iteration $x_{t+1} = x_t - \eta\nabla f(x_t)$ actually do, and when does it converge?

The honest answer for a transformer's loss is "we don't know": it is non-convex, the iterates are stochastic (Chapter 13), and rigorous global convergence is open. But the *convex* case admits complete proofs, and those proofs supply the only quantitative vocabulary we have for the non-convex one — smoothness, condition number, descent, contraction. Phrases like "smooth loss landscape," "warmup," and "$1/L$ step size" trace directly back to the two theorems below.

Definitions

Definition 0.55 (Convex set)**.** *$C \subseteq \mathbb{R}^n$ is* convex *iff for all $x, y \in C$ and $t \in [0,1]$, $tx+(1-t)y \in C$.*

Definition 0.56 (Convex function)**.** *$f : \mathbb{R}^n \to \mathbb{R}$ is* convex *iff*

$$f(tx + (1-t)y) \le tf(x) + (1-t)f(y) \quad \forall x, y \in \mathbb{R}^n,\ t \in [0,1].$$

When f is differentiable this is equivalent to the first-order *characterization $f(y) \ge f(x)+\langle\nabla f(x), y-x\rangle$: every tangent hyperplane lies below the graph.*

Definition 0.57 (L-smoothness)**.** *f is L-*smooth *iff ∇f is L-Lipschitz: $\|\nabla f(x)-\nabla f(y)\| \le L\|x-y\|$ for all x, y, with the Euclidean norm of Chapter 6.*

Definition 0.58 (μ-strong convexity)**.** *f is* μ-strongly convex *iff for all x, y,*

$$f(y) \geq f(x) + \langle \nabla f(x), y - x \rangle + \tfrac{\mu}{2}\|y - x\|^2.$$

The ratio $\kappa = L/\mu \geq 1$ is the condition number*.*

Definition 0.59 (Gradient descent)**.** *Fix $x_0 \in \mathbb{R}^n$, step size $\eta > 0$. The* gradient descent *iterates are $x_{t+1} = x_t - \eta \nabla f(x_t)$ for $t = 0, 1, 2, \ldots$.*

Theorems and proofs

Lemma 0.60 (Descent lemma)**.** *If f is L-smooth, then for all $x, y \in \mathbb{R}^n$,*

$$f(y) \leq f(x) + \langle \nabla f(x), y - x \rangle + \tfrac{L}{2}\|y - x\|^2.$$

Proof. Let $g(t) = f(x + t(y - x))$ on $[0, 1]$. By the chain rule (Chapter 4), $g'(t) = \langle \nabla f(x + t(y - x)), y - x \rangle$, and the fundamental theorem of calculus gives

$$f(y) - f(x) = g(1) - g(0) = \int_0^1 \langle \nabla f(x + t(y - x)),\, y - x \rangle \, dt.$$

Subtract $\langle \nabla f(x), y - x \rangle$ from both sides:

$$f(y) - f(x) - \langle \nabla f(x), y - x \rangle = \int_0^1 \langle \nabla f(x + t(y - x)) - \nabla f(x),\, y - x \rangle \, dt.$$

By Cauchy–Schwarz (Chapter 6) and the Lipschitz hypothesis,

$$\langle \nabla f(x + t(y - x)) - \nabla f(x),\, y - x \rangle \leq \|\nabla f(x + t(y - x)) - \nabla f(x)\| \, \|y - x\| \leq Lt\|y - x\|^2.$$

Integrating, $\int_0^1 Lt\|y - x\|^2 \, dt = \frac{L}{2}\|y - x\|^2$. □

Theorem 0.61 (GD on L-smooth convex)**.** *Let f be convex and L-smooth with minimizer x^* and minimum $f^* = f(x^*)$. Then gradient descent with $\eta = 1/L$ satisfies*

$$f(x_T) - f^* \;\leq\; \frac{L\|x_0 - x^*\|^2}{2T}, \qquad T \geq 1.$$

Proof. Apply the descent lemma at $y = x_{t+1} = x_t - \frac{1}{L}\nabla f(x_t)$, $x = x_t$:

$$f(x_{t+1}) \leq f(x_t) - \tfrac{1}{2L}\|\nabla f(x_t)\|^2. \tag{$\star$}$$

In particular $f(x_t)$ is non-increasing in t. Convexity at x_t versus x^* gives $f(x_t) - f^* \leq \langle \nabla f(x_t), x_t - x^* \rangle$. Expand the gradient step:

$$\|x_{t+1} - x^*\|^2 = \|x_t - x^*\|^2 - \tfrac{2}{L}\langle \nabla f(x_t), x_t - x^* \rangle + \tfrac{1}{L^2}\|\nabla f(x_t)\|^2,$$

so

$$\tfrac{2}{L}\langle \nabla f(x_t), x_t - x^* \rangle = \|x_t - x^*\|^2 - \|x_{t+1} - x^*\|^2 + \tfrac{1}{L^2}\|\nabla f(x_t)\|^2,$$

hence

$$f(x_t) - f^* \leq \tfrac{L}{2}\big(\|x_t - x^*\|^2 - \|x_{t+1} - x^*\|^2\big) + \tfrac{1}{2L}\|\nabla f(x_t)\|^2.$$

Adding $(\star)$ rewritten as $\frac{1}{2L}\|\nabla f(x_t)\|^2 \le f(x_t) - f(x_{t+1})$ to the right-hand side and rearranging,

$$f(x_{t+1}) - f^* \le \tfrac{L}{2}\big(\|x_t - x^*\|^2 - \|x_{t+1} - x^*\|^2\big).$$

Telescope $t = 0, \dots, T-1$:

$$\sum_{t=0}^{T-1} \big(f(x_{t+1}) - f^*\big) \le \tfrac{L}{2}\|x_0 - x^*\|^2.$$

Since $f(x_t)$ is non-increasing, $T(f(x_T) - f^*) \le \sum_{t=0}^{T-1}(f(x_{t+1}) - f^*)$, which yields the claim. □

Theorem 0.62 (GD on L-smooth μ-strongly convex)**.** *Let f be μ-strongly convex and L-smooth, with $\eta = 1/L$. Then*

$$\|x_t - x^*\|^2 \le \big(1 - \mu/L\big)^t \|x_0 - x^*\|^2.$$

Proof. Strong convexity at x_t versus x^* rearranges to

$$\langle \nabla f(x_t), x_t - x^* \rangle \ge f(x_t) - f^* + \tfrac{\mu}{2}\|x_t - x^*\|^2,$$

and from $(\star)$ in the previous proof, $\frac{1}{2L}\|\nabla f(x_t)\|^2 \le f(x_t) - f(x_{t+1}) \le f(x_t) - f^*$, i.e. $\frac{1}{L^2}\|\nabla f(x_t)\|^2 \le \frac{2}{L}(f(x_t) - f^*)$. Substituting both into the gradient-step expansion,

$$\begin{aligned}
\|x_{t+1} - x^*\|^2 &= \|x_t - x^*\|^2 - \tfrac{2}{L}\langle \nabla f(x_t), x_t - x^* \rangle + \tfrac{1}{L^2}\|\nabla f(x_t)\|^2 \\
&\le \|x_t - x^*\|^2 - \tfrac{2}{L}\big(f(x_t) - f^* + \tfrac{\mu}{2}\|x_t - x^*\|^2\big) + \tfrac{2}{L}\big(f(x_t) - f^*\big) \\
&= \big(1 - \tfrac{\mu}{L}\big)\|x_t - x^*\|^2.
\end{aligned}$$

Iterate. □

Remark 0.63. The contraction factor $1 - 1/\kappa$ degrades as the condition number $\kappa = L/\mu$ grows: ill-conditioned problems converge slowly. The optimal step $\eta = 2/(L + \mu)$ improves the per-step factor to $((\kappa - 1)/(\kappa + 1))^2$, but does not change the $O(\kappa \log(1/\varepsilon))$ iteration complexity scaling.

Code sketch

Instantiate $f(x, y) = (x - 1)^2 + 2(y + 1)^2$, a strongly convex quadratic with Hessian $\mathrm{diag}(2, 4)$, so $\mu = 2$, $L = 4$. Gradient descent with $\eta = 1/L$ contracts $\|x_t - x^*\|^2$ by factor $1 - \mu/L = 1/2$ per step. For a degenerate $f(x) = \|Ax - b\|^2$ with $A \in \mathbb{R}^{20 \times 10}$ ill-conditioned, the strong-convexity rate degrades to the convex $O(1/T)$, visible as slope ≈ -1 on a log-log plot of $f(x_T) - f^*$ versus T. A side-by-side run with $\eta = 1/L$ and the optimal $\eta = 2/(L + \mu)$ confirms the sharper contraction predicted by the remark.

Connection to LLMs

Transformer losses are spectacularly non-convex: permuting attention heads or flipping signs in a layernorm gives an equivalent minimum. None of the theorems above apply globally. They apply *locally*, and they supply the operating intuitions that survive the jump.

- **Smooth loss landscape.** The descent lemma says $\eta < 2/L$ is sufficient for monotone decrease. Empirically estimating local L via the Hessian's top eigenvalue is exactly what learning-rate range tests and gradient-norm clipping approximate.

- **Warmup and LR scheduling** (Chapter 27). Early in training the local L is large (sharp minima nearby); a small η avoids the quadratic blow-up term in the descent lemma. As the trajectory enters flatter regions, L drops and η can grow.
- **SGD and AdamW** (Chapters 13, 14). Stochastic gradients break the deterministic descent lemma; the convergence proofs replace $(\star)$ with an expectation inequality plus a variance term, but the algebraic skeleton — descent lemma, telescoping, contraction — is preserved.

The convex theory is a load-bearing analogy: the only place where the rates are honest, and the conceptual scaffold for every heuristic layered on top.

Block B — Probability and Information

Chapter 8: Probability foundations: sample spaces, sigma-algebras, Kolmogorov axioms

Motivation

Every language model is a probability distribution over token sequences. To make rigorous sense of statements like "the model assigns probability p to token t given context c" we need a precise theory of *what* a probability is, *what* it acts on, and *what* consistency rules it must obey. Naive frequentism does not, on its own, justify additivity, well-definedness of conditioning, or continuity in limits. Kolmogorov's 1933 axiomatization solves this by reducing probability to measure theory on a σ-algebra. This chapter builds that machinery from the set-theoretic foundations of Chapter 1 up through inclusion–exclusion, the union bound, Bayes, the law of total probability, and continuity of measure.

Sample spaces, σ-algebras, probability measures

Definition 0.64 (Sample space, event)**.** *A* sample space *is a non-empty set Ω. Its elements $\omega \in \Omega$ are* outcomes*. An* event *is a subset $A \subseteq \Omega$.*

Definition 0.65 (σ-algebra)**.** *A collection $\mathcal{F} \subseteq 2^\Omega$ is a* σ-algebra *on Ω if*

1. *$\Omega \in \mathcal{F}$;*
2. *$A \in \mathcal{F} \Rightarrow A^c \in \mathcal{F}$;*
3. *$A_1, A_2, \ldots \in \mathcal{F} \Rightarrow \bigcup_{n=1}^{\infty} A_n \in \mathcal{F}$.*

Remark 0.66. By De Morgan's laws (Chapter 1), $\mathcal{F}$ is closed under countable intersections as well: $\bigcap_n A_n = (\bigcup_n A_n^c)^c$. The closure axioms mirror set algebra exactly, extended to the countably infinite operations probability needs.

Definition 0.67 (Probability measure, probability space)**.** *A function $\mathbb{P} : \mathcal{F} \to [0,1]$ is a* probability measure *if it satisfies the* Kolmogorov axioms*:*

(K1) *$\mathbb{P}(A) \geq 0$ for all $A \in \mathcal{F}$;*

(K2) *$\mathbb{P}(\Omega) = 1$;*

(K3) *for pairwise disjoint* $A_1, A_2, \ldots \in \mathcal{F}$*,*

$$\mathbb{P}\left(\bigsqcup_{n=1}^{\infty} A_n\right) = \sum_{n=1}^{\infty} \mathbb{P}(A_n).$$

The triple $(\Omega, \mathcal{F}, \mathbb{P})$ *is a* probability space*.*

Definition 0.68 (Conditional probability, independence)**.** *For* $B \in \mathcal{F}$ *with* $\mathbb{P}(B) > 0$*, the* conditional probability *of* A *given* B *is*

$$\mathbb{P}(A \mid B) := \frac{\mathbb{P}(A \cap B)}{\mathbb{P}(B)}.$$

Events A, B *are* independent *iff* $\mathbb{P}(A \cap B) = \mathbb{P}(A)\,\mathbb{P}(B)$*.*

Elementary consequences

Lemma 0.69 (Finite additivity, complement, monotonicity)**.** $\mathbb{P}(\emptyset) = 0$*. For disjoint* $A, B \in \mathcal{F}$*,* $\mathbb{P}(A \sqcup B) = \mathbb{P}(A) + \mathbb{P}(B)$*. For any* A*,* $\mathbb{P}(A^c) = 1 - \mathbb{P}(A)$*. If* $A \subseteq B$*, then* $\mathbb{P}(A) \le \mathbb{P}(B)$*.*

Proof. Apply (K3) to $\Omega, \emptyset, \emptyset, \ldots$ (disjoint, with union Ω) to get $1 = 1 + \sum_{n \ge 2} \mathbb{P}(\emptyset)$, so $\mathbb{P}(\emptyset) = 0$. For finite additivity, apply (K3) to $A, B, \emptyset, \emptyset, \ldots$. From $\Omega = A \sqcup A^c$ and (K2), $1 = \mathbb{P}(A) + \mathbb{P}(A^c)$. If $A \subseteq B$, then $B = A \sqcup (B \setminus A)$, so $\mathbb{P}(B) = \mathbb{P}(A) + \mathbb{P}(B \setminus A) \ge \mathbb{P}(A)$ by (K1). □

Theorem 0.70 (Inclusion–exclusion, two events)**.** *For all* $A, B \in \mathcal{F}$*,*

$$\mathbb{P}(A \cup B) = \mathbb{P}(A) + \mathbb{P}(B) - \mathbb{P}(A \cap B).$$

Proof. Decompose into three disjoint pieces:

$$A \cup B = (A \setminus B) \sqcup (B \setminus A) \sqcup (A \cap B), \quad A = (A \setminus B) \sqcup (A \cap B), \quad B = (B \setminus A) \sqcup (A \cap B).$$

By Lemma 0.69,

$$\mathbb{P}(A) + \mathbb{P}(B) = \mathbb{P}(A \setminus B) + \mathbb{P}(B \setminus A) + 2\,\mathbb{P}(A \cap B) = \mathbb{P}(A \cup B) + \mathbb{P}(A \cap B).$$ □

Theorem 0.71 (Union bound / Boole's inequality)**.** *For any countable family* $\{A_n\} \subseteq \mathcal{F}$*,*

$$\mathbb{P}\left(\bigcup_n A_n\right) \le \sum_n \mathbb{P}(A_n).$$

Proof. Disjointify: set $B_1 = A_1$ and $B_n = A_n \setminus \bigcup_{k<n} A_k$ for $n \ge 2$. Each $B_n \in \mathcal{F}$ (closure under complements and countable unions), the B_n are pairwise disjoint, $B_n \subseteq A_n$, and $\bigsqcup_n B_n = \bigcup_n A_n$. By (K3) and monotonicity,

$$\mathbb{P}\left(\bigcup_n A_n\right) = \sum_n \mathbb{P}(B_n) \le \sum_n \mathbb{P}(A_n).$$ □

Theorem 0.72 (Bayes)**.** *If* $\mathbb{P}(A), \mathbb{P}(B) > 0$*, then*

$$\mathbb{P}(A \mid B) = \frac{\mathbb{P}(B \mid A)\,\mathbb{P}(A)}{\mathbb{P}(B)}.$$

Proof. By definition, $\mathbb{P}(A \mid B)\,\mathbb{P}(B) = \mathbb{P}(A \cap B) = \mathbb{P}(B \cap A) = \mathbb{P}(B \mid A)\,\mathbb{P}(A)$. Divide by $\mathbb{P}(B)$. □

Theorem 0.73 (Law of total probability)**.** *If $\{B_i\}_{i \in I}$ is a countable partition of Ω with $\mathbb{P}(B_i) > 0$, then for every $A \in \mathcal{F}$,*

$$\mathbb{P}(A) = \sum_i \mathbb{P}(A \mid B_i)\,\mathbb{P}(B_i).$$

Proof. $A = A \cap \Omega = A \cap \bigsqcup_i B_i = \bigsqcup_i (A \cap B_i)$, a disjoint union. By (K3), $\mathbb{P}(A) = \sum_i \mathbb{P}(A \cap B_i) = \sum_i \mathbb{P}(A \mid B_i)\,\mathbb{P}(B_i)$. □

Theorem 0.74 (Continuity of measure)**.** *Let $A_n \in \mathcal{F}$.*

1. (From below) *If $A_1 \subseteq A_2 \subseteq \cdots$ and $A = \bigcup_n A_n$, then $\mathbb{P}(A_n) \uparrow \mathbb{P}(A)$.*
2. (From above) *If $A_1 \supseteq A_2 \supseteq \cdots$ and $A = \bigcap_n A_n$, then $\mathbb{P}(A_n) \downarrow \mathbb{P}(A)$.*

Proof. (Below.) Set $B_1 = A_1$, $B_n = A_n \setminus A_{n-1}$ for $n \geq 2$. The B_n are disjoint, $\bigsqcup_{k \leq n} B_k = A_n$, and $\bigsqcup_k B_k = A$. By (K3),

$$\mathbb{P}(A) = \sum_{k=1}^{\infty} \mathbb{P}(B_k) = \lim_{n \to \infty} \sum_{k=1}^{n} \mathbb{P}(B_k) = \lim_{n \to \infty} \mathbb{P}(A_n).$$

(Above.) The sequence $C_n := A_1 \setminus A_n$ is increasing with union $A_1 \setminus A$. By the previous case, $\mathbb{P}(A_1) - \mathbb{P}(A_n) = \mathbb{P}(C_n) \to \mathbb{P}(A_1 \setminus A) = \mathbb{P}(A_1) - \mathbb{P}(A)$. Since $\mathbb{P}(A_1) \leq 1 < \infty$, we may subtract: $\mathbb{P}(A_n) \to \mathbb{P}(A)$. □

Code sketch

The companion notebook instantiates $\Omega = \{1, \ldots, 6\}^2$ with $\mathcal{F} = 2^{\Omega}$ and uniform $\mathbb{P}(A) = |A|/36$. It verifies (K2), finite additivity for two disjoint events, two-event inclusion–exclusion, the union bound on three events, an independence vs. non-independence comparison, and a Bayes computation for disease testing checked both analytically and by Monte Carlo.

Connection to LLMs

A causal language model (Chapter 25, forward reference) defines, for each context c, a probability measure on the finite sample space $\Omega = \mathcal{V}$ (the vocabulary), with $\mathcal{F} = 2^{\mathcal{V}}$ and $\mathbb{P}(\{t\} \mid c) = p_\theta(t \mid c)$. The softmax output is precisely a Kolmogorov probability measure: non-negative, normalized, additive on disjoint subsets of tokens. The factorization

$$\mathbb{P}(t_1, \ldots, t_n) = \prod_{i=1}^{n} \mathbb{P}(t_i \mid t_{<i})$$

is iterated conditioning. Sampling, top-k truncation, nucleus filtering, beam search, and importance-weighted training reduce to operations on this measure. Bayes' theorem reappears whenever we invert a generative model into a posterior over latents (alignment, reward modeling). Continuity of measure licenses limiting arguments needed for safety claims about infinite or arbitrarily long generations.

Chapter 9: Random variables, distributions, CDF/PMF/PDF

Motivation

Chapter 8 built probability spaces $(\Omega, \mathcal{F}, \mathbb{P})$ as the bedrock for modeling uncertainty. But raw Ω is rarely what we *measure*: we measure a temperature, a token id, a pixel intensity. A *random variable* is the bridge that turns abstract outcomes into real numbers we can sum, integrate, optimize, and—most importantly—differentiate. Every loss function in deep learning is an expectation over random variables; every sampler in a language model is a draw from a distribution; every softmax is a categorical PMF. This chapter formalizes the machinery.

Definitions

Definition 0.75 (Random variable)**.** *Let $(\Omega, \mathcal{F}, \mathbb{P})$ be a probability space. A function $X : \Omega \to \mathbb{R}$ is a* random variable *(RV) if for every $x \in \mathbb{R}$,*

$$\{X \le x\} := \{\omega \in \Omega : X(\omega) \le x\} \in \mathcal{F}.$$

Equivalently, $X^{-1}(B) \in \mathcal{F}$ for every Borel $B \in \mathcal{B}(\mathbb{R})$.

Definition 0.76 (Distribution)**.** *The* distribution *(or* law*) of X is the pushforward measure μ_X on $(\mathbb{R}, \mathcal{B}(\mathbb{R}))$,*

$$\mu_X(B) = \mathbb{P}(X \in B), \qquad B \in \mathcal{B}(\mathbb{R}).$$

Definition 0.77 (CDF)**.** *The* cumulative distribution function *of X is $F_X : \mathbb{R} \to [0, 1]$,*

$$F_X(x) = \mathbb{P}(X \le x) = \mu_X((-\infty, x]).$$

Definition 0.78 (Discrete RV, PMF)**.** *X is* discrete *if it takes values in a countable set $S \subset \mathbb{R}$. The* probability mass function *is*

$$p_X(x) = \mathbb{P}(X = x), \quad x \in S.$$

Definition 0.79 (Continuous RV, PDF)**.** *X is* (absolutely) continuous *if there exists a non-negative measurable $f_X : \mathbb{R} \to [0, \infty)$, the* probability density function*, with*

$$\mathbb{P}(X \in A) = \int_A f_X(x)\, dx \quad \text{for every Borel } A \subseteq \mathbb{R}.$$

Example 0.80 (Standard families).

- Bernoulli(p): $p_X(1) = p,\ p_X(0) = 1 - p$.
- Binomial(n, p): $p_X(k) = \binom{n}{k} p^k (1-p)^{n-k}$.
- Categorical($\pi_1, \dots, \pi_K$): $p_X(k) = \pi_k$, $\sum_k \pi_k = 1$.
- Geometric(p): $p_X(k) = (1-p)^{k-1} p$, $k \ge 1$.
- Uniform$[a, b]$: $f_X(x) = \frac{1}{b-a} \mathbf{1}_{[a,b]}(x)$.
- Gaussian $\mathcal{N}(\mu, \sigma^2)$: $f_X(x) = \frac{1}{\sqrt{2\pi}\sigma} \exp\Big(-\frac{(x-\mu)^2}{2\sigma^2}\Big)$.

Definition 0.81 (Joint, marginal, conditional)**.** *For (X, Y) on the same space the* joint *distribution is on $\mathbb{R}^2$.* Marginals *integrate/sum out: $f_X(x) = \int f_{X,Y}(x, y)\, dy$. The* conditional *density is $f_{Y|X}(y \mid x) = f_{X,Y}(x, y)/f_X(x)$ when $f_X(x) > 0$.*

Theorems and Proofs

Theorem 0.82 (Properties of CDF)**.** *F_X is (i) non-decreasing, (ii) right-continuous, (iii)* $\lim_{x\to-\infty} F_X(x) = 0$*, (iv)* $\lim_{x\to\infty} F_X(x) = 1$*.*

Proof. (i) If $x \le y$ then $\{X \le x\} \subseteq \{X \le y\}$, so $\mathbb{P}$-monotonicity gives $F_X(x) \le F_X(y)$.

(ii) Fix x and let $x_n \downarrow x$. Then $\{X \le x_n\} \downarrow \{X \le x\}$ (countable intersection). By continuity of $\mathbb{P}$ from above (Chapter 8, since $\mathbb{P}(\{X \le x_1\}) \le 1 < \infty$),

$$F_X(x_n) = \mathbb{P}(X \le x_n) \to \mathbb{P}(X \le x) = F_X(x).$$

(iii) Take $x_n \downarrow -\infty$. Then $\{X \le x_n\} \downarrow \emptyset$, hence $F_X(x_n) \to \mathbb{P}(\emptyset) = 0$.

(iv) Take $x_n \uparrow \infty$. Then $\{X \le x_n\} \uparrow \Omega$; continuity from below gives $F_X(x_n) \to \mathbb{P}(\Omega) = 1$. □

Theorem 0.83 (Normalization)**.** $\sum_{x\in S} p_X(x) = 1$ *in the discrete case, and* $\int_{\mathbb{R}} f_X(x)\,dx = 1$ *in the continuous case.*

Proof. Discrete: $\{X = x\}$ for $x \in S$ are disjoint with union $\{X \in S\} = \Omega$. Countable additivity:

$$1 = \mathbb{P}(\Omega) = \sum_{x\in S} \mathbb{P}(X = x) = \sum_{x} p_X(x).$$

Continuous: take $A = \mathbb{R}$ in the defining property: $1 = \mathbb{P}(X \in \mathbb{R}) = \int_{\mathbb{R}} f_X$. □

Theorem 0.84 (Change of variables, 1-D)**.** *Let X have density f_X and let g be strictly monotone and C^1 on the support of X. Then $Y = g(X)$ has density*

$$f_Y(y) = f_X(g^{-1}(y))\,\big|(g^{-1})'(y)\big|.$$

Proof. Suppose g is strictly increasing. For $y \in g(\operatorname{supp} X)$,

$$F_Y(y) = \mathbb{P}(g(X) \le y) = \mathbb{P}(X \le g^{-1}(y)) = F_X(g^{-1}(y)).$$

Differentiate using the chain rule (Chapter 3) and the fact that $F_X' = f_X$:

$$f_Y(y) = f_X(g^{-1}(y))\,(g^{-1})'(y).$$

Since g^{-1} is increasing, $(g^{-1})'(y) > 0$, matching the absolute value. If g is decreasing, $\{g(X) \le y\} = \{X \ge g^{-1}(y)\}$, so $F_Y(y) = 1 - F_X(g^{-1}(y))$ and differentiation gives $-f_X(g^{-1}(y))(g^{-1})'(y)$ with $(g^{-1})'(y) < 0$, again the absolute value. □

Theorem 0.85 (Inverse-CDF sampling)**.** *Let F be a CDF on $\mathbb{R}$ and define the generalized inverse* $F^{-1}(u) = \inf\{x : F(x) \ge u\}$*. If $U \sim$ Uniform$[0,1]$, then $X := F^{-1}(U)$ has CDF F.*

Proof. We claim $\{F^{-1}(U) \le x\} = \{U \le F(x)\}$ up to a null set. If $F^{-1}(u) \le x$, the infimum definition together with right-continuity of F implies $F(x) \ge u$. Conversely, if $u \le F(x)$ then $x \in \{x' : F(x') \ge u\}$, so $F^{-1}(u) \le x$. Hence

$$\mathbb{P}(X \le x) = \mathbb{P}(U \le F(x)) = F(x),$$

using uniformity of U on $[0,1]$. □

Code Sketch

The notebook (`cells.json`): (1) builds a 5-class categorical via softmax of fixed logits and verifies $\sum p_X = 1$; (2) computes its CDF and samples 10000 draws by inverse-CDF, comparing empirical to true; (3) implements $f(x) = \frac{1}{\sqrt{2\pi}} e^{-x^2/2}$, verifies $\int f \approx 1$ by Riemann sum, and computes $F(x)$ by cumulative sum; (4) verifies Theorem 9.3 on $Y = -\ln X$ with $X \sim U[0,1]$, yielding $Y \sim \text{Exp}(1)$; (5) re-runs inverse-CDF sampling on the categorical with growing n to demonstrate convergence.

Connection to LLMs

A causal language model produces logits $z_t \in \mathbb{R}^V$, which softmax maps to a categorical distribution

$$p_\theta(x_t \mid x_{<t}) = \text{softmax}(z_t).$$

Token sampling at temperature τ rescales logits to z_t/τ and draws from this categorical. The standard implementation is exactly inverse-CDF sampling (Theorem 9.4) on the cumulative softmax, or equivalently the *Gumbel-max trick* $\arg\max_k(z_k + G_k)$ with $G_k \sim \text{Gumbel}(0,1)$. Cross-entropy loss (Chapter 17) is $-\log p_\theta(x_t \mid x_{<t})$, an expectation under the data distribution; the causal LM training objective (Chapter 25) is the joint log-likelihood factored through the conditional distributions defined here.

Chapter 10: Expectation, variance, covariance; Jensen's inequality

Motivation

Random variables (Chapter 9) describe uncertain outcomes. To compress an entire distribution into a single representative number we use the *expectation*; to describe spread and joint linear association we use *variance* and *covariance*. Together with *Jensen's inequality*, these objects form the structural backbone of every loss function and convergence proof in modern deep learning. Training an LLM is, formally, the minimization of an expectation $\mathbb{E}_{x\sim\mathcal{D}}[\ell(\theta; x)]$; SGD is a Monte Carlo estimator of that expectation; the variance of the estimator dictates convergence speed (Chapter 13).

Definitions

Definition 0.86 (Expectation)**.** *For a discrete random variable X with pmf p_X,*

$$\mathbb{E}[X] \;=\; \sum_x x\, p_X(x), \qquad \textit{provided } \sum_x |x|\, p_X(x) < \infty.$$

For continuous X with density f_X,

$$\mathbb{E}[X] \;=\; \int_{\mathbb{R}} x\, f_X(x)\, dx,$$

provided the integral is absolutely convergent. In full generality, $\mathbb{E}[X] := \int_\Omega X\, d\mathbb{P}$ is the Lebesgue integral against the underlying probability measure (cited without proof; see Billingsley, Probability and Measure*).*

Definition 0.87 (Variance and covariance)**.** *For X with $\mathbb{E}[X^2] < \infty$, $\text{Var}(X) := \mathbb{E}[(X - \mathbb{E}X)^2]$, and $\sigma_X := \sqrt{\text{Var}(X)}$. For X, Y with finite variance,*

$$\text{Cov}(X,Y) := \mathbb{E}\big[(X - \mathbb{E}X)(Y - \mathbb{E}Y)\big], \qquad \rho_{X,Y} := \frac{\text{Cov}(X,Y)}{\sigma_X \sigma_Y} \in [-1, 1].$$

The bound $|\rho| \le 1$ is the Cauchy–Schwarz inequality applied to the inner product $\langle U, V\rangle := \mathbb{E}[UV]$ on the Hilbert space $L^2(\Omega, \mathcal{F}, \mathbb{P})$.

Definition 0.88 (Conditional expectation)**.** *For discrete X, Y and any y with $p_Y(y) > 0$,*

$$\mathbb{E}[X \mid Y = y] \;=\; \sum_{x} x\, p_{X|Y}(x \mid y).$$

Viewed as y varies, $\mathbb{E}[X \mid Y]$ is itself a random variable: a measurable function of Y. The general definition characterizes $\mathbb{E}[X \mid \mathcal{G}]$ as the unique (a.s.) $\mathcal{G}$-measurable random variable satisfying $\int_A \mathbb{E}[X \mid \mathcal{G}]\, d\mathbb{P} = \int_A X\, d\mathbb{P}$ for every $A \in \mathcal{G}$ (Radon–Nikodym).

Theorems with proofs

Theorem 0.89 (Linearity of expectation)**.** *For random variables X, Y with finite expectation and scalars $a, b \in \mathbb{R}$,*

$$\mathbb{E}[aX + bY] \;=\; a\,\mathbb{E}[X] + b\,\mathbb{E}[Y].$$

Independence is not *required.*

Proof. In the discrete case, with joint pmf $p_{X,Y}$,

$$\mathbb{E}[aX + bY] = \sum_{x,y} (ax + by)\, p_{X,Y}(x, y) = a \sum_{x,y} x\, p_{X,Y}(x, y) + b \sum_{x,y} y\, p_{X,Y}(x, y).$$

Marginalizing, $\sum_y p_{X,Y}(x, y) = p_X(x)$ and $\sum_x p_{X,Y}(x, y) = p_Y(y)$, giving $a\,\mathbb{E}[X] + b\,\mathbb{E}[Y]$. The continuous case is identical with sums replaced by integrals; in full generality, linearity is a defining property of the Lebesgue integral. □

Theorem 0.90 (Variance of a sum)**.**

$$\operatorname{Var}(X + Y) \;=\; \operatorname{Var}(X) + \operatorname{Var}(Y) + 2\operatorname{Cov}(X, Y).$$

Proof. Let $\mu_X = \mathbb{E}[X]$, $\mu_Y = \mathbb{E}[Y]$. By linearity, $\mathbb{E}[X + Y] = \mu_X + \mu_Y$. Then

$$\begin{aligned} \operatorname{Var}(X + Y) &= \mathbb{E}\big[((X - \mu_X) + (Y - \mu_Y))^2\big] \\ &= \mathbb{E}[(X - \mu_X)^2] + \mathbb{E}[(Y - \mu_Y)^2] + 2\,\mathbb{E}[(X - \mu_X)(Y - \mu_Y)] \\ &= \operatorname{Var}(X) + \operatorname{Var}(Y) + 2\operatorname{Cov}(X, Y). \end{aligned}$$

□

Remark 0.91. If $X \perp Y$ then $\operatorname{Cov}(X, Y) = 0$ and variances add. The converse is false: zero covariance does not imply independence.

Theorem 0.92 (Jensen's inequality)**.** *Let $\phi : \mathbb{R} \to \mathbb{R}$ be convex and X a random variable with $\mathbb{E}|X| < \infty$ and $\mathbb{E}|\phi(X)| < \infty$. Then*

$$\phi(\mathbb{E}[X]) \;\le\; \mathbb{E}[\phi(X)].$$

Proof. Recall (Chapter 7) that a convex function on $\mathbb{R}$ admits a *supporting line* at every interior point of its domain: at $x_0 := \mathbb{E}[X]$ there exists a subgradient $g \in \partial\phi(x_0)$ such that

$$\phi(x) \;\ge\; \phi(x_0) + g(x - x_0) \qquad \text{for all } x \in \mathbb{R}.$$

Substitute X pointwise and take expectations. By linearity (Theorem above),

$$\mathbb{E}[\phi(X)] \;\ge\; \phi(x_0) + g\,(\mathbb{E}[X] - x_0) \;=\; \phi(\mathbb{E}[X]),$$

since $\mathbb{E}[X] - x_0 = 0$. □

Theorem 0.93 (Markov's inequality)**.** *Let $X \geq 0$ a.s. and $a > 0$. Then*

$$\mathbb{P}(X \geq a) \;\leq\; \frac{\mathbb{E}[X]}{a}.$$

Proof. Pointwise, $a \cdot \mathbf{1}_{\{X \geq a\}} \leq X$: when $X \geq a$, both sides are bounded above by X; when $X < a$, the left side is 0 and $X \geq 0$. Take expectations: $a\,\mathbb{P}(X \geq a) \leq \mathbb{E}[X]$, then divide by a. □

Theorem 0.94 (Chebyshev's inequality)**.** *Let $\mathbb{E}[X] = \mu$ and $\sigma^2 := \mathrm{Var}(X) < \infty$. For $k > 0$,*

$$\mathbb{P}(|X - \mu| \geq k\sigma) \;\leq\; \frac{1}{k^2}.$$

Proof. Apply Markov to $Y := (X - \mu)^2 \geq 0$ with $a = k^2\sigma^2$:

$$\mathbb{P}((X - \mu)^2 \geq k^2\sigma^2) \;\leq\; \frac{\mathbb{E}[(X - \mu)^2]}{k^2\sigma^2} \;=\; \frac{1}{k^2}.$$

The event $\{(X - \mu)^2 \geq k^2\sigma^2\}$ equals $\{|X - \mu| \geq k\sigma\}$. □

Theorem 0.95 (Weak law of large numbers)**.** *Let $X_1, X_2, \ldots$ be i.i.d. with $\mathbb{E}[X_i] = \mu$ and $\mathrm{Var}(X_i) = \sigma^2 < \infty$. Set $\bar{X}_n := \frac{1}{n}\sum_{i=1}^{n} X_i$. Then for every $\epsilon > 0$,*

$$\mathbb{P}(|\bar{X}_n - \mu| \geq \epsilon) \xrightarrow{n \to \infty} 0.$$

Proof. By linearity, $\mathbb{E}[\bar{X}_n] = \mu$. By the variance-of-a-sum theorem and independence (covariances vanish), $\mathrm{Var}(\bar{X}_n) = \sigma^2/n$. Chebyshev's inequality yields

$$\mathbb{P}(|\bar{X}_n - \mu| \geq \epsilon) \;\leq\; \frac{\mathrm{Var}(\bar{X}_n)}{\epsilon^2} \;=\; \frac{\sigma^2}{n\epsilon^2} \;\to\; 0.$$ □

Code sketch

The companion notebook (`cells.json`) verifies each result numerically: a five-outcome categorical for analytic vs. empirical $\mathbb{E}$, Var; the pair $(X, 2X+1)$ for linearity without independence; $\phi(x) = e^x$ on Unif$\{-1, +1\}$ for Jensen; and i.i.d. Unif$[0, 1]$ samples for the LLN inside a Chebyshev confidence band.

Connection to LLMs

Every modern language-model objective has the form

$$\mathcal{L}(\theta) \;=\; \mathbb{E}_{x \sim \mathcal{D}}[\ell(\theta; x)],$$

where $\mathcal{D}$ is the data distribution and ℓ is (typically) the next-token cross-entropy loss. The data distribution is intractable, so we replace the expectation by an empirical mean over a mini-batch of size B:

$$\hat{\mathcal{L}}(\theta) \;=\; \frac{1}{B}\sum_{i=1}^{B} \ell(\theta; x_i).$$

Linearity of expectation gives $\mathbb{E}[\nabla\hat{\mathcal{L}}] = \nabla\mathcal{L}$, so the SGD gradient is *unbiased*. Theorem on variance of a sum, plus independence within the batch, yields $\mathrm{Var}(\nabla\hat{\mathcal{L}}) = \Theta(1/B)$ — exactly the reason larger batches give smoother training curves. The weak LLN guarantees recovery of the true loss in probability as $B \to \infty$ (formalized in Chapter 13). Jensen's inequality, applied to $-\log$, underwrites the evidence lower bound (ELBO) of Chapter 22:

$$\log \mathbb{E}_q[Z] \;\geq\; \mathbb{E}_q[\log Z], \qquad Z > 0,$$

which is the bedrock of every modern likelihood-based generative model.

Chapter 11: Information theory: self-information, entropy, cross-entropy, KL

Motivation

Chapter 9 gave us random variables; Chapter 10 gave us expectation and Jensen's inequality. We now ask: how much *information* does an outcome carry, and how *different* are two distributions on the same space? The answers — entropy, cross-entropy, and the Kullback–Leibler divergence — are not optional flavor. They *are* the loss surface of every modern language model. Cross-entropy is the next-token training objective (Chapters 17, 25); KL is the trust-region regularizer in RLHF / DPO (Chapter 28); the entropy of the softmax is what people mean by "sampling diversity." Every result below follows from one fact already proved: $-\ln$ is convex.

We use the natural log ln throughout for clean derivatives. Replacing ln by $\log_2$ multiplies every formula by $1/\ln 2$ and converts *nats* to *bits*; nothing else changes.

Definitions

Definition 0.96 (Self-information)**.** *For a discrete random variable X with PMF p and $x \in \operatorname{supp}(p)$, the* self-information *of the outcome x is $I(x) := -\ln p(x)$. Rare outcomes carry more information; $I(x) \to \infty$ as $p(x) \to 0$.*

Definition 0.97 (Shannon entropy)**.** *The* entropy *of p is the expected self-information,*

$$H(p) := \mathbb{E}_{x\sim p}[I(x)] = -\sum_x p(x) \ln p(x),$$

with the convention $0 \ln 0 = 0$, justified by $\lim_{t\downarrow 0} t \ln t = 0$.

Definition 0.98 (Joint and conditional entropy)**.** *For a joint PMF $p_{X,Y}$,*

$$H(X,Y) = -\sum_{x,y} p_{X,Y}(x,y) \ln p_{X,Y}(x,y), \qquad H(Y \mid X) = -\sum_{x,y} p_{X,Y}(x,y) \ln p_{Y|X}(y \mid x).$$

Definition 0.99 (Cross-entropy)**.** *For PMFs p, q on the same alphabet with $\operatorname{supp}(p) \subseteq \operatorname{supp}(q)$,*

$$H(p,q) := -\sum_x p(x) \ln q(x) = \mathbb{E}_{x\sim p}[-\ln q(x)].$$

Definition 0.100 (Kullback–Leibler divergence)**.**

$$D_{\mathrm{KL}}(p \,\|\, q) := \sum_x p(x) \ln \frac{p(x)}{q(x)} = \mathbb{E}_{x\sim p}\left[\ln \frac{p(x)}{q(x)}\right].$$

D_{KL} is not *symmetric and* not *a metric.*

Definition 0.101 (Mutual information)**.** *For joint $p_{X,Y}$ with marginals p_X, p_Y,*

$$I(X;Y) := D_{\mathrm{KL}}(p_{X,Y} \,\|\, p_X \otimes p_Y) = \sum_{x,y} p_{X,Y}(x,y) \ln \frac{p_{X,Y}(x,y)}{p_X(x) p_Y(y)}.$$

Theorems and proofs

Theorem 0.102 (Gibbs' inequality / KL nonnegativity)**.** *For PMFs p, q on the same finite alphabet with $q(x) > 0$ wherever $p(x) > 0$,*

$$D_{\mathrm{KL}}(p \,\|\, q) \geq 0,$$

with equality iff $p(x) = q(x)$ for every x with $p(x) > 0$.

Proof. The function $\phi(t) = -\ln t$ is strictly convex on $(0, \infty)$ since $\phi''(t) = 1/t^2 > 0$. Compute, viewing the sum as $\mathbb{E}_{x\sim p}$:

$$-D_{\mathrm{KL}}(p \,\|\, q) = \sum_{x:p(x)>0} p(x) \ln \frac{q(x)}{p(x)} = -\,\mathbb{E}_{x\sim p}\Big[\phi\Big(\tfrac{q(x)}{p(x)}\Big)\Big].$$

By Jensen's inequality (Chapter 10), $\mathbb{E}[\phi(Z)] \geq \phi(\mathbb{E}[Z])$ for convex ϕ, hence

$$-D_{\mathrm{KL}}(p \,\|\, q) \leq -\phi\Big(\mathbb{E}_{x\sim p}\Big[\tfrac{q(x)}{p(x)}\Big]\Big) = \ln\Big(\sum_{x:p(x)>0} q(x)\Big) \leq \ln 1 = 0,$$

using $\sum_x q(x) = 1$. Therefore $D_{\mathrm{KL}}(p \,\|\, q) \geq 0$. By strict convexity, Jensen is an equality iff $q(x)/p(x)$ is p-a.s. constant; since both p and q sum to 1 on the support of p, that constant must be 1, i.e. $p = q$ on $\mathrm{supp}(p)$. □

Theorem 0.103 (Cross-entropy decomposition)**.** $H(p, q) = H(p) + D_{\mathrm{KL}}(p \,\|\, q)$.

Proof. Direct algebra:

$$H(p, q) = -\sum_x p(x) \ln q(x) = -\sum_x p(x)\big[\ln p(x) + \ln \tfrac{q(x)}{p(x)}\big] = H(p) + D_{\mathrm{KL}}(p \,\|\, q).$$ □

Corollary 0.104. *$H(p, q) \geq H(p)$, with equality iff $q = p$. In particular, $\min_q H(p, q) = H(p)$, attained at $q = p$.*

Theorem 0.105 (Maximum entropy on a finite alphabet)**.** *For any PMF p supported on a set of size K,*

$$H(p) \leq \ln K,$$

with equality iff p is the uniform distribution $u(x) = 1/K$.

Proof. Let u denote the uniform distribution on the K-element support. Compute

$$D_{\mathrm{KL}}(p \,\|\, u) = \sum_x p(x) \ln \frac{p(x)}{1/K} = -H(p) + \ln K \cdot \sum_x p(x) = \ln K - H(p).$$

By Gibbs' inequality this is ≥ 0, so $H(p) \leq \ln K$, with equality iff $p = u$. □

Theorem 0.106 (Chain rule for entropy)**.** $H(X, Y) = H(X) + H(Y \mid X)$.

Proof. Apply $\ln p_{X,Y}(x, y) = \ln p_X(x) + \ln p_{Y|X}(y \mid x)$:

$$H(X, Y) = -\sum_{x,y} p_{X,Y}(x, y) \ln p_X(x) \;-\; \sum_{x,y} p_{X,Y}(x, y) \ln p_{Y|X}(y \mid x).$$

Marginalizing y in the first sum gives $-\sum_x p_X(x) \ln p_X(x) = H(X)$; the second sum is $H(Y \mid X)$ by definition. □

Theorem 0.107 (Mutual information nonnegativity)**.** *$I(X;Y) \geq 0$, with equality iff X and Y are independent.*

Proof. By definition $I(X;Y) = D_{\mathrm{KL}}(p_{X,Y} \,\|\, p_X \otimes p_Y)$. Apply Gibbs (Theorem 1): this is ≥ 0 with equality iff $p_{X,Y}(x,y) = p_X(x)p_Y(y)$ for all x, y — exactly independence. □

Remark 0.108. Combining the chain rule with the definition of I yields the standard symmetric identity

$$I(X;Y) = H(X) + H(Y) - H(X,Y) = H(Y) - H(Y \mid X) = H(X) - H(X \mid Y).$$

Conditioning never increases entropy: $H(Y \mid X) \leq H(Y)$.

Code sketch

The notebook builds two categorical distributions p, q on a 5-token alphabet, implements `entropy`, `cross_entropy`, and `kl` from the definitions above, and verifies $H(p,q) = H(p) + D_{\mathrm{KL}}(p \,\|\, q)$ to machine precision. It samples 50 random Dirichlet pairs (seed 0) and confirms $D_{\mathrm{KL}} \geq 0$ for every pair, with $D_{\mathrm{KL}}(p \,\|\, p) = 0$. For $K = 8$, it confirms $H(\text{uniform}) = \ln 8$ and that $H(p) < \ln 8$ for 100 non-uniform draws. Finally it constructs a correlated joint on $\{0,1\}^2$, computes $I(X;Y)$ via both formulas, verifies agreement, and shows $I = 0$ for the product distribution.

Connection to LLMs

A causal language model defines $p_\theta(x_t \mid x_{<t})$ via softmax (Chapter 9). On a corpus drawn from p_{data}, the training objective is

$$\mathcal{L}(\theta) = \mathbb{E}_{x \sim p_{\text{data}}}[-\ln p_\theta(x_t \mid x_{<t})] = H(p_{\text{data}}, p_\theta) = H(p_{\text{data}}) + D_{\mathrm{KL}}(p_{\text{data}} \,\|\, p_\theta).$$

Theorem 2 makes precise why minimizing cross-entropy *is* minimizing KL to the data: the entropy term $H(p_{\text{data}})$ does not depend on θ. Theorem 4 sets the absolute floor: a uniform LM over a 50000-token vocabulary has entropy $\ln 50000 \approx 10.82$ nats, which is the loss every trained model improves on (Chapter 17, 25). In RLHF and DPO (Chapter 28), the policy update is regularized by $D_{\mathrm{KL}}(\pi_\theta \,\|\, \pi_{\text{ref}})$ to keep the fine-tuned policy close to the SFT initialization — Gibbs' inequality is exactly what makes that penalty a meaningful divergence. Finally, the entropy of the sampler's softmax, controlled by temperature, is the "diversity" knob in decoding.

Chapter 12: Statistical inference: likelihood, MLE, ERM, bias-variance

Motivation

Probability theory (Chapters 7–10) reasons forward from a known distribution to data. *Statistical inference* runs the arrow backwards: given data, recover the distribution. Two principles dominate modern practice and underwrite essentially all of deep learning: *maximum likelihood estimation* (MLE) and its generalization, *empirical risk minimization* (ERM). We will show that MLE is precisely the minimizer of cross-entropy between the empirical distribution and the model, connecting Chapter 11's information theory to Chapter 17's language-model objective.

Definitions

Definition 0.109 (Statistical model and iid sample)**.** *A* statistical model *is a family* $\mathcal{P} = \{p_\theta : \theta \in \Theta\}$ *of probability densities (or mass functions) on a sample space* $\mathcal{X}$*, indexed by* $\theta \in \Theta \subseteq \mathbb{R}^d$*. A sample* $X_1, \dots, X_n$ *is* iid *from* p_{θ^*} *if the joint density factors as* $\prod_{i=1}^n p_{\theta^*}(x_i)$*.*

Definition 0.110 (Likelihood and MLE)**.** *The* likelihood *and* log-likelihood *are*

$$L(\theta; \mathbf{x}) = \prod_{i=1}^n p_\theta(x_i), \qquad \ell(\theta; \mathbf{x}) = \sum_{i=1}^n \log p_\theta(x_i).$$

The maximum likelihood estimator *is* $\hat{\theta}_{\mathrm{MLE}} = \arg\max_{\theta \in \Theta} \ell(\theta; \mathbf{x})$*.*

Definition 0.111 (Empirical risk minimization)**.** *Given a loss* $\ell(\theta; x)$ *(not necessarily* $-\log p_\theta$*),*

$$\hat{\theta}_{\mathrm{ERM}} = \arg\min_{\theta \in \Theta} \frac{1}{n} \sum_{i=1}^n \ell(\theta; x_i).$$

MLE is ERM with $\ell(\theta; x) = -\log p_\theta(x)$*.*

Definition 0.112 (Bias, variance, MSE)**.** *For an estimator* $\hat{\theta} = \hat{\theta}(X_1, \dots, X_n)$ *of a scalar* θ^**:*

$$\mathrm{Bias}(\hat{\theta}) = \mathbb{E}[\hat{\theta}] - \theta^*, \quad \mathrm{Var}(\hat{\theta}) = \mathbb{E}[(\hat{\theta} - \mathbb{E}\hat{\theta})^2], \quad \mathrm{MSE}(\hat{\theta}) = \mathbb{E}[(\hat{\theta} - \theta^*)^2].$$

Theorems

Theorem 0.113 (MLE equals minimum cross-entropy)**.** *Let* $\hat{p}_n(x) = \frac{1}{n} \sum_{i=1}^n \mathbf{1}\{x_i = x\}$ *be the empirical distribution. Then*

$$\arg\max_\theta \ell(\theta; \mathbf{x}) \;=\; \arg\min_\theta H(\hat{p}_n,\, p_\theta),$$

where $H(\hat{p}_n, p_\theta) = -\sum_x \hat{p}_n(x) \log p_\theta(x)$ *is the cross-entropy from Chapter 11.*

Proof. Direct algebra:

$$\tfrac{1}{n}\, \ell(\theta) = \tfrac{1}{n} \sum_{i=1}^n \log p_\theta(x_i) = \sum_x \hat{p}_n(x) \log p_\theta(x) = -H(\hat{p}_n, p_\theta).$$

Maximizing ℓ is equivalent to maximizing ℓ/n, which is equivalent to minimizing $H(\hat{p}_n, p_\theta)$. □

Corollary 0.114 (MLE equals minimum KL)**.** *By the Chapter 11 identity* $H(\hat{p}_n, p_\theta) = H(\hat{p}_n) + D_{\mathrm{KL}}(\hat{p}_n \,\|\, p_\theta)$*, and since* $H(\hat{p}_n)$ *does not depend on* θ*,*

$$\hat{\theta}_{\mathrm{MLE}} = \arg\min_\theta D_{\mathrm{KL}}(\hat{p}_n \,\|\, p_\theta).$$

Theorem 0.115 (Bias–variance decomposition)**.** *For any square-integrable estimator* $\hat{\theta}$ *of* $\theta^* \in \mathbb{R}$*,*

$$\mathrm{MSE}(\hat{\theta}) = \mathrm{Bias}(\hat{\theta})^2 + \mathrm{Var}(\hat{\theta}).$$

Proof. Let $\bar{\theta} = \mathbb{E}[\hat{\theta}]$. Add and subtract $\bar{\theta}$ inside the square:

$$\mathbb{E}[(\hat{\theta} - \theta^*)^2] = \mathbb{E}\big[((\hat{\theta} - \bar{\theta}) + (\bar{\theta} - \theta^*))^2\big].$$

Expand:

$$= \mathbb{E}[(\hat{\theta} - \bar{\theta})^2] + 2(\bar{\theta} - \theta^*)\,\mathbb{E}[\hat{\theta} - \bar{\theta}] + (\bar{\theta} - \theta^*)^2.$$

The cross term vanishes because $\mathbb{E}[\hat{\theta} - \bar{\theta}] = 0$, leaving $\mathrm{Var}(\hat{\theta}) + \mathrm{Bias}(\hat{\theta})^2$. □

Theorem 0.116 (MLE for Gaussian mean, known variance)**.** *Let $X_1, \dots, X_n \overset{\text{iid}}{\sim} \mathcal{N}(\mu, \sigma^2)$ with σ^2 known. Then $\hat{\mu}_{\text{MLE}} = \bar{X}_n = \frac{1}{n}\sum_{i=1}^n X_i$.*

Proof. The log-likelihood is

$$\ell(\mu) = -\tfrac{n}{2}\log(2\pi\sigma^2) - \tfrac{1}{2\sigma^2}\sum_{i=1}^n (X_i - \mu)^2.$$

Differentiate:

$$\ell'(\mu) = \tfrac{1}{\sigma^2}\sum_{i=1}^n (X_i - \mu).$$

Setting $\ell'(\mu) = 0$ gives $\sum_i X_i = n\mu$, i.e. $\hat{\mu} = \bar{X}_n$. The second derivative is $-n/\sigma^2 < 0$, so this is a strict maximum. □

Theorem 0.117 (Consistency of MLE, sketch)**.** *Under regularity conditions (identifiability, compact Θ, dominated $\log p_\theta$), $\hat{\theta}_n \xrightarrow{P} \theta^*$ as $n \to \infty$.*

Sketch. By the Chapter 10 LLN, $\frac{1}{n}\ell_n(\theta) \xrightarrow{P} M(\theta) := \mathbb{E}_{\theta^*}[\log p_\theta(X)]$. Gibbs' inequality (Chapter 11) gives $M(\theta) \le M(\theta^*)$ with equality iff $p_\theta = p_{\theta^*}$ a.s., so θ^* is the unique maximizer of M. A uniform LLN over Θ transfers the maximizer of ℓ_n/n to that of M, yielding $\hat{\theta}_n \xrightarrow{P} \theta^*$. □

Code sketch

The accompanying notebook (`cells.json`): (i) simulates $n = 1000$ samples from $\mathcal{N}(2, 1)$, evaluates $\ell(\mu)$ on a grid, and confirms $\arg\max = \bar{X}_n$; (ii) draws $n = 500$ from a categorical with $p = (0.1, 0.2, 0.3, 0.4)$, computes $\hat{p}_n$, and checks that $H(\hat{p}_n, p_\theta)$ is minimized exactly at $\theta = \hat{p}_n$; (iii) empirically decomposes MSE for two estimators of a Gaussian mean; (iv) solves linear regression as ERM via the normal equation.

Connection to LLMs

Remark 0.118. Language-model *pre-training* is MLE. For an autoregressive model, $p_\theta(x_{1:T}) = \prod_t p_\theta(x_t \mid x_{<t})$, so the corpus log-likelihood is $\sum_{\text{seq}}\sum_t \log p_\theta(x_t \mid x_{<t})$. The training loss $-\ell(\theta)/N$ is exactly the cross-entropy $H(\hat{p}_n, p_\theta)$ between the empirical token distribution and the model. By Theorem 0.113 and its corollary, every gradient step of a transformer (Chapter 25) is one step of MLE = ERM with log-loss = KL projection toward the empirical corpus. Bias–variance reasoning (Theorem 0.115) then governs the scaling laws and generalization gap discussed in Chapter 27.

Block C — Stochastic Optimization

Chapter 13: SGD: stochastic-approximation theorem; mini-batching; convergence sketch

Motivation

Chapter 7 proved that gradient descent on an L-smooth convex function reaches an ϵ-stationary point in $O(1/\epsilon)$ iterations using the *full* gradient $\nabla F(\theta)$. For a transformer pre-trained on 10^{13} tokens, a single full-gradient step would require a forward/backward pass over the entire corpus — weeks of compute per update. The fix is to estimate ∇F from a random *mini-batch* and accept a noisy step in exchange for a cheap one. This chapter justifies the trade. We define the stochastic objective, prove that mini-batching reduces variance as $1/B$, and combine the descent lemma of Chapter 7 with the linearity of expectation (Chapter 10) to derive the canonical $O(1/\sqrt{T})$ rate of stochastic gradient descent on smooth (possibly non-convex) losses.

Definitions

Definition 0.119 (Stochastic objective)**.** *Let $\xi \sim \mathcal{D}$ be a random data point and $f(\cdot;\xi) : \mathbb{R}^n \to \mathbb{R}$ a loss. The* population risk *is*

$$F(\theta) := \mathbb{E}_{\xi\sim\mathcal{D}}[f(\theta;\xi)],$$

in the sense of Chapter 10. Training data are i.i.d. draws $\xi_1, \xi_2, \dots$ from $\mathcal{D}$.

Definition 0.120 (Stochastic gradient)**.** $\hat{g}(\theta;\xi) := \nabla_\theta f(\theta;\xi)$. *Under mild regularity (interchange of ∇ and $\mathbb{E}$),*

$$\mathbb{E}_\xi[\hat{g}(\theta;\xi)] = \nabla F(\theta) \qquad (\textit{unbiasedness}).$$

Definition 0.121 (SGD update)**.** *Given step size $\eta_t > 0$ and an independent sample $\xi_t \sim \mathcal{D}$,*

$$\theta_{t+1} = \theta_t - \eta_t \hat{g}(\theta_t;\xi_t).$$

Definition 0.122 (Mini-batch SGD)**.** *Draw B i.i.d. samples $\xi_1, \dots, \xi_B$ and average:*

$$\hat{g}_B(\theta) := \frac{1}{B}\sum_{j=1}^{B} \nabla_\theta f(\theta;\xi_j).$$

Definition 0.123 (Bounded variance)**.** $\mathbb{E}_\xi\|\hat{g}(\theta;\xi) - \nabla F(\theta)\|^2 \le \sigma^2$ *uniformly in θ.*

Theorems and proofs

Theorem 0.124 (Variance reduction by batching)**.** *Under bounded variance and i.i.d. sampling,*

$$\mathbb{E}\|\hat{g}_B(\theta) - \nabla F(\theta)\|^2 \le \frac{\sigma^2}{B}.$$

Proof. Let $Z_j := \hat{g}(\theta;\xi_j) - \nabla F(\theta)$. The Z_j are i.i.d., zero-mean, with $\mathbb{E}\|Z_j\|^2 \le \sigma^2$. Then $\hat{g}_B - \nabla F = \frac{1}{B}\sum_j Z_j$, and

$$\mathbb{E}\Big\|\tfrac{1}{B}\sum_j Z_j\Big\|^2 = \tfrac{1}{B^2}\sum_{j,k}\mathbb{E}\langle Z_j, Z_k\rangle = \tfrac{1}{B^2}\sum_j \mathbb{E}\|Z_j\|^2 \le \frac{\sigma^2}{B},$$

where cross terms vanish by independence and zero mean (Theorem 10.6). □

Remark 0.125. This is the law of large numbers (Chapter 10) applied to the gradient: doubling the batch halves the noise variance.

Theorem 0.126 (SGD on L-smooth, possibly non-convex F)**.** *Suppose F is L-smooth, bounded below by $F^\star$, and run SGD with constant step $\eta = \min(1/L,\, c/\sqrt{T})$ for T iterations, with stochastic gradients of variance at most σ^2/B. Then*

$$\frac{1}{T}\sum_{t=0}^{T-1} \mathbb{E}\|\nabla F(\theta_t)\|^2 \le \frac{2\big(F(\theta_0) - F^\star\big)}{\eta T} + \frac{L\eta\sigma^2}{B} = O(1/\sqrt{T}).$$

Proof. Let $g_t := \nabla F(\theta_t)$ and write $\hat{g}_t = g_t + \zeta_t$ with $\mathbb{E}[\zeta_t \mid \theta_t] = 0$ and $\mathbb{E}\|\zeta_t\|^2 \le \sigma^2/B$. The descent lemma of Chapter 7, applied to the displacement $\theta_{t+1} - \theta_t = -\eta\hat{g}_t$, gives

$$F(\theta_{t+1}) \le F(\theta_t) - \eta\langle g_t, \hat{g}_t\rangle + \tfrac{L\eta^2}{2}\|\hat{g}_t\|^2.$$

Take conditional expectation given θ_t. Using $\mathbb{E}[\hat{g}_t \mid \theta_t] = g_t$ and $\mathbb{E}\|\hat{g}_t\|^2 = \|g_t\|^2 + \mathbb{E}\|\zeta_t\|^2$,

$$\mathbb{E}[F(\theta_{t+1}) \mid \theta_t] \le F(\theta_t) - \eta\|g_t\|^2 + \tfrac{L\eta^2}{2}\big(\|g_t\|^2 + \sigma^2/B\big).$$

With $\eta \le 1/L$ we have $\frac{L\eta^2}{2} \le \eta/2$, so the coefficient of $\|g_t\|^2$ becomes $-\eta + \eta/2 = -\eta/2$:

$$\mathbb{E}[F(\theta_{t+1}) \mid \theta_t] \le F(\theta_t) - \tfrac{\eta}{2}\|g_t\|^2 + \tfrac{L\eta^2\sigma^2}{2B}.$$

Take the full expectation, sum $t = 0, \dots, T-1$, and telescope:

$$\tfrac{\eta}{2}\sum_{t=0}^{T-1} \mathbb{E}\|g_t\|^2 \le F(\theta_0) - \mathbb{E}F(\theta_T) + \tfrac{L\eta^2\sigma^2 T}{2B} \le F(\theta_0) - F^\star + \tfrac{L\eta^2\sigma^2 T}{2B}.$$

Divide by $\eta T/2$:

$$\frac{1}{T}\sum_{t=0}^{T-1} \mathbb{E}\|g_t\|^2 \le \frac{2(F(\theta_0) - F^\star)}{\eta T} + \frac{L\eta\sigma^2}{B}.$$

Choose $\eta = c/\sqrt{T}$ (subject to $\eta \le 1/L$); both right-hand terms become $O(1/\sqrt{T})$, with $c = \sqrt{2(F(\theta_0) - F^\star)B/(L\sigma^2)}$ minimizing the bound. □

Remark 0.127. The average squared gradient norm decays like $1/\sqrt{T}$ and the noise floor is proportional to $\eta\sigma^2/B$. Larger batches permit a larger η at the same noise level.

Theorem 0.128 (SGD on L-smooth, μ-strongly convex F; sketch)**.** *With diminishing step $\eta_t = \frac{2}{\mu(t+t_0)}$ for t_0 large enough that $\eta_0 \le 1/L$,*

$$\mathbb{E}\|\theta_T - \theta^\star\|^2 \le \frac{C}{T}.$$

Sketch. Let $r_t^2 := \mathbb{E}\|\theta_t - \theta^\star\|^2$. Expand $\|\theta_{t+1} - \theta^\star\|^2 = \|\theta_t - \theta^\star - \eta_t\hat{g}_t\|^2$ and take expectation. Strong convexity gives $\langle g_t, \theta_t - \theta^\star\rangle \ge \mu\|\theta_t - \theta^\star\|^2$ (Chapter 7), and the noise contributes $\eta_t^2\sigma^2/B$:

$$r_{t+1}^2 \le (1 - \eta_t\mu)\, r_t^2 + \eta_t^2\sigma^2/B.$$

With $\eta_t\mu = 2/(t+t_0)$, induction $r_t^2 \le C/(t+t_0)$ closes for $C \ge 4\sigma^2/(\mu^2 B)$, giving $r_T^2 = O(1/T)$. □

Proposition 0.129 (Robbins–Monro conditions, 1951)**.** *For $\theta_{t+1} = \theta_t - \eta_t \hat{g}_t$ to converge almost surely to a stationary point of F under standard regularity, the step sizes must satisfy*

$$\sum_{t=0}^{\infty} \eta_t = \infty, \qquad \sum_{t=0}^{\infty} \eta_t^2 < \infty.$$

Remark 0.130. $\sum \eta_t = \infty$ guarantees the iterates travel far enough to escape any bounded region; $\sum \eta_t^2 < \infty$ controls the cumulative noise $\sum \eta_t \zeta_t$ in second moment (martingale-convergence). The schedule $\eta_t = c/(t+1)$ satisfies both; $\eta_t = c/\sqrt{t+1}$ fails square-summability; a constant step satisfies neither.

Code sketch

The accompanying notebook compares full-batch GD against SGD on a 1-D least-squares loss; empirically verifies $\mathrm{Var}(\hat{g}_B) \propto 1/B$ for $B \in \{1, 8, 64, 256\}$; runs SGD on the smooth non-convex toy $F(\theta) = \theta^2/2 + 0.5\sin(5\theta)$ to display $T^{-1/2}$ decay of the running average of $\|\nabla F(\theta_t)\|^2$; and contrasts a constant step (which plateaus at a noise floor) with $\eta_t = c/(t+1)$ (which converges).

Connection to LLMs

Pre-training a language model is mini-batch SGD on the cross-entropy loss (Chapter 11) of next-token prediction. The "batch size" of scaling-law papers is exactly the B of Theorem 13.1; doubling it halves the per-step gradient variance but doubles the FLOPs per step. The $O(1/\sqrt{T})$ rate of Theorem 13.2 is why pre-training, even of huge models, must consume *many* tokens — there is no $1/T$ shortcut without strong convexity. In practice we use AdamW (Chapter 14), which adds momentum and per-coordinate adaptive scaling on top of the SGD skeleton derived here; the convergence intuition (variance reduction by batching, noise floor $\propto \eta\sigma^2/B$, $1/\sqrt{T}$ asymptotics) carries over essentially unchanged.

Chapter 14: Momentum, RMSProp, AdamW: derivation and bias-correction proof

Motivation

Chapter 13 established that stochastic gradient descent (SGD) converges, but its rate is governed by the condition number $\kappa = L/\mu$ of the loss landscape. For deep networks — and for transformer language models in particular — κ can be astronomical, with curvature varying by many orders of magnitude across coordinates. Plain SGD wastes most of its updates fighting oscillation in stiff directions while crawling along flat ones. Chapter 14 builds the optimizers that fix both pathologies: *momentum* smooths the trajectory, *RMSProp* rescales per coordinate, *Adam* combines both with bias correction, and *AdamW* decouples weight decay so it survives Adam's rescaling. AdamW is the optimizer behind essentially every transformer pre-training run from GPT-2 onward (forward reference: Chapter 27).

Definitions

Definition 0.131 (Polyak's heavy-ball momentum)**.** *Given gradient estimate $g_t = \nabla f(\theta_t; \xi_t)$ and momentum coefficient $\beta \in [0, 1)$,*

$$v_{t+1} = \beta v_t + g_t, \qquad \theta_{t+1} = \theta_t - \eta\, v_{t+1},$$

with $v_0 = 0$. The state v_t accumulates an exponentially weighted history of past gradients.

Definition 0.132 (Nesterov accelerated gradient)**.** *NAG evaluates the gradient at the look-ahead point $\theta_t - \eta\beta v_t$ rather than at θ_t. Empirically it improves the constant factor on convex problems; in deep learning practice it is a minor variation on heavy-ball.*

Definition 0.133 (RMSProp)**.** *With $\beta_2 \in [0, 1)$ and small $\varepsilon > 0$,*

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2)\, g_t^{\odot 2}, \qquad \theta_{t+1} = \theta_t - \eta\, g_t \oslash (\sqrt{v_t} + \varepsilon),$$

where $\odot, \oslash$ are element-wise. RMSProp normalizes each coordinate by an EMA of its squared gradient.

Definition 0.134 (Adam)**.**

$$\begin{aligned} m_t &= \beta_1 m_{t-1} + (1 - \beta_1) g_t, \\ v_t &= \beta_2 v_{t-1} + (1 - \beta_2) g_t^{\odot 2}, \\ \hat{m}_t &= \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}, \\ \theta_{t+1} &= \theta_t - \eta\, \hat{m}_t \oslash (\sqrt{\hat{v}_t} + \varepsilon). \end{aligned}$$

Defaults: $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 10^{-8}$.

Definition 0.135 (AdamW: decoupled weight decay)**.**

$$\theta_{t+1} = \theta_t - \eta\Big(\hat{m}_t \oslash (\sqrt{\hat{v}_t} + \varepsilon) + \lambda\, \theta_t\Big).$$

The decay term is applied directly to the parameters and is not *funneled through the $\hat{v}_t$ rescaling.*

Theorems and proofs

Lemma 0.136 (EMA closed form)**.** *With $m_0 = 0$, the recursion $m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$ unrolls to*

$$m_t = (1 - \beta_1) \sum_{i=1}^{t} \beta_1^{t-i}\, g_i.$$

Proof. Induction on t. Base $t = 1$: $m_1 = (1 - \beta_1) g_1$. Step: assume the formula for $t - 1$; then

$$\begin{aligned} m_t &= \beta_1 \Big[(1 - \beta_1) \sum_{i=1}^{t-1} \beta_1^{t-1-i} g_i\Big] + (1 - \beta_1) g_t \\ &= (1 - \beta_1) \Big[\sum_{i=1}^{t-1} \beta_1^{t-i} g_i + g_t\Big] = (1 - \beta_1) \sum_{i=1}^{t} \beta_1^{t-i} g_i. \end{aligned}$$

□

Theorem 0.137 (Bias correction for Adam, first moment)**.** *Assume g_t is stationary with $\mathbb{E}[g_t] = g$. Then $\mathbb{E}[m_t] = (1 - \beta_1^t)\, g$, hence $\mathbb{E}[\hat{m}_t] = g$ for every $t \geq 1$.*

Proof. By the lemma and linearity of expectation,

$$\mathbb{E}[m_t] = (1 - \beta_1) \sum_{i=1}^{t} \beta_1^{t-i}\, \mathbb{E}[g_i] = (1 - \beta_1)\, g \sum_{j=0}^{t-1} \beta_1^j = (1 - \beta_1)\, g\, \frac{1 - \beta_1^t}{1 - \beta_1} = (1 - \beta_1^t)\, g.$$

Dividing by $1 - \beta_1^t$ yields $\mathbb{E}[\hat{m}_t] = g$. □

The identical argument with $g_t^{\odot 2}$ in place of g_t shows $\mathbb{E}[\hat{v}_t] = \mathbb{E}[g_t^{\odot 2}]$ under stationarity. Without bias correction, $m_1 = (1-\beta_1)g_1 \approx 0.1\, g$ at the default $\beta_1 = 0.9$ — a tenfold underestimate that would cripple the first hundred or so steps of training.

Proposition 0.138 (Momentum as a low-pass filter)**.** *For heavy-ball with $v_0 = 0$, $v_t = \sum_{i=1}^{t} \beta^{t-i} g_i$. The effective averaging horizon is $\sum_{j=0}^{\infty} \beta^j = 1/(1-\beta)$; e.g. $\beta = 0.9$ gives a horizon of about 10 steps. High-frequency gradient noise is attenuated; the consistent low-frequency signal accumulates.*

Proof. Identical unrolling to the Adam lemma but without the $(1-\beta)$ prefactor. The geometric series gives $\sum_{j=0}^{t-1} \beta^j \to 1/(1-\beta)$. □

Theorem 0.139 (Heavy-ball acceleration on convex quadratics, sketch)**.** *For $f(\theta) = \frac{1}{2}\theta^\top A\theta$ with $\mu I \preceq A \preceq LI$ and $\kappa = L/\mu$, GD requires $O(\kappa \log \frac{1}{\epsilon})$ steps to reach ϵ-accuracy, while heavy-ball with optimally tuned η, β achieves $O(\sqrt{\kappa} \log \frac{1}{\epsilon})$.*

Sketch. Diagonalize $A = U\Lambda U^\top$ and decompose θ_t in eigen-coordinates. In an eigen-direction with eigenvalue $\lambda \in [\mu, L]$ the heavy-ball recurrence reads

$$\begin{pmatrix} c_{t+1} \\ c_t \end{pmatrix} = M_\lambda \begin{pmatrix} c_t \\ c_{t-1} \end{pmatrix}, \qquad M_\lambda = \begin{pmatrix} 1+\beta-\eta\lambda & -\beta \\ 1 & 0 \end{pmatrix}.$$

The convergence rate is $\max_{\lambda \in [\mu, L]} \rho(M_\lambda)$, where ρ is the spectral radius. The eigenvalues of M_λ satisfy $z^2 - (1+\beta-\eta\lambda)z + \beta = 0$. With

$$\eta = \frac{4}{(\sqrt{L}+\sqrt{\mu})^2}, \qquad \beta = \left(\frac{\sqrt{L}-\sqrt{\mu}}{\sqrt{L}+\sqrt{\mu}}\right)^2,$$

both roots are complex conjugate with modulus $\sqrt{\beta} = 1 - 2/(\sqrt{\kappa}+1) = 1 - O(1/\sqrt{\kappa})$ uniformly in λ, giving the claimed $\sqrt{\kappa}$ acceleration. □

Proposition 0.140 (Decoupled weight decay $\neq L_2$ regularization for Adam)**.** *For SGD, augmenting the loss by $\frac{\lambda}{2}\|\theta\|^2$ adds $\lambda\theta$ to the gradient, producing the multiplicative shrinkage $\theta \mapsto (1-\eta\lambda)\theta$; the two formulations coincide. For Adam, the L_2 term enters g_t and is rescaled by $1/(\sqrt{\hat{v}_t}+\varepsilon)$: parameters with large historical gradients are decayed less, parameters with small historical gradients decayed more. AdamW restores the intended uniform shrinkage by applying $\lambda\theta_t$ outside the adaptive rescaling.*

Code sketch

```
def adam_step(theta, g, m, v, t, eta=1e-3, b1=0.9, b2=0.999, eps=1e-8):
    m = b1*m + (1 - b1)*g
    v = b2*v + (1 - b2)*g*g
    m_hat = m / (1 - b1**t)
    v_hat = v / (1 - b2**t)
    theta = theta - eta * m_hat / (np.sqrt(v_hat) + eps)
    return theta, m, v
```

AdamW differs by a single line: `theta = theta - eta*(m_hat/(np.sqrt(v_hat)+eps) + lam*theta)`.

Connection to LLMs

AdamW is the universal pre-training optimizer for transformers (GPT-2/3/4 family, Llama, Mistral, Gemma). Two facts from the analysis above explain why. (i) Per-coordinate rescaling matters: embedding rows, LayerNorm scales, and dense projection weights see gradients differing in magnitude by 3–6 orders of magnitude; only adaptive optimizers train them all under one global learning rate. (ii) Decoupled weight decay is essential for generalization at scale — with vanilla L_2-Adam, infrequently-updated parameters such as rare-token embeddings would be barely regularized while high-gradient parameters would be over-shrunk. Bias correction matters most in the first $\sim 10^1$–10^3 steps; the $1/(1-\beta_1)$ effective horizon explains why warmup of a few hundred steps is standard. Concrete AdamW configuration ($\lambda = 0.1$, $\beta_2 = 0.95$, gradient clipping, warmup-then-decay schedule) is taken up in Chapter 27.

Block D — Neural Networks

Chapter 15: MLPs as compositional functions; universal approximation

Motivation

A linear map $x \mapsto Wx+b$ (Chapter 5) can only carve $\mathbb{R}^n$ into half-spaces and stretch them affinely. The continuous functions on a compact set $K \subset \mathbb{R}^n$ (Chapter 4) form a vastly richer space. To bridge the gap we interleave linear maps with a fixed pointwise nonlinearity. The resulting object — a *multilayer perceptron* (MLP) — is dense in $C(K)$. This is the engine inside every transformer's feed-forward block.

Definitions

Definition 0.141 (Multilayer perceptron)**.** *Let $L \geq 1$ and widths $d_0, d_1, \ldots, d_L \in \mathbb{N}$. A* multilayer perceptron of depth L *with activation $\sigma : \mathbb{R} \to \mathbb{R}$ (applied componentwise) is the function $f : \mathbb{R}^{d_0} \to \mathbb{R}^{d_L}$ defined by*

$$\begin{aligned} h^{(0)} &= x, \\ h^{(\ell)} &= \sigma\big(W^{(\ell)} h^{(\ell-1)} + b^{(\ell)}\big), \quad 1 \leq \ell \leq L-1, \\ f(x) &= W^{(L)} h^{(L-1)} + b^{(L)}, \end{aligned}$$

where $W^{(\ell)} \in \mathbb{R}^{d_\ell \times d_{\ell-1}}$ and $b^{(\ell)} \in \mathbb{R}^{d_\ell}$. We call $\max_{0 \leq \ell \leq L} d_\ell$ the width *and L the* depth*.*

Definition 0.142 (Universal approximator)**.** *A class $\mathcal{F} \subset C(K, \mathbb{R})$ is* dense *in $C(K)$ — equivalently a* universal approximator *— if*

$$\forall f \in C(K)\ \ \forall \varepsilon > 0\ \ \exists g \in \mathcal{F} : \ \sup_{x \in K} |f(x) - g(x)| < \varepsilon.$$

Definition 0.143 (Sigmoidal, discriminatory)**.** *A measurable $\sigma : \mathbb{R} \to \mathbb{R}$ is* sigmoidal *if $\sigma(t) \to 1$ as $t \to +\infty$ and $\sigma(t) \to 0$ as $t \to -\infty$. It is* discriminatory *on $K = [0,1]^n$ if for every finite signed regular Borel measure μ on K,*

$$\int_K \sigma(w^\top x + b)\, d\mu(x) = 0 \ \ \forall\, w \in \mathbb{R}^n,\ b \in \mathbb{R} \ \implies \ \mu = 0.$$

Theorems and proofs

Theorem 0.144 (Cybenko 1989; Hornik 1989)**.** *Let $\sigma : \mathbb{R} \to \mathbb{R}$ be a continuous sigmoidal function and let $K \subset \mathbb{R}^n$ be compact. The class*

$$\mathcal{F}_\sigma = \Big\{ g(x) = \sum_{j=1}^{N} \alpha_j \, \sigma(w_j^\top x + b_j) : N \in \mathbb{N},\, \alpha_j, b_j \in \mathbb{R},\, w_j \in \mathbb{R}^n \Big\}$$

is dense in $C(K)$.

Proof sketch (Hahn–Banach + Riesz). Suppose, for contradiction, $\mathcal{F}_\sigma$ is *not* dense in $C(K)$. Then $S := \overline{\mathcal{F}_\sigma}$ is a proper closed subspace of the Banach space $\big(C(K), \|\cdot\|_\infty\big)$. Pick $f_0 \in C(K) \setminus S$. By the Hahn–Banach theorem there exists a continuous linear functional $\Lambda : C(K) \to \mathbb{R}$ with

$$\Lambda \not\equiv 0, \qquad \Lambda(g) = 0 \text{ for all } g \in S.$$

The Riesz–Markov representation theorem identifies $C(K)^*$ with the space of finite signed regular Borel measures on K: there exists a nonzero such measure μ with

$$\Lambda(g) = \int_K g(x)\, d\mu(x), \qquad g \in C(K).$$

Each ridge function $x \mapsto \sigma(w^\top x + b)$ lies in $\mathcal{F}_\sigma \subset S$, hence

$$\int_K \sigma(w^\top x + b)\, d\mu(x) = 0 \qquad \forall w \in \mathbb{R}^n,\ b \in \mathbb{R}.$$

Cybenko's lemma asserts that every continuous sigmoidal σ is discriminatory. (Sketch: as $\lambda \to \infty$, $\sigma(\lambda(w^\top x + b) + \varphi)$ converges pointwise (and boundedly) to a step function of the half-space $\{w^\top x + b > 0\}$; the dominated-convergence theorem transfers vanishing of the ridge integrals to vanishing of μ on every half-space, and finite signed measures are determined by their values on half-spaces, e.g. via the Fourier transform.) Therefore $\mu = 0$, contradicting $\Lambda \not\equiv 0$. Hence $\mathcal{F}_\sigma$ is dense. □

Proposition 0.145 (Linear-only networks collapse)**.** *If $\sigma = \mathrm{id}$, every depth-L MLP is itself a single affine map. In particular, the class is not dense in $C(K)$ whenever K contains a nontrivial open set.*

Proof. By induction on ℓ, $h^{(\ell)} = W^{(\ell)} h^{(\ell-1)} + b^{(\ell)}$. Unrolling,

$$f(x) = W' x + b', \qquad W' = W^{(L)} W^{(L-1)} \cdots W^{(1)}, \qquad b' = \sum_{\ell=1}^{L} \Big(\prod_{k=L}^{\ell+1} W^{(k)} \Big) b^{(\ell)},$$

where the empty product (when $\ell = L$) is the identity. Thus f is affine. An affine f cannot uniformly approximate, e.g., $x \mapsto \sin(2\pi x)$ on $[-1, 1]$ to error $< 1/4$, since the best affine approximant on a symmetric interval is 0 and $\|\sin(2\pi\cdot)\|_\infty = 1$. □

Remark 0.146. Proposition 0.145 shows the nonlinearity σ is not cosmetic: *depth without nonlinearity buys nothing.* UAT therefore relies on σ being genuinely nonlinear.

Theorem 0.147 (Depth separation, Telgarsky 2016)**.** *For every $k \in \mathbb{N}$ there is a function $f_k : [0, 1] \to [0, 1]$ realizable* exactly *by a ReLU network of depth $O(k^3)$ and width $O(1)$ such that any ReLU network of depth at most k that approximates f_k in $L^1([0, 1])$ to error $< 1/32$ must have width $\geq 2^k$.*

Proof sketch. Let $\Delta : [0,1] \to [0,1]$ be the triangular sawtooth $\Delta(x) = 2x$ for $x \in [0, 1/2]$, $\Delta(x) = 2(1-x)$ for $x \in [1/2, 1]$, and define $f_k = \Delta^{\circ k}$. Each composition doubles the number of monotone pieces, so f_k has 2^k such pieces and L^1-distance $1/4$ from any function with $\leq 2^{k-1}$ pieces. A ReLU network of width w and depth d realizes a continuous piecewise-linear function with at most $(2w)^d$ pieces (each ReLU at most doubles the count). Forcing $(2w)^d \geq 2^{k-1}$ at $d = k$ yields $w \geq 2^{(k-1)/k-1} \cdot 2^{k/k}$; a careful counting argument tightens this to the stated 2^k. The composition $\Delta^{\circ k}$ on the other hand is realised by stacking k copies of the constant-width gadget that implements Δ. □

Remark 0.148. Theorem 0.144 says shallow nets *can* represent everything; Theorem 0.147 says compositionality (depth) is *exponentially more parameter-efficient* for hierarchical targets. Modern deep learning lives in the gap between these two statements.

Code sketch

The companion notebook `cells.json`: (i) hand-builds a numpy MLP with tanh activation; (ii) fits a 32-hidden-unit MLP to $\sin(2\pi x)$ on $[-1, 1]$ via least-squares on hidden features and verifies $\sup_x |f(x) - \hat{f}(x)| < 0.05$; (iii) compares width-32-depth-1 against width-8-depth-3 at matched parameter budget and observes lower error from the deeper net; (iv) numerically confirms the linear-only collapse $f(x) = W'x + b'$ predicted by Proposition 0.145.

Connection to LLMs

Every transformer block (Chapter 23, forthcoming) contains a position-wise feed-forward module

$$\mathrm{FFN}(x) = W_2\, \sigma(W_1 x + b_1) + b_2,$$

i.e. a single-hidden-layer MLP applied independently to each token. This is exactly the universal approximator of Theorem 0.144: attention mixes information *across* positions, while the FFN does the nonlinear pointwise computation that — by Cybenko — can in principle realise any continuous transformation of the embedding. LLMs make W_1 wide ($4d$ or $8d$ inner dimension) precisely to exploit Theorem 0.144's expressive guarantee, while stacking L such blocks exploits Theorem 0.147's exponential depth advantage.

Chapter 16: Activation functions: ReLU/GELU/softmax with derivatives

Motivation

A multi-layer perceptron (Chapter 15) without a nonlinear activation collapses into a single affine map: $W_2(W_1 x + b_1) + b_2 = (W_2 W_1)x + (W_2 b_1 + b_2)$. Universal approximation requires a pointwise nonlinearity ϕ between affine layers. The choice of ϕ controls the gradient signal that backpropagation (Chapter 3) is allowed to push through the network, the geometry of hidden activations, and the numerical conditioning of training. We derive the five activations that dominate modern deep learning—sigmoid, tanh, ReLU, GELU, and softmax—computing each derivative or Jacobian from first principles and analyzing saturation.

Definitions

Definition 0.149 (Sigmoid)**.** $\sigma : \mathbb{R} \to (0,1)$, $\sigma(x) = \dfrac{1}{1+e^{-x}}$.

Definition 0.150 (Hyperbolic tangent)**.** $\tanh(x) = \dfrac{e^{x}-e^{-x}}{e^{x}+e^{-x}} = 2\sigma(2x) - 1$.

Definition 0.151 (ReLU)**.** $\mathrm{ReLU}(x) = \max(0, x)$. *Differentiable except at $x = 0$; the Clarke subdifferential at 0 is $[0, 1]$.*

Definition 0.152 (GELU)**.** $\mathrm{GELU}(x) = x\,\Phi(x)$, *where* $\Phi(x) = \frac{1}{2}\big(1 + \operatorname{erf}(x/\sqrt{2})\big)$ *is the standard-normal CDF. The tanh-approximation is*

$$\mathrm{GELU}(x) \;\approx\; \tfrac{1}{2}x\big(1 + \tanh(\sqrt{2/\pi}\,(x + 0.044715\,x^{3}))\big).$$

Definition 0.153 (Softmax)**.** $\mathrm{softmax} : \mathbb{R}^{n} \to \Delta^{n-1}$, $\mathrm{softmax}(z)_i = \dfrac{e^{z_i}}{\sum_{k=1}^{n} e^{z_k}}$.

Theorems and Proofs

Theorem 0.154 (Sigmoid derivative)**.** $\sigma'(x) = \sigma(x)\big(1 - \sigma(x)\big)$.

Proof. Write $\sigma(x) = (1 + e^{-x})^{-1}$. By the chain rule, $\sigma'(x) = -(1 + e^{-x})^{-2} \cdot (-e^{-x}) = \dfrac{e^{-x}}{(1+e^{-x})^{2}}$. Factor:

$$\frac{e^{-x}}{(1+e^{-x})^{2}} = \frac{1}{1+e^{-x}} \cdot \frac{e^{-x}}{1+e^{-x}} = \sigma(x)\big(1-\sigma(x)\big),$$

using $\frac{e^{-x}}{1+e^{-x}} = 1 - \sigma(x)$. □

Theorem 0.155 (Tanh derivative)**.** $\tanh'(x) = 1 - \tanh^{2}(x)$.

Proof. From $\tanh(x) = 2\sigma(2x) - 1$ and Theorem 0.154, $\tanh'(x) = 4\sigma(2x)(1 - \sigma(2x))$. Substituting $\sigma(2x) = (\tanh x + 1)/2$,

$$4 \cdot \tfrac{\tanh x + 1}{2} \cdot \tfrac{1 - \tanh x}{2} = (1 + \tanh x)(1 - \tanh x) = 1 - \tanh^{2} x.$$ □

Theorem 0.156 (ReLU subdifferential)**.** *For $x \neq 0$,* $\mathrm{ReLU}'(x) = \mathbf{1}_{x>0}$. *At $x = 0$,* $\partial\,\mathrm{ReLU}(0) = [0, 1]$.

Proof. For $x > 0$, $\mathrm{ReLU}(x) = x$ has derivative 1. For $x < 0$, derivative 0. At $x = 0$ the left derivative is 0 and the right derivative is 1; the convex hull of these one-sided limits, $[0, 1]$, defines the Clarke subdifferential. PyTorch and JAX conventionally pick 0. □

Theorem 0.157 (GELU derivative)**.** $\mathrm{GELU}'(x) = \Phi(x) + x\,\phi(x)$, *where* $\phi(x) = \frac{1}{\sqrt{2\pi}} e^{-x^{2}/2}$.

Proof. By the product rule, $\frac{d}{dx}\big[x\Phi(x)\big] = \Phi(x) + x\,\Phi'(x)$. The fundamental theorem of calculus gives $\Phi'(x) = \phi(x)$. □

Theorem 0.158 (Softmax Jacobian)**.** *Let $s = \mathrm{softmax}(z)$ and $S = \sum_k e^{z_k}$. Then*

$$\frac{\partial s_i}{\partial z_j} = s_i(\delta_{ij} - s_j), \qquad J(z) = \operatorname{diag}(s) - ss^{\top}.$$

Proof. By the quotient rule,

$$\frac{\partial s_i}{\partial z_j} = \frac{(\partial e^{z_i}/\partial z_j)\, S - e^{z_i}\,(\partial S/\partial z_j)}{S^2} = \frac{\delta_{ij}\, e^{z_i} S - e^{z_i} e^{z_j}}{S^2} = \delta_{ij}\, s_i - s_i s_j. \qquad \square$$

The matrix $J = \operatorname{diag}(s) - ss^\top$ is symmetric PSD with $J\mathbf{1} = 0$, reflecting softmax's translation invariance.

Proposition 0.159 (Saturation)**.** *$\sigma'(x), \tanh'(x) \to 0$ as $|x| \to \infty$, so backpropagated gradients $\prod_\ell \phi'(z_\ell)$ shrink geometrically in deep networks (vanishing gradients). ReLU avoids saturation for $x > 0$ but a unit with $z \le 0$ on every training input is permanently dead. GELU is smooth;* $\mathrm{GELU}'(x) \to 1$ *as $x \to +\infty$ (since $\Phi \to 1$, $x\phi \to 0$) and $\to 0$ as $x \to -\infty$, combining ReLU-like sparsity with smoothness.*

Theorem 0.160 (Temperature limits)**.** *Let $T > 0$ and assume $i^* = \arg\max_i z_i$ is unique. Then*

$$\lim_{T\to 0^+} \mathrm{softmax}(z/T)_i = \mathbf{1}_{i=i^*}, \qquad \lim_{T\to\infty} \mathrm{softmax}(z/T)_i = \tfrac{1}{n}.$$

Proof. Write $\mathrm{softmax}(z/T)_i = \dfrac{e^{(z_i - z_{i^*})/T}}{\sum_k e^{(z_k - z_{i^*})/T}}$. As $T \to 0^+$, every term with $k \neq i^*$ has exponent $\to -\infty$, so $\to 0$, while $k = i^*$ contributes 1, yielding $\mathbf{1}_{i=i^*}$. As $T \to \infty$, every exponent $\to 0$, so each term $\to 1$ and the ratio $\to 1/n$. $\square$

Proposition 0.161 (Numerical stability)**.** $\mathrm{softmax}(z) = \mathrm{softmax}(z - c\mathbf{1})$ *for any c. With $c = \max_i z_i$, all exponents are ≤ 0 and overflow is avoided.*

Proof. $\dfrac{e^{z_i - c}}{\sum_k e^{z_k - c}} = \dfrac{e^{-c} e^{z_i}}{e^{-c} \sum_k e^{z_k}} = \mathrm{softmax}(z)_i.$ $\square$

Corollary 0.162 (Log-sum-exp)**.** $\log \sum_j e^{z_j} = \max_j z_j + \log \sum_j e^{z_j - \max_k z_k}$.

Code Sketch

The companion notebook plots all five activations on $[-4, 4]$ and tabulates them at $x \in \{-2, -1, 0, 1, 2\}$; verifies each analytic derivative against centered finite differences; implements `softmax_jacobian(z) = np.diag(s) - np.outer(s,s)` and confirms it matches a numerical Jacobian for $z = (1, 0.5, -1, 2)$; sweeps $T \in \{0.1, 1, 5, 100\}$ to visualize the temperature limits; and shows that naive softmax of $(1000, 1001, 1002)$ overflows while the stabilized version returns $(0.0900, 0.2447, 0.6652)$.

Connection to LLMs

GELU is the standard feedforward activation in GPT-2/3 and BERT, applied elementwise to the $4d_{\text{model}}$ hidden state of every Transformer MLP block (Chapter 15). Llama-family models replace it with SwiGLU, a gated variant covered in Chapter 17. Softmax appears in two distinct loci of an LLM:

1. *Attention* (Chapter 21): row-wise $\mathrm{softmax}(qK^\top/\sqrt{d_k})$ converts compatibility scores into a distribution over keys.
2. *Output head* (Chapter 25): final vocabulary logits are passed through softmax to produce $p(x_{t+1} \mid x_{\le t})$. The temperature limits (Theorem above) are the standard sampling knob: $T \to 0$ recovers greedy decoding, $T \to \infty$ uniform random sampling.

Both invocations use the stabilized form of Proposition above in production kernels (FlashAttention, fused log-softmax + cross-entropy), making the "subtract the max" identity load-bearing for trillion-parameter training.

Chapter 17: Loss functions: MSE, cross-entropy; gradients from first principles

Motivation

A neural network is a parametric map $f_\theta : \mathcal{X} \to \mathcal{Y}$. Training requires a scalar "badness" score whose gradient with respect to θ tells us how to nudge weights. That scalar is the *loss function*. Two losses dominate modern deep learning: **mean squared error** (MSE) for regression, and **categorical cross-entropy** (CE) for classification, including next-token prediction in every language model. Neither is arbitrary: each is the negative log-likelihood of a generative model (Ch. 12), and in both cases the gradient with respect to the network's pre-activation output collapses to $\hat{p} - y$, an identity that makes backpropagation through deep stacks tractable.

Definitions

Definition 0.163 (MSE loss)**.** *For scalar prediction $\hat{y}$ and target y,*

$$\ell_{\mathrm{MSE}}(y, \hat{y}) \;=\; \tfrac{1}{2}(y - \hat{y})^2.$$

The factor $\frac{1}{2}$ is conventional; it cancels in the gradient.

Definition 0.164 (Categorical cross-entropy)**.** *For a one-hot label $y \in \{0,1\}^K$ with $\sum_k y_k = 1$ and a predicted distribution $\hat{p} \in \Delta^{K-1}$,*

$$\ell_{\mathrm{CE}}(y, \hat{p}) \;=\; -\sum_{k=1}^{K} y_k \log \hat{p}_k.$$

This is $H(y, \hat{p})$ from Ch. 11 evaluated on a single example.

Definition 0.165 (Cross-entropy with logits)**.** *For logits $z \in \mathbb{R}^K$ and true class $c \in \{1, \dots, K\}$,*

$$\ell(z, c) \;=\; -\log \mathrm{softmax}(z)_c \;=\; -z_c + \log \sum_{j=1}^{K} e^{z_j}.$$

The right-hand log-sum-exp form is the only numerically stable evaluation route.

Definition 0.166 (Binary cross-entropy)**.** *For $K = 2$ with logit z, $\hat{p} = \sigma(z)$, and $y \in \{0, 1\}$,*

$$\ell(z, y) \;=\; -y \log \sigma(z) - (1 - y) \log(1 - \sigma(z)).$$

Theorems and proofs

Theorem 0.167 (MSE gradient)**.** $\nabla_{\hat{y}}\, \ell_{\mathrm{MSE}}(y, \hat{y}) = \hat{y} - y$.

Proof. $\frac{d}{d\hat{y}} \left[\frac{1}{2}(y - \hat{y})^2\right] = -(y - \hat{y}) = \hat{y} - y$. □

Theorem 0.168 (MSE = MLE under Gaussian noise)**.** *If $y = f_\theta(x) + \varepsilon$ with $\varepsilon \sim \mathcal{N}(0, \sigma^2)$, then maximum likelihood estimation is equivalent to MSE minimization.*

Proof. The Gaussian density gives

$$-\log p_\theta(y \mid x) \;=\; \tfrac{1}{2}\log(2\pi\sigma^2) + \frac{(y - f_\theta(x))^2}{2\sigma^2} \;=\; \frac{1}{2\sigma^2}(y - f_\theta(x))^2 + C,$$

where C is independent of θ. Summing across an i.i.d. dataset and dropping the additive constant and the positive multiplier $1/\sigma^2$ yields $\sum_n \frac{1}{2}(y_n - f_\theta(x_n))^2$, the MSE objective. The argmin in θ coincides with the argmax of the log-likelihood. □

Theorem 0.169 (Categorical CE = MLE under softmax)**.** *Let $p_\theta(y = c \mid x) = \mathrm{softmax}(z_\theta(x))_c$. Then $-\log p_\theta(y = c \mid x) = \ell(z, c)$, so MLE summed over an i.i.d. dataset equals categorical CE minimization.*

Proof. Direct from the definition of $\ell(z, c)$ and the identity $\arg\max \log L = \arg\min H_{\text{cross}}$ proved in Ch. 12. □

Theorem 0.170 (Softmax + CE gradient)**.** *For $\ell(z, c) = -z_c + \log \sum_j e^{z_j}$,*

$$\frac{\partial \ell}{\partial z_j} \;=\; \mathrm{softmax}(z)_j - \mathbf{1}_{[j=c]} \;=\; \hat{p}_j - y_j.$$

Proof. $\partial(-z_c)/\partial z_j = -\mathbf{1}_{[j=c]}$. For the log-sum-exp,

$$\frac{\partial}{\partial z_j} \log \sum_k e^{z_k} \;=\; \frac{e^{z_j}}{\sum_k e^{z_k}} \;=\; \mathrm{softmax}(z)_j.$$

Adding gives $\hat{p}_j - \mathbf{1}_{[j=c]}$. The cancellation occurs because the $\sum_k e^{z_k}$ in softmax's denominator is the *same* normalizer that appears inside ℓ; the matrix-vector product implicit in the softmax Jacobian (Ch. 16) collapses to a vector subtraction. One subtraction per output unit replaces a $K \times K$ matvec. □

Theorem 0.171 (Binary CE + sigmoid gradient)**.** *For $\ell(z, y) = -y \log \sigma(z) - (1-y)\log(1-\sigma(z))$, $\partial \ell / \partial z = \sigma(z) - y$.*

Proof. Use $\log \sigma(z) = -\log(1 + e^{-z})$ and $\log(1 - \sigma(z)) = -z - \log(1 + e^{-z})$, so $\ell = (1 - y)\, z + \log(1 + e^{-z})$. Differentiating,

$$\frac{\partial \ell}{\partial z} \;=\; (1 - y) - \frac{e^{-z}}{1 + e^{-z}} \;=\; (1 - y) - (1 - \sigma(z)) \;=\; \sigma(z) - y. \qquad \square$$

Theorem 0.172 (CE minimum at $\hat{p} = y$)**.** *Treating $y, \hat{p} \in \Delta^{K-1}$ as distributions, $H(y, \hat{p})$ is minimized over $\hat{p}$ at $\hat{p} = y$.*

Proof. $H(y, \hat{p}) - H(y, y) = -\sum_k y_k \log(\hat{p}_k / y_k) = D_{\mathrm{KL}}(y \,\|\, \hat{p}) \geq 0$ by Gibbs' inequality (Ch. 11), with equality iff $\hat{p} = y$. □

Code sketch

```
def softmax_ce_with_logits(z, c):
    z = z - z.max()
    lse = np.log(np.exp(z).sum())
    loss = -z[c] + lse
    p = np.exp(z - lse)
    grad = p.copy(); grad[c] -= 1.0      # p - y
    return loss, grad
```

The forward and backward share the cached softmax; the backward is a single in-place subtraction.

Connection to LLMs

A causal language model (Ch. 25) emits, at every position t, logits $z_t \in \mathbb{R}^V$ over a vocabulary of size V. The training objective on a sequence $x_1, \ldots, x_T$ is

$$\mathcal{L}(\theta) \;=\; \sum_{t=1}^{T} -\log p_\theta(x_t \mid x_{<t}) \;=\; \sum_{t=1}^{T} \ell(z_t, x_t),$$

i.e. cross-entropy with logits, summed across positions. By Theorem 4, the gradient at the output layer is $\hat{p}_t - y_t$ at every position — a single subtraction per token per vocabulary entry. This signal flows backwards through the unembedding, the transformer blocks (Ch. 27 forward), and into the token embeddings. With $V \approx 10^5$, evaluating the softmax Jacobian explicitly would be prohibitive; the $\hat{p} - y$ collapse is what makes large-scale language modeling computationally feasible.

Remark 0.173. The pattern "output-side gradient = prediction − target" generalizes well beyond softmax-CE and sigmoid-BCE: it is the canonical gradient for any *matched* pairing of an exponential-family likelihood with its natural-parameter activation (a fact we will revisit when discussing conjugate priors and generalized linear models).

Chapter 18: Backpropagation: chain rule applied; reverse-mode AD as a graph algorithm

Motivation

Chapter 17 ended with a clean fact: for softmax + cross-entropy, the gradient of the loss with respect to the pre-softmax logits is $\hat{p} - y$. That gives us the *output-layer* gradient. To train an actual network we need the gradient of the loss with respect to *every* parameter. Symbolic differentiation produces expressions whose size explodes exponentially with depth; finite differences over P parameters cost $O(P)$ forward passes, prohibitive when $P \approx 10^{11}$. Backpropagation solves both: it is the multivariate chain rule (Ch. 4) applied along a DAG in the right order.

Computational graphs and AD

Definition 0.174 (Computational graph)**.** *A* computational graph *is a DAG whose nodes are intermediate values $v_i \in \mathbb{R}^{d_i}$ and whose edges are elementary operations. Source nodes are inputs; sink nodes are outputs. A* forward pass *computes all v_i in topological order given the inputs.*

Definition 0.175 (Forward-mode AD / JVP)**.** *Given a tangent $\dot{x}$, forward-mode AD propagates $\dot{v}_i = \sum_{j \in \text{parents}(i)} (\partial v_i / \partial v_j)\, \dot{v}_j$ in topological order. One sweep evaluates $J\dot{x}$. Cost: one sweep per* input *dimension.*

Definition 0.176 (Reverse-mode AD / VJP)**.** *Given a cotangent $\bar{y}$ at the output, reverse-mode AD propagates* adjoints *$\bar{v}_i$ in reverse topological order:*

$$\bar{v}_j \;=\; \sum_{i \in \text{children}(j)} \Big(\frac{\partial v_i}{\partial v_j}\Big)^T \bar{v}_i.$$

One sweep evaluates $J^T \bar{y}$. Cost: one sweep per output *dimension. For a scalar loss L, a single backward pass yields $\nabla_x L$; the adjoint $\bar{v}_i := \partial L / \partial v_i$ is the central object of backprop.*

The backprop theorem

Theorem 0.177 (Backprop = reverse-mode AD on a feedforward net)**.** *Let $L(x) = \ell(h^{(L)})$ where $h^{(\ell)} = f^{(\ell)}(h^{(\ell-1)})$, $h^{(0)} = x$, and each $f^{(\ell)}$ is differentiable with Jacobian $J^{(\ell)} := \partial f^{(\ell)} / \partial h^{(\ell-1)}$. Define adjoints $\bar{h}^{(\ell)} := \partial L / \partial h^{(\ell)}$. Then*

$$\bar{h}^{(L)} \;=\; \nabla \ell(h^{(L)}), \qquad \bar{h}^{(\ell-1)} \;=\; \big(J^{(\ell)}\big)^T \bar{h}^{(\ell)}.$$

Proof. By the multivariate chain rule (Ch. 4), for any ℓ,

$$\frac{\partial L}{\partial h^{(\ell-1)}} \;=\; \frac{\partial L}{\partial h^{(\ell)}} \cdot \frac{\partial h^{(\ell)}}{\partial h^{(\ell-1)}} \;=\; \bar{h}^{(\ell)}\, J^{(\ell)},$$

which transposed gives the adjoint recurrence. The base case is the gradient of ℓ at $h^{(L)}$. Induction over $\ell = L, L-1, \ldots, 1$. □

Remark 0.178. For Ch. 17's softmax+cross-entropy block, $\bar{h}^{(L)} = \hat{p} - y$ is the base case; the recurrence does the rest.

VJPs for an MLP layer

Proposition 0.179 (MLP layer VJPs)**.** *For $z^{(\ell)} = W^{(\ell)} h^{(\ell-1)} + b^{(\ell)}$, $h^{(\ell)} = \sigma(z^{(\ell)})$ with elementwise σ,*

$$\bar{z}^{(\ell)} \;=\; \bar{h}^{(\ell)} \odot \sigma'(z^{(\ell)}), \quad \bar{W}^{(\ell)} \;=\; \bar{z}^{(\ell)} (h^{(\ell-1)})^T, \quad \bar{b}^{(\ell)} \;=\; \bar{z}^{(\ell)}, \quad \bar{h}^{(\ell-1)} \;=\; (W^{(\ell)})^T \bar{z}^{(\ell)}.$$

Proof. The chain rule gives $\bar{z}_i^{(\ell)} = \sum_k \bar{h}_k^{(\ell)}\, \partial h_k^{(\ell)} / \partial z_i^{(\ell)} = \bar{h}_i^{(\ell)}\, \sigma'(z_i^{(\ell)})$ since σ is elementwise. For W, $z_i = \sum_j W_{ij} h_j^{(\ell-1)}$ so $\partial z_i / \partial W_{kj} = \delta_{ik} h_j^{(\ell-1)}$, hence $\bar{W}_{kj} = \bar{z}_k h_j^{(\ell-1)}$, an outer product. The bias is $\partial z_i / \partial b_j = \delta_{ij}$. For $h^{(\ell-1)}$, $\partial z_i / \partial h_j^{(\ell-1)} = W_{ij}$, giving $\bar{h}^{(\ell-1)} = W^T \bar{z}^{(\ell)}$. □

Cost and memory

Theorem 0.180 (Baur–Strassen)**.** *For a function defined by K elementary operations with bounded fan-out, the gradient can be computed in arithmetic cost at most $\sim 5K$ — i.e. the backward pass costs only a small constant times the forward pass,* independent of the number of parameters P.

This is the magic. We pay a constant overhead, not a P-fold one, to differentiate with respect to all P parameters simultaneously. Forward-mode AD does the opposite: $O(n)$ for n inputs, $O(1)$ in outputs.

Remark 0.181 (Memory). The recurrence $\bar{h}^{(\ell-1)} = (J^{(\ell)})^T \bar{h}^{(\ell)}$ requires evaluating $J^{(\ell)}$, which typically depends on the forward activation $h^{(\ell-1)}$ (e.g. via $\sigma'(z^{(\ell)})$). Hence reverse-mode AD must *cache every intermediate activation* produced during the forward pass, giving memory cost $O(L)$ in depth times batch and width. This is exactly the constraint that motivates *gradient checkpointing* (Ch. 27): drop activations and recompute them on backward, trading compute for memory.

Code sketch

Example 0.182 (Two-line backward for an MLP layer).

```
def linear_vjp(W, h_in, dz):
    return dz @ h_in.T, dz, W.T @ dz   # dW, db, dh_in

def tanh_vjp(z, dh):
    return dh * (1.0 - np.tanh(z)**2)
```

The full backward pass over L layers is L applications of these two VJPs in reverse, threading the adjoint $\bar{h}^{(\ell)}$ through.

Connection to LLMs

A modern transformer has $L \in [12, 96]$ blocks, each composed of attention and MLP sub-layers. One training step is: (i) forward pass through all L blocks, caching activations; (ii) reverse-mode AD computing $\nabla_\theta L$ for every parameter (often $\theta \in \mathbb{R}^{10^{11}}$) in time $\leq 5\times$ the forward pass — Baur–Strassen in action; (iii) optimizer step. The per-token activation memory is the dominant scaling constraint and is exactly why frameworks ship gradient checkpointing, FlashAttention recomputation (Ch. 23), and ZeRO-style sharding by default. Every parameter update in a 175B model is the adjoint recurrence above, executed billions of times.

Block E — Sequence Models and Attention

Chapter 19: Embeddings: token to vector; lookup as a linear map; weight tying

Motivation

A transformer language model never sees the symbol *cat*. It sees an integer index $v \in \{0, 1, \ldots, V-1\}$ which is then mapped to a vector in $\mathbb{R}^d$. That vector is the only representation downstream layers ever touch: attention, MLPs, normalizations, residuals — everything is linear algebra on $\mathbb{R}^d$. The gateway between the discrete vocabulary $\mathcal{V}$ (Chapter 1) and the continuous geometry of $\mathbb{R}^d$ is the *embedding matrix*. We will show that this gateway is not a fancy table lookup at all: it is the linear map of Chapter 5 applied to a one-hot vector. The trick of *weight tying* — sharing parameters between the input embedding and the output projection — then falls out as a natural identification.

Definitions

Definition 0.183 (One-hot encoding)**.** *For a finite vocabulary $\mathcal{V} = \{0, 1, \ldots, V-1\}$ and $v \in \mathcal{V}$, the* one-hot vector *is $e_v \in \{0,1\}^V$ with $e_v[i] = \mathbf{1}_{i=v}$. Equivalently, e_v is the v-th standard basis vector of $\mathbb{R}^V$.*

Definition 0.184 (Embedding matrix and lookup)**.** *An* embedding matrix *is a parameter* $E \in \mathbb{R}^{d\times V}$*. Its columns* $\{E_{:,0}, \dots, E_{:,V-1}\}$ *are the* token embeddings*; the integer* d *is the* embedding dimension *(also written* d_{model}*). The* embedding lookup *is* $\text{emb}\colon \mathcal{V} \to \mathbb{R}^d$*,* $\text{emb}(v) := Ee_v$*.*

Definition 0.185 (Output projection / unembedding)**.** *An* output projection *is a parameter* $U \in \mathbb{R}^{V\times d}$ *taking a hidden state* $h \in \mathbb{R}^d$ *to a logit vector* $z := Uh \in \mathbb{R}^V$*. The probability over* $\mathcal{V}$ *is* $\hat{p} = \text{softmax}(z)$ *(Chapter 9, 17).*

Definition 0.186 (Weight tying)**.** *A model with embedding* $E \in \mathbb{R}^{d\times V}$ *and output projection* $U \in \mathbb{R}^{V\times d}$ *is* weight-tied *(Press & Wolf 2016; Inan et al. 2017) if* $U = E^\top$*. The two layers then share the same* dV *scalar parameters.*

Theorems and proofs

Theorem 0.187 (Lookup is a linear map)**.** *For every* $v \in \{0, \dots, V-1\}$*,* $Ee_v = E_{:,v}$*.*

Proof. By the definition of matrix–vector multiplication (Chapter 5),

$$(Ee_v)_i \;=\; \sum_{j=0}^{V-1} E_{ij}\,(e_v)_j \;=\; \sum_{j=0}^{V-1} E_{ij}\,\mathbf{1}_{j=v} \;=\; E_{iv}$$

for every $i \in \{0, \dots, d-1\}$. Stacking these scalars gives the column $E_{:,v}$, so $Ee_v = E_{:,v}$. □

Remark 0.188. Embedding-table lookup is therefore a linear map $\mathbb{R}^V \to \mathbb{R}^d$ in disguise. Implementations skip the matmul and slice the column; the *meaning* is the matrix product.

Theorem 0.189 (Weight-tying gradient identity)**.** *Let* $E \in \mathbb{R}^{d\times V}$ *and consider a model*

$$f(v, E) \;=\; U\,h(Ee_v) \;=\; E^\top h(Ee_v),$$

with tied $U = E^\top$*, where* $h\colon \mathbb{R}^d \to \mathbb{R}^d$ *is a sub-network with no further dependence on* E*. Let* $\mathcal{L}$ *be a scalar loss applied to* $z = E^\top h$*. Then*

$$\nabla_E \mathcal{L} \;=\; \underbrace{h\,(\nabla_z \mathcal{L})^\top}_{\textit{output-side}} \;+\; \underbrace{\big(J_h^\top E\,\nabla_z \mathcal{L}\big)\,e_v^\top}_{\textit{input-side}},$$

where $J_h \in \mathbb{R}^{d\times d}$ *is the Jacobian of* h *at* Ee_v*. The input-side contribution is rank-one; only column* v *is nonzero.*

Proof. Treat E as occurring in two roles: an output role $U = E^\top$ and an input role inside $h \circ (E \cdot e_v)$. By the multivariate chain rule (Chapter 18), the total gradient is the sum of the partial gradients with respect to each role.

Output role. $z_k = \sum_i E_{ik} h_i$, so $\partial z_k / \partial E_{ij} = \mathbf{1}_{k=j} h_i$. Hence

$$\frac{\partial \mathcal{L}}{\partial E_{ij}}\Big|_{\text{out}} \;=\; \sum_k \frac{\partial \mathcal{L}}{\partial z_k}\frac{\partial z_k}{\partial E_{ij}} \;=\; h_i\,\frac{\partial \mathcal{L}}{\partial z_j},$$

giving the outer product $h\,(\nabla_z \mathcal{L})^\top \in \mathbb{R}^{d\times V}$.

Input role. $x := Ee_v$ has $x_i = E_{iv}$, so $\partial x_i / \partial E_{ab} = \mathbf{1}_{a=i}\mathbf{1}_{b=v}$. By the chain rule,

$$\frac{\partial \mathcal{L}}{\partial E_{ab}}\Big|_{\text{in}} \;=\; \sum_i \frac{\partial \mathcal{L}}{\partial x_i}\frac{\partial x_i}{\partial E_{ab}} \;=\; \frac{\partial \mathcal{L}}{\partial x_a}\,\mathbf{1}_{b=v}.$$

Now $\nabla_x \mathcal{L} = J_h^\top \nabla_h \mathcal{L} = J_h^\top (E\,\nabla_z \mathcal{L})$, since $\nabla_h \mathcal{L} = U^\top \nabla_z \mathcal{L} = E\,\nabla_z \mathcal{L}$. Hence the input-side contribution is the rank-one matrix $g\,e_v^\top$ with $g := J_h^\top E\,\nabla_z \mathcal{L} \in \mathbb{R}^d$, whose only nonzero column sits in position v. Summing the two roles yields the claim. □

Remark 0.190 (Sparsity of the input gradient). The factor $e_v^\top$ kills every column except column v. This is why embedding tables are usually updated with *sparse* gradients: each minibatch only touches the columns of the tokens it actually contains. The output-side term, by contrast, hits all V columns through $h(\nabla_z \mathcal{L})^\top$.

Remark 0.191 (Distributional semantics, Mikolov et al. 2013). During training, two tokens u, v that appear in similar surrounding contexts produce similar gradients on their embeddings (the context enters via $\nabla_z \mathcal{L}$ and via h). Iterating SGD, $E_{:,u}$ and $E_{:,v}$ drift toward similar locations in $\mathbb{R}^d$, so cosine similarity of embeddings tracks distributional similarity in the corpus. This is an empirical claim, not a theorem; we illustrate it numerically in the code cells.

Theorem 0.192 (Dimensionality bound)**.** *If $V > d$, no choice of $E \in \mathbb{R}^{d \times V}$ makes the columns $E_{:,0}, \ldots, E_{:,V-1}$ pairwise orthogonal and nonzero.*

Proof. Pairwise orthogonal nonzero vectors in any inner-product space are linearly independent: from $\sum_v c_v E_{:,v} = 0$, take the inner product with $E_{:,u}$ to get $c_u \|E_{:,u}\|^2 = 0$, hence $c_u = 0$. The columns would then be a linearly independent set of size V in $\mathbb{R}^d$. By Chapter 5 (invariance of dimension), every independent set in $\mathbb{R}^d$ has size $\leq d$, so $V \leq d$, contradicting $V > d$. □

In LLMs we always have $V \gg d$ (e.g. $V \approx 5 \cdot 10^4$ versus $d \approx 4096$), so embeddings cannot be mutually orthogonal — they form an *overcomplete* set, and "near-orthogonality" is the best one can ask.

Code sketch

We build a tiny embedding matrix and verify $Ee_v = E_{:,v}$ exactly. We then construct tied $U = E^\top$ and check that the logit for token v equals $\langle E_{:,v}, h \rangle$. A Mikolov-style skip-gram on a synthetic corpus shows that embeddings of co-occurring tokens develop higher cosine similarity than non-co-occurring ones. Finally, we verify Theorem 0.192 by attempting to orthogonalize $V = 8$ vectors in $\mathbb{R}^3$ via Gram–Schmidt and observing dimension exhaustion.

Connection to LLMs

Every transformer language model contains exactly the data structures of this chapter: an input embedding $E \in \mathbb{R}^{d \times V}$ and an output projection $U \in \mathbb{R}^{V \times d}$. The embedding dimension d is the *model dimension* d_{model}, which dominates parameter counts and FLOPs (Chapters 23–25). GPT-2 (Radford et al. 2019), Llama 1/2/3 (Touvron et al. 2023), and most open-weight LMs use weight tying $U = E^\top$, halving embedding parameters and empirically improving perplexity. GPT-3/4 use an LM-head initialized from $E^\top$ but trained separately. Once a token is embedded, position information is added (Chapter 20), and the residual stream begins (Chapter 22). The final logit projection Uh_{final} is the same matrix E that began the forward pass, viewed from a different side.

Chapter 20: RNN intuition; vanishing-gradient proof; why we need attention

Motivation

Feedforward networks (Chapters 17–19) are fixed-depth pipelines, but language, audio, and code are *sequences*. We need a model that consumes one token at a time, accumulates context, and emits one prediction per step. The Recurrent Neural Network (RNN) was the dominant answer from the

late 1980s through 2017. We derive its dynamics, prove the *vanishing/exploding gradient theorem* (Pascanu, Mikolov, Bengio 2013) using the SVD of Chapter 6 and the chain rule of Chapter 18, and show that this failure mode—together with an information bottleneck—is the precise reason attention (Chapters 21–24) was invented.

Definitions

Definition 0.193 (Sequence model)**.** *A causal map $f : \mathcal{X}^T \to \mathcal{Y}^T$ such that $y_t = f_t(x_1, \dots, x_t)$ for every t.*

Definition 0.194 (RNN cell)**.** *Fix hidden dimension d and parameters $W_h \in \mathbb{R}^{d\times d}$, $W_x \in \mathbb{R}^{d\times \dim x}$, $b \in \mathbb{R}^d$, and a pointwise nonlinearity σ. The recursion is*

$$h_t = \sigma(W_h h_{t-1} + W_x x_t + b), \qquad h_0 = 0,$$

with output $y_t = W_y h_t$.

Definition 0.195 (Backpropagation through time (BPTT))**.** *Unroll the recursion into an acyclic graph of depth T with the parameters (W_h, W_x, b) shared across every time step, then apply the backprop algorithm of Chapter 18 to that unrolled graph. The gradient of a shared weight is the sum of its per-step gradients.*

Theorems and proofs

Theorem 0.196 (Linear vanishing/exploding gradient; Pascanu et al. 2013)**.** *Let $h_t = W h_{t-1}$ with $W \in \mathbb{R}^{d\times d}$, and let L be a scalar loss depending on h_T. Then for any $t < T$,*

$$\frac{\partial L}{\partial h_t} = (W^\top)^{T-t} \frac{\partial L}{\partial h_T}, \qquad \left\| \tfrac{\partial L}{\partial h_t} \right\|_2 \le \sigma_{\max}(W)^{T-t} \left\| \tfrac{\partial L}{\partial h_T} \right\|_2,$$

with the upper bound achievable.

Proof. By the multivariable chain rule of Chapter 18, $\partial h_t / \partial h_{t-1} = W$, so $\partial L/\partial h_{t-1} = W^\top \partial L/\partial h_t$. Iterating $T-t$ times gives the claimed product. By Chapter 6, $W = U\Sigma V^\top$ with $\Sigma = \operatorname{diag}(\sigma_1, \dots, \sigma_d)$ and U, V orthogonal. Then $W^\top = V\Sigma U^\top$ and

$$(W^\top)^{T-t} = V\Sigma U^\top V\Sigma U^\top \cdots$$

which does *not* simplify in general unless we reuse the SVD of W^{T-t} instead: writing $W^{T-t} = \tilde{U}\tilde{\Sigma}\tilde{V}^\top$, the singular values are bounded by $\sigma_{\max}(W)^{T-t}$ via submultiplicativity $\|AB\|_2 \le \|A\|_2 \|B\|_2$ of the operator norm. Hence $\|(W^\top)^{T-t}\|_2 = \|W^{T-t}\|_2 \le \sigma_{\max}(W)^{T-t}$. Equality is achieved when W is normal (so W^{T-t} has singular values $|\lambda_i(W)|^{T-t}$) and $\partial L/\partial h_T$ is aligned with the top singular direction. □

Corollary 0.197 (Saturating nonlinearity)**.** *If σ has $|\sigma'| \le c$ pointwise (for* tanh*, $c = 1$), then with $D_t = \operatorname{diag}(\sigma'(\cdot))$ a diagonal Jacobian,*

$$\left\| \tfrac{\partial L}{\partial h_t} \right\|_2 \le c^{T-t} \, \|W\|_2^{T-t} \left\| \tfrac{\partial L}{\partial h_T} \right\|_2.$$

Proof. The Jacobian of one step is $D_t W$ with $\|D_t\|_2 \le c$. Apply submultiplicativity $T-t$ times. □

Remark 0.198. For tanh, saturation pushes $\|D_t\|_2$ *below* 1 in the asymptotic regions, so nonlinearity tends to *worsen* vanishing. This is the regime in which standard RNNs fail to learn dependencies beyond roughly $T - t \sim 20$–50 steps.

Theorem 0.199 (Information bottleneck of an RNN)**.** *The hidden state $h_t \in \mathbb{R}^d$ is a deterministic function of $(x_1, \ldots, x_t)$, so by the data-processing inequality applied to the Markov chain $(x_1, \ldots, x_t) \to h_t \to \hat{x}_s$ for $s \le t$, the mutual information satisfies $I(x_s; \hat{x}_s) \le I(x_s; h_t) \le d \cdot B$, where B is the per-coordinate bit precision. Once $t \log_2 |\mathcal{X}| \gg dB$, some information about earlier tokens is provably destroyed.*

Sketch. Apply the data-processing inequality (a standard result of information theory) to the deterministic Markov chain. The right inequality is the channel capacity of a d-dimensional vector at B bits per coordinate. □

Proposition 0.200 (Attention dissolves both bottlenecks — sketch)**.** *Let $h_T = \sum_{s=1}^{T} \alpha_{Ts} V_s$ with values $V_s = V x_s$ and weights α_{Ts}. Then $\partial h_T / \partial V_s = \alpha_{Ts} I_d$ and $\partial L / \partial V_s = \alpha_{Ts} \, \partial L / \partial h_T$. This is a* single *linear step, independent of $T - s$, so no quantity is exponentiated in $T - s$. Each past token also has its own addressable slot, so the bottleneck of Theorem 0.199 is replaced by an $O(Td)$-sized cache. The full derivation appears in Chapter 21.*

Code sketch

The accompanying notebook (`cells.json`) implements an RNN cell with $d = 8$, $T = 50$, and seed 0. It empirically measures $\|\partial L / \partial h_t\|$ for spectral norms $\sigma_{\max}(W) \in \{0.5, 0.95, 1.0, 1.05, 1.5\}$ and plots log-norm versus $T - t$, recovering the linear (in log) decay/growth of Theorem 0.196. It then verifies $\sigma_{\max}(W^{T-t}) = \sigma_{\max}(W)^{T-t}$ (for normal W) via both SVD and power iteration, and shows that a one-shot attention average has gradient norm *independent* of position.

Connection to LLMs

The vanishing-gradient theorem is the historical reason transformer-based LLMs displaced RNNs. Bahdanau, Cho, and Bengio (2015) added attention to a seq2seq RNN to fix translation of long sentences; Vaswani et al. (2017, "Attention Is All You Need") removed recurrence entirely. Without an $O(1)$ gradient path, scaling to 10^4–10^6-token contexts is hopeless: a $\sigma_{\max} = 0.99$ RNN attenuates by e^{-100} over 10,000 steps. We formalize attention in Chapter 21, generalize to multi-head in Chapter 22, embed in the Transformer block in Chapter 23, and scale in Chapter 24.

Chapter 21: Scaled dot-product attention: derivation, softmax-temperature analysis

Motivation

Chapter 20 closed with a sharp negative result: a recurrent network propagates gradients between tokens j and i through $|i - j|$ multiplicative steps, so $\|\partial h_i / \partial h_j\|$ shrinks or grows geometrically. Long-range dependencies become statistically unreachable. We now construct a primitive that links every output position to every input position through a *single* matrix multiplication, so the gradient between any pair of tokens travels in $O(1)$ depth: **scaled dot-product attention** [1].

Definitions

Let $X \in \mathbb{R}^{T \times d_{\text{model}}}$ be a sequence of T token vectors, e.g. embeddings from Chapter 19. Fix projection matrices

$$W_Q, W_K \in \mathbb{R}^{d_{\text{model}} \times d_k}, \qquad W_V \in \mathbb{R}^{d_{\text{model}} \times d_v},$$

and define the **queries**, **keys**, and **values**

$$Q = XW_Q, \qquad K = XW_K, \qquad V = XW_V.$$

These are linear maps in the sense of Chapter 5.

Definition 0.201 (Scaled dot-product attention)**.**

$$\boxed{\text{Attn}(Q, K, V) \;=\; \text{softmax}\!\left(\frac{QK^\top}{\sqrt{d_k}}\right) V.}$$

The softmax (Chapter 16) is applied row-wise. We write

$$A \;:=\; \text{softmax}\!\left(\tfrac{QK^\top}{\sqrt{d_k}}\right) \in \mathbb{R}^{T \times T}$$

for the attention weights. *Each row $A_{i,:}$ is a probability vector: $A_{ij} \geq 0$ and $\sum_j A_{ij} = 1$.*

Definition 0.202 (Self- vs. cross-attention)**.** *When Q, K, V are all derived from the same source X, Definition 0.201 is called* self-attention. *When Q comes from a sequence X but K, V come from a different sequence Y (e.g. encoder outputs in a seq-to-seq model), it is* cross-attention.

Theorems

Theorem 0.203 (Attention as a soft database lookup)**.** *For each query index i, the output row $\text{Attn}(Q, K, V)_i$ is a convex combination of the value rows $V_1, \ldots, V_T$.*

Proof. By Chapter 16, $A_{i,:}$ is the image of a softmax, hence $A_{ij} \in (0, 1)$ and $\sum_j A_{ij} = 1$. Therefore

$$(AV)_i \;=\; \sum_{j=1}^{T} A_{ij}\, V_j \;\in\; \text{conv}\{V_1, \ldots, V_T\}. \qquad \square$$

Theorem 0.204 (Variance of dot products and the $\sqrt{d_k}$ scaling)**.** *Suppose the entries of $Q_{i,:}$ and $K_{j,:}$ are i.i.d., mean zero, variance* 1*, and the two vectors are independent. Then*

$$\mathbb{E}\big[Q_i \cdot K_j\big] = 0, \qquad \text{Var}\big(Q_i \cdot K_j\big) = d_k.$$

Consequently $\big(QK^\top\big)_{ij}/\sqrt{d_k}$ has unit variance.

Proof. Write

$$S \;=\; Q_i \cdot K_j \;=\; \sum_{k=1}^{d_k} Q_{ik} K_{jk}.$$

By linearity of expectation and independence,

$$\mathbb{E}[S] = \sum_{k=1}^{d_k} \mathbb{E}[Q_{ik}]\, \mathbb{E}[K_{jk}] = 0.$$

For the second moment, expand

$$S^2 = \sum_{k=1}^{d_k} Q_{ik}^2 K_{jk}^2 \;+\; \sum_{k \neq k'} Q_{ik} K_{jk} Q_{ik'} K_{jk'}.$$

The diagonal terms satisfy $\mathbb{E}[Q_{ik}^2 K_{jk}^2] = \mathbb{E}[Q_{ik}^2]\,\mathbb{E}[K_{jk}^2] = 1 \cdot 1 = 1$. The off-diagonal terms factor (by independence across k and across Q, K) into products of zero means, hence vanish in expectation. Therefore

$$\mathrm{Var}(S) = \mathbb{E}[S^2] - 0 = d_k. \qquad \square$$

Corollary 0.205 (Saturation without scaling)**.** *Without the $1/\sqrt{d_k}$ factor, the logits feeding softmax have standard deviation $\sqrt{d_k} \to \infty$. The maximum logit dominates and softmax concentrates on a single index $j^\star$. Since $\partial \mathrm{softmax}_i / \partial s_j = \mathrm{softmax}_i(\delta_{ij} - \mathrm{softmax}_j)$, the Jacobian collapses to (approximately) the zero matrix off $j^\star$, vanishing gradients with respect to Q and K.*

Remark 0.206. Scaling by $1/\sqrt{d_k}$ is therefore a *temperature normalization*. It keeps softmax in the well-conditioned regime where its Jacobian has rank $T - 1$ and gradients propagate informatively through every position.

Theorem 0.207 (Permutation equivariance)**.** *Let $\pi \in S_T$ be a permutation of the sequence axis, with permutation matrix P_π. Then*

$$\mathrm{Attn}(P_\pi Q,\, P_\pi K,\, P_\pi V) \;=\; P_\pi\, \mathrm{Attn}(Q, K, V).$$

Proof. The pre-softmax logits transform as

$$\big(P_\pi Q\big)\big(P_\pi K\big)^\top / \sqrt{d_k} \;=\; P_\pi \big(Q K^\top / \sqrt{d_k}\big) P_\pi^\top.$$

Row-wise softmax acts independently on each row, and the column permutation $P_\pi^\top$ on the right merely reorders the entries within each row, so the softmax permutes consistently:

$$A' := \mathrm{softmax}\big(P_\pi (Q K^\top / \sqrt{d_k}) P_\pi^\top\big) = P_\pi A P_\pi^\top.$$

Multiplying by the permuted values,

$$A'(P_\pi V) = P_\pi A P_\pi^\top P_\pi V = P_\pi A V,$$

since $P_\pi^\top P_\pi = I$. $\square$

Remark 0.208. This symmetry is *too strong* for natural language: attention cannot distinguish "the cat sat" from "sat cat the". Chapter 24 fixes this by injecting positional information into X before the projections.

Theorem 0.209 ($O(1)$ gradient depth between any two tokens)**.** *For any pair of indices i, j,*

$$\frac{\partial (AV)_i}{\partial V_j} \;=\; A_{ij}\, I_{d_v},$$

which has spectral norm $A_{ij} \in (0, 1)$, independent of $|i - j|$.

Proof. $(AV)_i = \sum_{\ell=1}^{T} A_{i\ell} V_\ell$, so $\partial (AV)_i / \partial V_j = A_{ij} I_{d_v}$. $\square$

Remark 0.210. Compare to Chapter 20: an RNN with spectral norm ρ satisfies $\|\partial h_i / \partial h_j\| \lesssim \rho^{|i-j|}$, exponentially small for $\rho < 1$. Attention replaces this exponential dependence by a single matmul, paying $O(T^2 d_k)$ compute for the matrix A.

Connection to LLMs

Scaled dot-product attention is *the* primitive of every transformer language model. Chapter 22 stacks h heads of it in parallel (multi-head attention); Chapter 23 wires attention into the residual + MLP transformer block; Chapter 24 repairs the permutation symmetry of Theorem 4 with positional encodings. Production-scale variants — Flash Attention (IO-aware tiling), multi-query and grouped-query attention (sharing K, V across heads), sliding-window and linear attention — all preserve the scaled-softmax form derived here. Whenever you call an LLM, every token-to-token interaction inside it is an instance of $\mathrm{softmax}(QK^\top/\sqrt{d_k})V$.

References

[1] A. Vaswani et al. *Attention Is All You Need*. NeurIPS, 2017.

Chapter 22: Multi-head attention: parallel heads as concat-then-project; complexity

Motivation

Chapter 21 built a single attention head: a $T \times T$ soft-lookup linking every output position to every input position in $O(1)$ depth. A single head must compress *every* relational pattern — syntactic, semantic, positional — into one softmax-weighted average over V. The Vaswani et al. (2017) fix is **multi-head attention** (MHA): run H heads in parallel on disjoint low-rank subspaces, concatenate, and re-project. We will prove that (i) MHA equals a single attention with block-diagonal projections, (ii) the parameter count is independent of H, and (iii) compute scales as $\Theta(Td^2 + T^2 d)$, with the $T^2 d$ term motivating the inference-time variants **multi-query** (MQA) and **grouped-query** (GQA) attention used by PaLM and Llama 2/3.

Definitions

Let $X \in \mathbb{R}^{T \times d}$ with $d := d_{\text{model}}$. Fix $H \mid d$ and set $d_k = d_v = d/H$. Write $\mathrm{Attn}(\cdot, \cdot, \cdot)$ for scaled dot-product attention (Chapter 21).

Definition 0.211 (Multi-head attention)**.** *For $h = 1, \ldots, H$, fix*

$$W_Q^{(h)}, W_K^{(h)} \in \mathbb{R}^{d \times d_k}, \qquad W_V^{(h)} \in \mathbb{R}^{d \times d_v}, \qquad W_O \in \mathbb{R}^{H d_v \times d}.$$

Define

$$\mathrm{head}_h \;=\; \mathrm{Attn}\big(XW_Q^{(h)},\, XW_K^{(h)},\, XW_V^{(h)}\big) \;\in\; \mathbb{R}^{T \times d_v},$$

$$\boxed{\mathrm{MHA}(X) \;=\; \mathrm{Concat}(\mathrm{head}_1, \ldots, \mathrm{head}_H)\, W_O \;\in\; \mathbb{R}^{T \times d}.}$$

Definition 0.212 (Multi-query attention)**.** *All H heads share a single W_K, W_V; only $W_Q^{(h)}$ varies per head. Hence the cached K, V tensors are H times smaller.*

Definition 0.213 (Grouped-query attention)**.** *Partition the H heads into G groups, $G \mid H$. Heads within a group share W_K, W_V. MHA $= G = H$, MQA $= G = 1$. Llama 2/3 use $G \in \{4, 8\}$.*

Theorems

Theorem 0.214 (MHA = block-structured single attention)**.** *Let* $W_Q^{\text{blk}} := [W_Q^{(1)} \mid \cdots \mid W_Q^{(H)}] \in \mathbb{R}^{d \times Hd_k}$ *be the horizontal concatenation of the per-head Q-projections, and similarly* $W_K^{\text{blk}}, W_V^{\text{blk}}$*. Set*

$$Q^{\text{blk}} = XW_Q^{\text{blk}}, \quad K^{\text{blk}} = XW_K^{\text{blk}}, \quad V^{\text{blk}} = XW_V^{\text{blk}} \ \in \ \mathbb{R}^{T \times Hd_k}.$$

Reshape each into $\mathbb{R}^{T \times H \times d_k}$ *by splitting the last axis into* H *blocks of width* d_k*. Then the per-head attention applied to slice* h *of these reshaped tensors equals* head_h*.*

Proof. By construction

$$XW_Q^{\text{blk}} \ = \ X\big[W_Q^{(1)} \mid \cdots \mid W_Q^{(H)}\big] \ = \ \big[XW_Q^{(1)} \mid \cdots \mid XW_Q^{(H)}\big],$$

so the h-th block of width d_k is exactly $XW_Q^{(h)} = Q^{(h)}$. The same holds for K and V. The reshape is a memory-layout no-op: slice h of $Q^{\text{blk}} \in \mathbb{R}^{T \times H \times d_k}$ is $Q^{(h)}$. Since Attn is computed independently per head and uses only that head's $(Q^{(h)}, K^{(h)}, V^{(h)})$, the outputs match. □

Remark 0.215. The score matrix is *not* a single $HT \times HT$ block-diagonal matmul: the softmax is taken per head, with no cross-head mixing until W_O. The block picture says only that the linear projections factor cleanly; the nonlinearity preserves head independence.

Theorem 0.216 (Parameter count is independent of H)**.** *The total parameter count of MHA's* $W_Q^{(\cdot)}, W_K^{(\cdot)}, W_V^{(\cdot)}, W_O$ *is exactly* $4d^2$*.*

Proof. Each per-head projection is $d \times d_k = d \times d/H$, with d^2/H parameters. Three projections per head and H heads yield

$$3H \cdot \frac{d^2}{H} \ = \ 3d^2.$$

The output projection W_O is $Hd_v \times d = d \times d$, contributing d^2. Total: $4d^2$. The $1/H$ shrinkage of each head exactly cancels the H-fold replication. □

Theorem 0.217 (Compute complexity)**.** *For input* $X \in \mathbb{R}^{T \times d}$*,*

$$\boxed{\text{cost(MHA)} \ = \ \Theta(Td^2 + T^2d).}$$

Proof. We sum FLOPs across the five steps.

1. *Q, K, V projections.* By Theorem 0.214 we may treat them as three $T \times d$ by $d \times d$ matmuls: $\Theta(Td^2)$ in total.
2. *Score* $Q^{(h)}K^{(h)\top}$ *per head.* Each is $(T \times d_k)(d_k \times T) = \Theta(T^2 d_k)$. Summed across H heads: $H \cdot \Theta(T^2 d_k) = \Theta(T^2 d)$ since $Hd_k = d$.
3. *Softmax.* $\Theta(T^2)$ per head, $\Theta(HT^2)$ total — subdominant.
4. *Output AV.* Per head $(T \times T)(T \times d_v) = \Theta(T^2 d_v)$, total $\Theta(T^2 d)$.
5. W_O*.* $T \times (Hd_v) \cdot (Hd_v \times d) = \Theta(Td^2)$.

Adding: $\Theta(Td^2) + \Theta(T^2 d) + \Theta(HT^2) + \Theta(T^2 d) + \Theta(Td^2) = \Theta(Td^2 + T^2 d)$. □

Corollary 0.218 (Long-context regime)**.** *When $T \gg d$, the $T^2 d$ term dominates. Doubling the sequence quadruples attention FLOPs but only doubles projection FLOPs. This is the structural reason transformer inference becomes attention-bound at long context lengths and motivates the efficient-attention literature of Chapter 27.*

Theorem 0.219 (KV-cache memory)**.** *Autoregressive decoding caches K, V for all past tokens. Per layer, MHA stores $2Td$ scalars; GQA with G groups stores $2Td \cdot G/H$; MQA stores $2Td/H$.*

Proof. Each head needs $K \in \mathbb{R}^{T \times d_k}$, $V \in \mathbb{R}^{T \times d_v}$, totalling $2Td_k$ scalars. MHA replicates this H times (one per head), giving $2THd_k = 2Td$. GQA replicates G times, MQA once. Multiplying through gives the stated counts. □

Example 0.220. A 70 B-parameter model with $d = 8192$, $H = 64$ at $T = 8192$ stores $2 \cdot 8192 \cdot 8192 \approx 1.34 \times 10^8$ scalars per layer for MHA, but only $\approx 1.7 \times 10^7$ for GQA with $G = 8$ — an $8\times$ inference-memory reduction at small quality cost.

Code sketch

`cells.json` implements MHA in numpy with $T = 8, d = 12, H = 3$, verifies Theorem 0.214 numerically (max-abs-diff between per-head loop and block-projected reshape is at machine precision), times the forward pass for $T \in \{32, 64, 128, 256\}$ exhibiting the T^2 kink, and tabulates KV-cache sizes for MHA vs. GQA ($G = 2$) vs. MQA.

Connection to LLMs

GPT-2 and GPT-3 use vanilla MHA. PaLM (2022) deploys MQA following Shazeer (2019), motivated by inference throughput. Llama 2 (2023) introduces GQA as a quality/memory compromise; Llama 3, Mistral, and most open-weights successors retain it. The contemporary picture: train MHA-shaped compute, deploy GQA to fit the KV cache in HBM. Chapter 23 stacks these blocks into a transformer layer; Chapter 27 attacks the $T^2 d$ term proven above (FlashAttention, sliding-window, linear attention).

Chapter 23: Transformer block: residual + LayerNorm/RMSNorm + FFN + attention; gradient-flow argument

Motivation

A single attention head (Chapter 22) is a beautiful linear-algebraic object, but a transformer is not one head: it is a stack of dozens to hundreds of nearly identical *blocks*. Two empirical facts dominate the design of those blocks. First, deep stacks of pure compositions $f_L \circ \cdots \circ f_1$ suffer vanishing or exploding Jacobians (we proved a strict version of this for RNNs in Chapter 20). Second, the activations between layers drift in scale and mean unless we actively re-normalize. The transformer block solves both problems by wrapping every learned sublayer in a residual connection and a normalization layer, then alternating an attention sublayer with a position-wise feedforward MLP. This chapter pins down the algebra and proves the gradient-flow guarantee that makes deep transformer stacks trainable end-to-end.

Definitions

Definition 0.221 (Residual connection)**.** *Given a map $F : \mathbb{R}^d \to \mathbb{R}^d$, the* residual *(or* skip*) wrapper is* $\mathrm{Res}_F(x) = F(x) + x$.

Definition 0.222 (LayerNorm; Ba–Kiros–Hinton 2016)**.** *For $x \in \mathbb{R}^d$ let $\mu = \frac{1}{d}\sum_i x_i$ and $\sigma^2 = \frac{1}{d}\sum_i (x_i - \mu)^2$. Then*

$$\mathrm{LN}(x) \;=\; \gamma \odot \frac{x - \mu \mathbf{1}}{\sqrt{\sigma^2 + \varepsilon}} + \beta, \qquad \gamma, \beta \in \mathbb{R}^d.$$

Definition 0.223 (RMSNorm; Zhang–Sennrich 2019)**.** *With $r(x)^2 = \frac{1}{d}\sum_i x_i^2$,*

$$\mathrm{RMSN}(x) \;=\; \gamma \odot \frac{x}{\sqrt{r(x)^2 + \varepsilon}}.$$

RMSN drops mean-subtraction and the β shift, saving roughly 7% of the per-token FLOPs of LN.

Definition 0.224 (Position-wise FFN)**.** *Two affine maps separated by a pointwise nonlinearity σ (Chapter 16):*

$$\mathrm{FFN}(x) \;=\; W_2\, \sigma(W_1 x + b_1) + b_2, \qquad W_1 \in \mathbb{R}^{d_{\mathrm{ff}} \times d},\;\; W_2 \in \mathbb{R}^{d \times d_{\mathrm{ff}}},$$

typically $d_{\mathrm{ff}} = 4d$. This is exactly the one-hidden-layer MLP of Chapter 15, applied independently to each token position.

Definition 0.225 (Pre-norm transformer block)**.** *Given multi-head self-attention* MHA *(Chapter 22) and FFN as above, the modern block (GPT-2 onward) is*

$$z = x + \mathrm{MHA}\big(\mathrm{LN}(x)\big), \qquad y = z + \mathrm{FFN}\big(\mathrm{LN}(z)\big).$$

The original Vaswani 2017 post-norm *block applied LN* after *the residual sum: $z = \mathrm{LN}(x + \mathrm{MHA}(x))$.*

Theorems and proofs

Theorem 0.226 (Residual gradient identity)**.** *Let $y = F(x) + x$ with $F : \mathbb{R}^d \to \mathbb{R}^d$ differentiable. Then $\partial y/\partial x = J_F(x) + I$. Iterating across L residual blocks $x_{\ell+1} = x_\ell + F^{(\ell)}(x_\ell)$,*

$$\frac{\partial x_L}{\partial x_0} \;=\; \prod_{\ell=L-1}^{0} \big(I + J_{F^{(\ell)}}(x_\ell)\big).$$

Proof. Direct from the chain rule (Chapter 18): $\partial x_{\ell+1}/\partial x_\ell = I + J_{F^{(\ell)}}(x_\ell)$, and Jacobians compose by left-multiplication. □

Corollary 0.227 (Non-vanishing gradient flow)**.** *Suppose $\|J_{F^{(\ell)}}\|_2 \le \kappa < 1$ uniformly. Without residuals, $\|\prod_\ell J_{F^{(\ell)}}\|_2 \le \kappa^L$, the exponential decay of Chapter 20's RNN bound. With residuals, expanding the product*

$$\frac{\partial x_L}{\partial x_0} \;=\; I + \sum_{\ell} J_{F^{(\ell)}} + \sum_{\ell<m} J_{F^{(m)}} J_{F^{(\ell)}} + \cdots$$

gives $\|\partial x_L/\partial x_0\|_2 \ge 1 - L\kappa - \binom{L}{2}\kappa^2 - \cdots$, bounded away from zero for any depth provided κ is small enough. The identity term guarantees *a non-vanishing path from output to input.*

Theorem 0.228 (LN invariances and Jacobian)**.** *Set $\gamma = \mathbf{1}$, $\beta = 0$, $\varepsilon = 0$ to isolate the normalizer. For any $\alpha > 0$ and $\beta_0 \in \mathbb{R}$, $\mathrm{LN}(\alpha x + \beta_0 \mathbf{1}) = \mathrm{LN}(x)$.*

Proof. Let $x' = \alpha x + \beta_0 \mathbf{1}$. Then $\mu' = \alpha\mu + \beta_0$ and $\sigma'^2 = \frac{1}{d}\sum_i (\alpha x_i + \beta_0 - \alpha\mu - \beta_0)^2 = \alpha^2\sigma^2$. Hence

$$\frac{x' - \mu'\mathbf{1}}{\sigma'} \;=\; \frac{\alpha(x - \mu\mathbf{1})}{\alpha\sigma} \;=\; \frac{x - \mu\mathbf{1}}{\sigma}. \qquad \square$$

Writing $\hat{x} = (x - \mu\mathbf{1})/\sigma$, $\partial\mu/\partial x_j = 1/d$, and $\partial\sigma/\partial x_j = (x_j - \mu)/(d\sigma) = \hat{x}_j/d$, the chain rule yields

$$\boxed{\frac{\partial \hat{x}_i}{\partial x_j} \;=\; \frac{1}{\sigma}\left(\delta_{ij} - \frac{1}{d} - \frac{1}{d}\hat{x}_i\hat{x}_j\right)}$$

The two subtracted terms project out the constant ($\mathbf{1}$) and $\hat{x}$ directions — exactly the directions LN is invariant to. The Jacobian is therefore rank $d - 2$, with kernel $\mathrm{span}\{\mathbf{1}, x - \mu\mathbf{1}\}$.

Proposition 0.229 (RMSN $\approx$ LN on centered data)**.** *If $\mu(x) = 0$ then $\mathrm{LN}(x) = \mathrm{RMSN}(x)$ (the LN β is absorbed into a downstream bias). For arbitrary x,*

$$\mathrm{RMSN}(x) - \mathrm{LN}(x) \;=\; \gamma \odot \frac{\mu(x)\,\mathbf{1}}{\sqrt{r(x)^2 + \varepsilon}} + O\!\left(\frac{\mu(x)^2}{r(x)^2}\right).$$

Proof. When $\mu = 0$, $r(x)^2 = \sigma(x)^2$ and $x - \mu\mathbf{1} = x$, so the two formulas coincide. The general expansion follows by writing $r^2 = \sigma^2 + \mu^2$ and Taylor-expanding the inverse square root. $\square$

The empirical observation behind RMSN, verified in our code cells, is that after a few transformer layers the residual stream has near-zero mean, so dropping centering costs essentially no quality.

Remark 0.230 (Pre-norm vs. post-norm gradient scale). With He/Xavier initialization (Chapter 17), each sublayer output has the same variance as its input. In a *post-norm* stack the residual sum $x + F(x)$ has variance $2\,\mathrm{Var}(x)$, which LN immediately rescales to $\mathrm{Var}(x)$, but the gradient picks up a factor of $1/\sqrt{2}$ per block, so after L blocks the gradient at the input scales like $2^{-L/2}$, requiring careful warmup. In a *pre-norm* stack the residual stream itself is never rescaled; only the input *to* each sublayer is normalized. The end-to-end Jacobian is Theorem 0.226's $\prod(I + J_F)$ with each J_F acting on a normalized input, so the gradient norm at x_0 remains $\Theta(1)$ independent of L. This is why every model from GPT-2 onward uses pre-norm.

Code sketch

The companion notebook (i) implements one full pre-norm block in numpy and verifies shape preservation; (ii) stacks 20 residual blocks against 20 plain blocks and measures backpropagated gradient norms by finite differences; (iii) checks LN invariance under $x \mapsto \alpha x + \beta_0\mathbf{1}$ across many random (α, β_0); (iv) compares LN vs. RMSN on centered and uncentered inputs.

Connection to LLMs

Every layer of every modern decoder LLM is a small variation of the block above. GPT-2/3/4 use pre-LN with GELU FFN. Llama, Mistral, and Qwen swap LN for RMSN, GELU for SwiGLU (Chapter 16), and MHA for grouped-query attention (Chapter 24); Llama also uses RoPE positional encoding (Chapter 25) inside the attention sublayer. The block remains

$$z = x + \mathrm{Sublayer}_1(\mathrm{Norm}(x)), \qquad y = z + \mathrm{Sublayer}_2(\mathrm{Norm}(z)).$$

Stacking 32–120 such blocks, plus the embedding (Chapter 21), the (often tied) unembedding, and a final norm before the LM head, gives the entire forward pass of a frontier-scale language model. The training-stability argument of Theorem 0.226 is precisely what makes the trillion-parameter regime (Chapters 27, 28) reachable at all.

Chapter 24: Positional encoding: sinusoidal derivation, RoPE construction

Motivation

In Chapter 21 we proved that scaled dot-product attention is *permutation-equivariant*: for any permutation matrix P and sequence $X \in \mathbb{R}^{T\times d}$,

$$\text{Attention}(PX, PX, PX) \;=\; P \cdot \text{Attention}(X, X, X).$$

A direct corollary is catastrophic for language modeling: shuffling the tokens of a sentence and feeding them to attention produces the same set of output vectors, only re-indexed. Attention by itself sees a *bag* of tokens, not a sequence. To recover order, we must inject the position $t \in \{0, 1, \ldots, T-1\}$ into the representations.

Three approaches dominate practice: **sinusoidal** positional encodings (Vaswani et al. 2017), **learned** positional embeddings (BERT, GPT-2), and **rotary position embedding** or **RoPE** (Su et al. 2021), used by Llama, Mistral, Claude, Gemini, and most modern LLMs. We derive all three and prove the central RoPE relative-position theorem.

Definitions

Definition 0.231 (Sinusoidal positional encoding)**.** *For position* $\text{pos} \in \mathbb{N}$ *and even embedding dimension* d*, define* $\text{PE}(\text{pos}) \in \mathbb{R}^d$ *by*

$$\text{PE}(\text{pos}, 2i) = \sin(\text{pos} \cdot \theta_i), \qquad \text{PE}(\text{pos}, 2i+1) = \cos(\text{pos} \cdot \theta_i),$$

with frequencies $\theta_i := 10000^{-2i/d}$ *for* $i = 0, \ldots, d/2 - 1$*. The position-aware token representation is* $x_t = E[\text{token}_t] + \text{PE}(t)$*.*

Definition 0.232 (Learned positional embedding)**.** *Fix* $T_{\max}$*. Let* $P \in \mathbb{R}^{T_{\max}\times d}$ *be a parameter matrix. The position embedding at* t *is the row* P_t*, learned end-to-end with the rest of the model.*

Definition 0.233 (Rotary position embedding (RoPE))**.** *Group the* d *coordinates of a query or key vector into* $d/2$ *consecutive pairs. For pair* i *at position* pos*, set*

$$R_{\text{pos},i} \;=\; \begin{pmatrix} \cos(\text{pos}\,\theta_i) & -\sin(\text{pos}\,\theta_i) \\ \sin(\text{pos}\,\theta_i) & \cos(\text{pos}\,\theta_i) \end{pmatrix}, \qquad \theta_i = 10000^{-2i/d}.$$

Let $R_{\text{pos}} \in \mathbb{R}^{d\times d}$ *be the block-diagonal matrix with these rotations on the diagonal. RoPE acts on the projected queries and keys multiplicatively:*

$$q_m^{\text{rot}} = R_m\, q_m, \qquad k_n^{\text{rot}} = R_n\, k_n.$$

Theorems and Proofs

Theorem 0.234 (Sinusoidal PE encodes shift as rotation)**.** *For every offset $k \in \mathbb{Z}$ there exists a fixed block-diagonal matrix $M_k \in \mathbb{R}^{d\times d}$, independent of* pos*, such that*

$$\mathrm{PE}(\mathrm{pos}+k) \;=\; M_k \cdot \mathrm{PE}(\mathrm{pos}).$$

Proof. Restrict to the i-th coordinate pair. By the trigonometric addition formulas,

$$\begin{aligned}\sin\big((\mathrm{pos}+k)\theta_i\big) &= \sin(\mathrm{pos}\,\theta_i)\cos(k\theta_i) + \cos(\mathrm{pos}\,\theta_i)\sin(k\theta_i),\\ \cos\big((\mathrm{pos}+k)\theta_i\big) &= \cos(\mathrm{pos}\,\theta_i)\cos(k\theta_i) - \sin(\mathrm{pos}\,\theta_i)\sin(k\theta_i).\end{aligned}$$

With $c := \cos(k\theta_i)$, $s := \sin(k\theta_i)$, the pair transforms as

$$\begin{pmatrix}\sin((\mathrm{pos}+k)\theta_i)\\ \cos((\mathrm{pos}+k)\theta_i)\end{pmatrix} = \begin{pmatrix}c & s\\ -s & c\end{pmatrix}\begin{pmatrix}\sin(\mathrm{pos}\,\theta_i)\\ \cos(\mathrm{pos}\,\theta_i)\end{pmatrix}.$$

The 2×2 matrix depends only on k and θ_i, not on pos. Stacking these blocks for $i = 0,\dots,d/2-1$ yields the claimed M_k. □

Theorem 0.235 (RoPE relative-position property)**.** *For all $m, n \in \mathbb{N}$ and all $q_m, k_n \in \mathbb{R}^d$,*

$$\big\langle R_m\, q_m,\; R_n\, k_n\big\rangle \;=\; q_m^\top\, R_{n-m}\, k_n.$$

Proof. Each $R_{\mathrm{pos},i}$ is a planar rotation, hence

$$R_{\mathrm{pos},i}^\top = R_{-\mathrm{pos},i}, \qquad R_{a,i}\,R_{b,i} = R_{a+b,i},$$

the latter being the angle-addition identity for $\mathrm{SO}(2)$. Lifting to block-diagonal matrices preserves both: $R_m^\top = R_{-m}$ and $R_a R_b = R_{a+b}$. Therefore

$$\langle R_m q_m,\, R_n k_n\rangle = q_m^\top R_m^\top R_n k_n = q_m^\top R_{-m} R_n k_n = q_m^\top R_{n-m} k_n. \qquad \square$$

Corollary 0.236 (Translation invariance)**.** *For any $c \in \mathbb{Z}$, $\langle R_{m+c}q_m,\, R_{n+c}k_n\rangle = \langle R_m q_m,\, R_n k_n\rangle$.*

Proof. By Theorem 0.235, the left-hand side equals $q_m^\top R_{(n+c)-(m+c)} k_n = q_m^\top R_{n-m} k_n$, which equals the right-hand side, again by Theorem 0.235. □

Proposition 0.237 (RoPE has no learnable parameters)**.** *The map $(q, k, \mathrm{pos}) \mapsto (R_{\mathrm{pos}}q,\, R_{\mathrm{pos}}k)$ contains no trainable weights: the frequencies $\theta_i = 10000^{-2i/d}$ are fixed and R_{pos} is defined for every integer* pos*.*

Proof. By inspection of the definition. □

Remark 0.238. A model trained with context length T_{train} can in principle be evaluated at $\mathrm{pos} > T_{\mathrm{train}}$ since R_{pos} is defined for all integers. Learned positional embeddings cannot: there is no row P_t for $t \ge T_{\max}$. RoPE thus offers a principled (if imperfect) path to length extrapolation, refined by NTK-aware scaling, YaRN, and Position Interpolation in Chapter 27.

Code Sketch

The accompanying notebook (i) confirms permutation-invariance of vanilla attention numerically, (ii) implements and visualizes the sinusoidal PE matrix, (iii) verifies Theorem 0.234 by checking $\|\mathrm{PE}(\mathrm{pos}+k) - M_k\mathrm{PE}(\mathrm{pos})\| < 10^{-12}$, (iv) implements RoPE for $d = 8$, $T = 6$ and confirms Theorem 0.235 by comparing $\langle R_m q_m, R_n k_n\rangle$ against $q_m^\top R_{n-m} k_n$, and (v) verifies Corollary 0.236 by shifting all positions by $c \in \{1, 5, 10\}$ and confirming the attention score matrix is unchanged.

Connection to LLMs

Vaswani et al.'s original Transformer used sinusoidal PEs. BERT and GPT-2 switched to learned PEs, trading off extrapolation for marginal accuracy gains within the training context. From Llama (2023) onward, RoPE is the de facto standard: Theorem 0.235 states that attention scores depend only on token *separation*, matching the linguistic intuition that "the dog *across the street*" should attend the same way regardless of where the phrase appears in the document. Combined with extrapolation techniques (Chapter 27), RoPE has enabled context windows from 2K (Llama 1) to 1M+ tokens (Claude, Gemini). Chapter 23 introduced multi-head attention; positional encoding is applied per-head before the dot product, so RoPE composes cleanly with multi-head splitting and with everything that follows.

Block F — Pre-training

Chapter 25: Causal masking; next-token prediction loss as MLE on the empirical distribution

Motivation

We have a transformer block that mixes information across positions (Ch. 23) and a positional encoding that injects order (Ch. 24). To turn this into a *language model* we must decide (i) what the network outputs, (ii) what loss to minimize, and (iii) how to reconcile parallel training with the constraint that the model never see future tokens. All three are settled by one design choice: *causal masking + next-token prediction* (NTP).

Definitions

Definition 0.239 (Causal language model)**.** *A causal (decoder-only) language model with parameters θ assigns to a sequence $x_{1:T} = (x_1, \dots, x_T) \in \mathcal{V}^T$ the joint*

$$p_\theta(x_{1:T}) \;=\; \prod_{t=1}^{T} p_\theta\big(x_t \,\big|\, x_{<t}\big), \qquad x_{<t} := (x_1, \dots, x_{t-1}).$$

Each conditional $p_\theta(\cdot \mid x_{<t})$ is a softmax over $\mathcal{V}$ produced by a transformer applied to the prefix.

Definition 0.240 (Causal attention mask)**.** *For $T \in \mathbb{N}$, the causal mask is the matrix $M \in \{0, -\infty\}^{T \times T}$ with*

$$M_{ij} = \begin{cases} 0, & j \le i, \\ -\infty, & j > i. \end{cases}$$

Causal scaled dot-product attention replaces the score matrix $S = QK^\top/\sqrt{d_k}$ by $S + M$ before the row-wise softmax. Because $\exp(-\infty) = 0$, row i of the softmax has zero mass on every column $j > i$.

Definition 0.241 (Teacher forcing)**.** *At training, the input to predict x_t is the ground-truth prefix $x_{<t}$, not the model's own previous predictions. Combined with the causal mask, this lets all T predictions be computed in one parallel forward pass on $x_{1:T}$.*

Theorems and proofs

Theorem 0.242 (Chain-rule factorization)**.** *For any joint distribution p on $\mathcal{V}^T$, $p(x_{1:T}) = \prod_{t=1}^{T} p(x_t \mid x_{<t})$.*

Proof. Iterate the conditional-probability identity (Ch. 8) $p(A, B) = p(A)\, p(B \mid A)$: $p(x_{1:T}) = p(x_1)\, p(x_2 \mid x_1)\, p(x_3 \mid x_{1:2}) \cdots p(x_T \mid x_{<T})$. □

This identity holds for *every* joint distribution; the modeling choice is to parameterize each conditional with a transformer. Causal masking is precisely what enforces this factorization at the level of the architecture.

Theorem 0.243 (NTP loss = empirical NLL = cross-entropy with $\hat{p}_{\mathcal{D}}$)**.** *Let $\mathcal{D} = \{x^{(n)}_{1:T_n}\}_{n=1}^{N}$ be a corpus of i.i.d. documents drawn from $p^\star$. Define*

$$\mathcal{L}(\theta) \;=\; -\sum_{n=1}^{N} \sum_{t=1}^{T_n} \log p_\theta\Big(x_t^{(n)} \mid x_{<t}^{(n)}\Big)\,.$$

Then $\arg\min_\theta \mathcal{L}(\theta) = \arg\min_\theta H\big(\hat{p}_{\mathcal{D}},\, p_\theta\big)$, where $\hat{p}_{\mathcal{D}}$ is the empirical distribution and H is cross-entropy.

Proof. By Theorem 0.242, $\log p_\theta(x^{(n)}_{1:T_n}) = \sum_t \log p_\theta(x_t^{(n)} \mid x_{<t}^{(n)})$, hence $\mathcal{L}(\theta) = -\sum_n \log p_\theta(x^{(n)})$. The empirical distribution $\hat{p}_{\mathcal{D}}(x) = \frac{1}{N}\sum_n \mathbb{1}[x = x^{(n)}]$ gives

$$H(\hat{p}_{\mathcal{D}}, p_\theta) = -\sum_{x \in \mathcal{V}^*} \hat{p}_{\mathcal{D}}(x) \log p_\theta(x) = -\frac{1}{N}\sum_n \log p_\theta(x^{(n)}) = \frac{1}{N}\,\mathcal{L}(\theta).$$

Multiplying by $N > 0$ does not change the argmin. By Ch. 12, this is exactly the MLE. □

Corollary 0.244 (Consistency)**.** *If $\{p_\theta\}$ is correctly specified ($p^\star = p_{\theta^\star}$ for some $\theta^\star$) and identifiable, then the minimizer $\hat{\theta}_N$ converges to $\theta^\star$ as $N \to \infty$.*

Sketch. By the law of large numbers, $\frac{1}{N}\mathcal{L}(\theta) \to -\mathbb{E}_{p^\star}[\log p_\theta(X)] = H(p^\star, p_\theta)$. Gibbs' inequality (Ch. 12) gives $H(p^\star, p_\theta) \geq H(p^\star)$ with equality iff $p_\theta = p^\star$. Identifiability + uniform convergence promote pointwise convergence of the minimizer to $\theta^\star$. □

Theorem 0.245 (Causal mask preserves backprop locality)**.** *With the causal mask, for every (t, s) with $s > t$ and every input embedding $h_s^{(0)}$,*

$$\frac{\partial \mathcal{L}_t}{\partial h_s^{(0)}} \;=\; 0, \qquad \mathcal{L}_t = -\log p_\theta(x_t \mid x_{<t}).$$

Proof. By induction on layer depth ℓ. At $\ell = 0$, position-i embedding depends only on $h_i^{(0)}$. Assume each layer's output $y_i^{(\ell)}$ depends only on $\{h_j^{(0)}\}_{j \leq i}$. The next masked-attention output is $y_i^{(\ell+1)} = \sum_{j \leq i} \alpha_{ij}^{(\ell+1)} V_j^{(\ell+1)}$ because $\alpha_{ij}^{(\ell+1)} = 0$ for $j > i$ (mask). Each $V_j^{(\ell+1)}$ is a linear function of $y_j^{(\ell)}$, which by hypothesis depends only on $\{h_k^{(0)}\}_{k \leq j} \subseteq \{h_k^{(0)}\}_{k \leq i}$. Pointwise sublayers (residuals, MLP, LayerNorm) act per-position. Hence $y_i^{(\ell+1)}$ depends only on $\{h_k^{(0)}\}_{k \leq i}$. Setting $i = t$ at the final layer and applying the chain rule, $\partial \mathcal{L}_t / \partial h_s^{(0)} = 0$ for $s > t$. □

Remark 0.246 (Why parallel training is sound). Theorem 0.245 implies that the gradient of $\mathcal{L} = \sum_t \mathcal{L}_t$ obtained from one forward pass on $x_{1:T}$ equals the sum of the T gradients we would obtain from T separate forward passes on the prefixes $x_{1:t}$. Causal masking thus turns T sequential supervision signals into one parallel forward/backward pass — the central engineering reason transformers train so much faster than RNNs.

Proposition 0.247 (Train vs. inference)**.** *Training (teacher forcing): one forward pass on $x_{1:T}$ in time $\Theta(T^2 d)$ produces all T logits and the cumulative loss. Inference (autoregressive): given prompt $x_{1:k}$, sample $\hat{x}_{k+1} \sim p_\theta(\cdot \mid x_{1:k})$, append, repeat for $T-k$ steps. The asymmetry is intrinsic: at train time x_{t+1} is known, at inference time it is whatever the model just produced.*

Definition 0.248 (Perplexity)**.** $\mathrm{PPL}(x_{1:T}) := \exp\Big(\frac{1}{T}\sum_{t=1}^{T} -\log p_\theta(x_t \mid x_{<t})\Big) = \big(\prod_t p_\theta(x_t \mid x_{<t})\big)^{-1/T}$. *Lower is better; a uniform model on $|\mathcal{V}|$ tokens achieves* $\mathrm{PPL} = |\mathcal{V}|$.

Code sketch

The accompanying notebook builds (i) an 8×8 causal mask, (ii) a single-head causal attention layer that verifies row t has zero weight on $j > t$, (iii) a tiny logits "model" confirming NTP loss equals summed cross-entropy with one-hot targets, (iv) a perturbation experiment in which changing the token at position T leaves the logit at position $T-1$ unchanged — an empirical witness of Theorem 0.245 — and (v) perplexity comparisons between a uniform model and a trained tiny model.

Connection to LLMs

Every modern decoder-only LLM — GPT-1/2/3/4, Llama, Claude, Gemini — is a causal language model trained with exactly the objective derived here. Pre-training scales the corpus $\mathcal{D}$ to trillions of tokens and the parameter count $|\theta|$ to hundreds of billions (Ch. 27 onward), but the objective never changes: minimize $-\sum_t \log p_\theta(x_t \mid x_{<t})$ on the empirical distribution, with the causal mask of Definition 2 making all T per-token losses backprop-independent. Perplexity (or its log, the per-token NLL in nats/bits) remains the canonical metric on held-out text. The next chapters lift the architecture from this single-document loss to data-parallel optimization, scaling laws, and post-training (instruction tuning, RLHF).

Chapter 26: Tokenization: BPE algorithm; greedy merge correctness

Motivation

A language model assigns probabilities over sequences drawn from a *finite* token vocabulary $\mathcal{V}$. Recall from Chapter 1 that $\mathcal{V}$ is a finite set; the embedding matrix of Chapter 19 has exactly one row per element of $\mathcal{V}$. Two extremes for choosing $\mathcal{V}$ are unsatisfactory:

- **Word-level vocabulary.** Treat each whitespace-delimited string as a token. Then $\mathcal{V}$ must grow without bound to cover proper nouns, typos, code identifiers, URLs, and morphological variants, and any unseen word is mapped to a single `<unk>` symbol. The OOV problem is unbounded.

- **Character-level vocabulary.** Take $\mathcal{V}$ to be the 256 raw bytes. Then $|\mathcal{V}|$ is small and OOV is impossible, but sequences become extremely long, and the quadratic cost of self-attention (Chapter 25) bites hard.

Subword tokenization is the middle ground: $\mathcal{V}$ is a learned set of byte strings where common substrings (`the`, `ing`, `_function`) are atomic tokens and rare strings fall back to bytes. We study the dominant algorithm: **byte-pair encoding** (BPE), introduced as a compression heuristic by Gage (1994) and adapted to neural machine translation by Sennrich, Haddow & Birch (2016).

Definitions

Fix a finite base alphabet Σ; for byte-level BPE, $\Sigma = \{0, 1, \ldots, 255\}$. A *corpus* is a finite multiset $\mathcal{C} \subset \Sigma^*$ of strings, with multiplicity $c(s) \in \mathbb{Z}_{>0}$.

Definition 0.249 (Vocabulary and segmentation)**.** *A* vocabulary *is a finite set $\mathcal{V} \subset \Sigma^+$ with $\Sigma \subseteq \mathcal{V}$. A* segmentation *of $s \in \Sigma^*$ under $\mathcal{V}$ is a tuple $(t_1, \ldots, t_k) \in \mathcal{V}^k$ with $t_1 t_2 \cdots t_k = s$ (concatenation).*

Definition 0.250 (Merge rule)**.** *A* merge rule *is an ordered pair $(a, b) \in \mathcal{V} \times \mathcal{V}$. Applying it to a sequence $(t_1, \ldots, t_k)$ replaces every adjacent occurrence of (a, b) with the single token ab, scanning left to right and non-overlappingly.*

Definition 0.251 (BPE training)**.** *Given $\mathcal{C}$ and a budget $M \in \mathbb{Z}_{\geq 0}$:*

1. *Initialize $\mathcal{V}_0 = \Sigma$ and segment each $s \in \mathcal{C}$ as the sequence of its bytes.*
2. *For $m = 1, \ldots, M$, count adjacent pair frequencies*

$$f_m(a, b) \;=\; \sum_{s \in \mathcal{C}} c(s) \cdot \#\{\text{occurrences of } (a, b) \text{ in } \mathrm{seg}_{m-1}(s)\}.$$

Pick $(a^\star, b^\star) = \arg\max_{(a,b)} f_m(a, b)$ (ties broken by a fixed total order on $\mathcal{V}^2$). Set $\mathcal{V}_m = \mathcal{V}_{m-1} \cup \{a^\star b^\star\}$ and apply the rule everywhere in the corpus.

The output is the pair $(\mathcal{V}_M, (r_1, \ldots, r_M))$ with $r_m = (a^\star, b^\star)$.

Definition 0.252 (BPE encoding/decoding)**.** *Given $(r_1, \ldots, r_M)$, encode $s \in \Sigma^*$ by: start from the byte sequence of s; repeatedly apply the smallest-rank applicable merge r_i; halt when no rule applies. Decoding is concatenation:* $\mathrm{dec}(t_1, \ldots, t_k) = t_1 t_2 \cdots t_k$.

Theorems

Theorem 0.253 (Determinism of BPE)**.** *Given a fixed merge list $(r_1, \ldots, r_M)$ and a fixed tie-breaking rule, the encoder is a total function* $\mathrm{enc} : \Sigma^* \to \mathcal{V}^*$.

Proof. Each merge replaces two adjacent tokens by one, strictly reducing sequence length by 1. Sequence length is a well-founded measure into $\mathbb{N}$; the inner loop therefore terminates after at most $|s| - 1$ steps. At each step the smallest-rank applicable rule is unique (rules are an ordered list with fixed tie-breaking), so the choice is deterministic. □

Theorem 0.254 (Decoding inverts encoding)**.** *For all $s \in \Sigma^*$,* $\mathrm{dec}(\mathrm{enc}(s)) = s$.

Proof. By induction on the merge sequence: every $t \in \mathcal{V}_m$ is a string in Σ^+, and a merge replaces (a, b) by ab — i.e. concatenation. Hence the concatenation $t_1 \cdots t_k$ is invariant under any sequence of merges starting from the byte sequence of s. □

Theorem 0.255 (Corpus-coverage monotonicity)**.** *Let*

$$L_m \;=\; \sum_{s \in \mathcal{C}} c(s) \cdot |\mathrm{seg}_m(s)|$$

be the total token count of the corpus after m training merges. Then $L_0 \geq L_1 \geq \cdots \geq L_M$, and $L_{m+1} < L_m$ whenever the chosen pair r_{m+1} has positive frequency.

Proof. Applying the merge $r_{m+1} = (a^\star, b^\star)$ replaces each non-overlapping occurrence of the adjacent pair with a single token, decreasing the length of each affected segmentation by exactly the number of occurrences. Summing over the corpus,

$$L_{m+1} \;=\; L_m - f_{m+1}(a^\star, b^\star).$$

Since $f_{m+1} \geq 0$, $L_{m+1} \leq L_m$, with strict decrease iff $f_{m+1}(a^\star, b^\star) > 0$. □

Proposition 0.256 (Vocabulary size bound)**.** $|\mathcal{V}_M| \leq |\Sigma| + M$*, with equality whenever every chosen pair produces a novel token (which holds in the byte-level case since each merge produces a string strictly longer than every $\sigma \in \Sigma$, and pairs are picked with positive count).*

Proof. By induction on m: $|\mathcal{V}_0| = |\Sigma|$, and $|\mathcal{V}_{m+1}| \leq |\mathcal{V}_m| + 1$. □

Remark 0.257 (Greedy vs. globally optimal)*.* Among all vocabularies of size $|\Sigma| + M$, finding the one that minimizes the corpus token count L is NP-hard: a reduction from Set Cover treats each candidate substring as a "set" of corpus positions it can cover, and the search for M substrings minimizing residual length is a weighted set-cover variant. BPE is the *greedy* approximation: at each step, take the merge with the largest immediate compression. There is no guarantee of global optimality; in practice BPE matches or beats more expensive alternatives on downstream perplexity.

Code sketch

```
from collections import Counter

def train_bpe(corpus, M):
    seqs = [tuple(bytes(s, "utf-8")) for s in corpus]
    merges = []
    for _ in range(M):
        pairs = Counter()
        for seq in seqs:
            pairs.update(zip(seq, seq[1:]))
        if not pairs: break
        best = max(pairs.items(), key=lambda kv: (kv[1], kv[0]))[0]
        merges.append(best)
        seqs = [apply_merge(seq, best) for seq in seqs]
    return merges
```

A full runnable implementation, with training, encoding, decoding, and empirical verification of corpus-coverage monotonicity, lives in the chapter notebook.

Connection to LLMs

The tokenizer is the *interface* between raw bytes and the model. GPT-2 / 3 / 4 use byte-level BPE with $|\mathcal{V}| \approx 50{,}257$ (gpt2) up to $\approx 100{,}000$ (cl100k_base). Llama uses SentencePiece BPE on Unicode with $\sim$32k vocab. Claude uses a custom BPE-like scheme. In every case, the chosen $\mathcal{V}$ fixes the row count of the embedding matrix $E \in \mathbb{R}^{|\mathcal{V}| \times d}$ from Chapter 19, and shapes everything downstream: the softmax cost in the LM head, the effective sequence length seen by attention, even what *concepts* the model can express atomically. Chapter 27 explores tokenization pitfalls (numeric tokenization, multilinguality, "SolidGoldMagikarp"-style glitch tokens) that follow directly from BPE's greedy, frequency-driven design.

Chapter 27: Pre-training pipeline: AdamW + warmup + cosine decay + gradient clipping; tiny-GPT training run

Motivation

Chapters 14, 23, 25, and 26 gave us all the pieces in isolation: AdamW, the transformer block, next-token-prediction (NTP) loss with a causal mask, and a tokenizer. This chapter assembles them into the *pre-training pipeline* used by every modern decoder-only language model – GPT-2/3/4, Llama, Mistral. The recipe is short enough to fit on a postcard:

$$\text{corpus} \xrightarrow{\tau} \text{ids} \to (B,T) \to \text{transformer} \to \text{logits} \xrightarrow{\text{CE}} L \xrightarrow{\nabla} g \xrightarrow{\text{clip+AdamW}} \theta_{t+1}.$$

Three engineering details – learning-rate warmup, cosine decay, and gradient clipping – separate "diverges in 200 steps" from "trains stably for 10^{12} tokens."

Definitions

Definition 0.258 (Pre-training pipeline)**.** *Given a corpus $\mathcal{C}$, tokenizer $\tau : \mathcal{C} \to \{1,\dots,V\}^*$, and a causal LM $p_\theta(x_t \mid x_{<t})$,* pre-training *consists of: (i) encoding $\tau(\mathcal{C})$ into $x_1,\dots,x_N$; (ii) at each step t, sampling a batch of B context windows of length T and computing*

$$L_t = -\frac{1}{BT}\sum_{b,i} \log p_\theta(x^{(b)}_{i+1} \mid x^{(b)}_{\le i});$$

(iii) backpropagating, clipping the global gradient norm, and taking an AdamW step under schedule η_t.

Definition 0.259 (Linear warmup)**.** *For horizon $W \in \mathbb{N}$ and peak $\eta_{\max}$,*

$$\eta_t^{warm} = \eta_{\max} \cdot \min\bigl(1, \tfrac{t}{W}\bigr)\,.$$

Definition 0.260 (Cosine decay)**.** *For total horizon T, floor $\eta_{\min}$, and warmup W,*

$$\eta_t \;=\; \eta_{\min} + \tfrac{1}{2}(\eta_{\max} - \eta_{\min})\Bigl(1 + \cos\Bigl(\pi \cdot \tfrac{t-W}{T-W}\Bigr)\Bigr), \quad t \in [W,T].$$

The combined schedule ramps linearly to $\eta_{\max}$ on $[0,W]$ and cosines down to $\eta_{\min}$ on $[W,T]$.

Definition 0.261 (Global-norm gradient clipping)**.** *Given $g = \nabla_\theta L \in \mathbb{R}^P$ and threshold $c > 0$,*

$$\widetilde{g} \;=\; g \cdot \min\Bigl(1, \tfrac{c}{\|g\|_2}\Bigr)\,.$$

Definition 0.262 (Mixed-precision (bf16) training)**.** *Forward/backward run in bf16 (8-bit exponent, 7-bit mantissa); the AdamW* master *weights and moments stay in fp32. Activation memory $\approx \frac{1}{2}$, throughput typically $2\times$–$3\times$ on tensor cores.*

Theorems

Theorem 0.263 (Warmup stabilizes large early gradients)**.** *Let L be β-smooth. The descent lemma gives*

$$L(\theta_{t+1}) \le L(\theta_t) - \eta_t \|\nabla L_t\|^2 + \tfrac{\beta}{2}\eta_t^2 \|\nabla L_t\|^2.$$

At initialization β is effectively unbounded along directions where pre-norm activations have not equilibrated; thus the second-order term dominates whenever $\eta_t > 2/\beta$. Ramping η_t from 0 keeps the second-order penalty controlled. Once LayerNorm scales and Adam's v_t stabilize, β shrinks and a larger $\eta_{\max}$ becomes safe.

Remark 0.264. RAdam (Liu et al. 2020) shows that Adam's variance estimate $\hat{v}_t$ has high variance for small t; warmup is essentially a poor man's variance correction.

Theorem 0.265 (Cosine decay shrinks the noise ball)**.** *Near a minimum $\theta^\star$, a quadratic approximation $L(\theta) \approx \frac{1}{2}(\theta - \theta^\star)^\top H(\theta - \theta^\star)$ implies the SGD iterate variance scales as $\Theta(\eta\sigma^2)$. Decaying $\eta_t \to \eta_{\min}$ shrinks the noise ball, allowing convergence to a sharper minimum.*

Remark 0.266. Empirically (Smith 2017, 1cycle; Loshchilov & Hutter SGDR), cosine outperforms step- and exponential-decay across vision and language. Cosine in particular keeps η near $\eta_{\max}$ during most of training (when exploration matters) and only collapses near the end.

Theorem 0.267 (Clipping preserves a per-step descent guarantee)**.** *Suppose L is β-smooth and we clip the gradient to norm $\le c$. Then the AdamW step at rate η satisfies*

$$\mathbb{E}[L(\theta_{t+1})] \le L(\theta_t) \; - \; \eta\, \mathbb{E}\left[\frac{\widetilde{g}_t^\top \nabla L(\theta_t)}{\sqrt{\hat{v}_t} + \epsilon}\right] \; + \; \frac{\beta \eta^2 c^2}{2}.$$

Proof sketch. Apply β-smoothness to $\theta_{t+1} = \theta_t - \eta m_t/(\sqrt{\hat{v}_t}+\epsilon)$; the second-order term is $O(\eta^2\|m_t\|^2)$, and $\|m_t\| \le c$ by EMA + clipping. The bound holds uniformly over batches, so a single bad gradient can no longer destroy the run. □

Remark 0.268 (Chinchilla compute-optimal scaling). Hoffmann et al. (2022): for fixed compute $C \approx 6ND$ FLOPs (parameters N, tokens D), the loss-minimizing allocation satisfies $N^\star \propto C^{1/2}$ and $D^\star \propto C^{1/2}$. Scale parameters and data *proportionally.* GPT-3 (175B/300B) was undertrained; Chinchilla (70B/1.4T) used the same compute and matched it. This is the rule that determines T_{train} once N and the compute budget are fixed.

Code sketch

```
def lr_schedule(t, W, T, eta_max, eta_min):
    if t < W:
        return eta_max * t / W
    p = (t - W) / (T - W)
    return eta_min + 0.5*(eta_max-eta_min)*(1 + math.cos(math.pi*p))

def clip_global_norm(grads, c=1.0):
    flat = np.concatenate([g.ravel() for g in grads])
    norm = np.linalg.norm(flat)
    return [g * min(1.0, c / (norm + 1e-12)) for g in grads]
```

The notebook implements forward/backward of a 2-layer tiny GPT in pure numpy, drives it with this schedule + clip, and trains to a clearly sub-trivial loss in under 5 minutes on a CPU.

Connection to LLMs

This chapter *is* the recipe. GPT-2 used exactly this pipeline (warmup $\approx$ 2K steps, cosine to $0.1\eta_{\max}$, clip = 1.0, AdamW $\beta_2 = 0.95$). Llama-2 used the same pipeline with longer W, longer T, and bf16 mixed precision. From "tiny GPT in this notebook" to "GPT-4 on a supercluster," only *scale* changes: V from 30 to 10^5, T_{ctx} from 16 to 10^4–10^5, d from 32 to 10^4, N from 10^4 to 10^{12}, D from 10^3 to 10^{13} tokens. The control flow `for step in range(T): batch -> forward -> loss -> backward -> clip -> adamw_step` is byte-identical.

Scaling up: a real torch / MLX GPT on TinyStories

The pure-numpy implementation above is pedagogically clear but stops at 18K parameters and 28-character vocabulary — coherent text generation requires more capacity. This subsection scales up by $\sim$3 orders of magnitude using PyTorch (canonical) and MLX (Apple-native parallel cell).

Model

A 6-layer pre-norm decoder with $d_{\text{model}} = 384$, 6 heads ($d_k = 64$), $d_{\text{ff}} = 1536$, context length 256. With the GPT-2 vocabulary (50,257) and weight-tied embeddings/head, total parameter count lands at $\approx$ 30M — GPT-2-small class but tinier, in the same ballpark as `nanoGPT`. Architecture is identical across the PyTorch and MLX implementations; only the framework changes.

Tokenizer

We use `tiktoken`'s `gpt2` BPE encoding (50,257 merges). The fallback path is character-level when `tiktoken` is unavailable, so the training script always runs.

Training

AdamW with $\beta_1 = 0.9$, $\beta_2 = 0.95$, weight decay 0.1. Linear warmup (5% of steps) then cosine decay from $\eta_{\max} = 3\times10^{-4}$ to $\eta_{\min} = 3\times10^{-5}$. Global gradient norm clipped at 1.0. Batch size 16, $\sim$3000 steps, $\sim$12M training tokens. Wall-clock target on M4 Pro: torch+MPS $\approx$ 45 min, MLX $\approx$ 25 min.

Validation

After training, the model generates $\sim$100-token coherent stories at temperature 0.7 / top-k 40. See `training_run.md` for sample generations and wall-clock numbers from the actual Mac run.

Cross-link to MLX

For Apple-silicon-native execution at $\sim$2–3$\times$ torch+MPS throughput, see `mlx_gpt.py`. Same architecture, same hyperparameters.

Block G — Post-training

Chapter 28: SFT, RLHF (PPO/GRPO), and DPO; train + post-train a tiny GPT

Motivation

Pre-training (Chapter 27) yields a base policy π_{base} optimized solely to assign high probability to next-token sequences from a corpus. Such a model is fluent but neither helpful nor safe. *Post-training* converts π_{base} into an instruction-following policy aligned with human preferences. We treat the three dominant approaches: supervised fine-tuning (SFT), reinforcement learning from human feedback (RLHF) with PPO/GRPO, and direct preference optimization (DPO). The DPO derivation — solving the KL-regularized RLHF objective in closed form and inverting the Bradley–Terry model — is the centerpiece.

Definitions

Definition 0.269 (Supervised fine-tuning)**.** *Given $\mathcal{D}_{\text{SFT}} = \{(x_i, y_i)\}$ of (prompt, response) pairs, continue training π_{base} with the next-token loss of Chapter 25, masking the prompt so the loss only fires on the response:*

$$\mathcal{L}_{\text{SFT}}(\theta) = -\mathbb{E}_{(x,y)\sim\mathcal{D}_{\text{SFT}}} \sum_{t=1}^{|y|} \log \pi_\theta(y_t \mid x, y_{<t}).$$

The result is the reference model $\pi_{\text{ref}} := \pi_{\text{SFT}}$. *This is MLE (Chapter 12) on the conditional response distribution.*

Definition 0.270 (Reward model and Bradley–Terry)**.** *A scalar $r_\phi : (x, y) \to \mathbb{R}$ trained on preference triples (x, y_w, y_l) via the Bradley–Terry (1952) likelihood*

$$\mathbb{P}(y_w \succ y_l \mid x) = \sigma\big(r_\phi(x, y_w) - r_\phi(x, y_l)\big), \quad \ell_{\text{RM}} = -\log \sigma\big(r_\phi(x, y_w) - r_\phi(x, y_l)\big).$$

Definition 0.271 (RLHF objective; Christiano et al. 2017, Ouyang et al. 2022)**.**

$$\mathcal{J}(\theta) = \mathbb{E}_{x\sim\mathcal{D}}\, \mathbb{E}_{y\sim\pi_\theta(\cdot|x)}\big[r_\phi(x, y)\big] - \beta\, D_{\text{KL}}\big(\pi_\theta(\cdot \mid x) \,\|\, \pi_{\text{ref}}(\cdot \mid x)\big), \qquad \beta > 0.$$

The KL leash (Chapter 11) prevents π_θ from drifting far from the SFT initialization.

Definition 0.272 (PPO clipped surrogate; Schulman et al. 2017)**.** *With importance ratio $\rho_t(\theta) = \pi_\theta(a_t \mid s_t)/\pi_{\text{old}}(a_t \mid s_t)$ and advantage $\hat{A}_t$,*

$$\mathcal{L}^{\text{PPO}}(\theta) = \mathbb{E}\Big[\min\big(\rho_t(\theta)\hat{A}_t,\, \text{clip}(\rho_t(\theta), 1-\varepsilon, 1+\varepsilon)\hat{A}_t\big)\Big].$$

Definition 0.273 (GRPO; DeepSeek 2024)**.** *Drop the value network. Sample G responses $\{y^{(i)}\}_{i=1}^G$ per prompt x, score each with r_ϕ, and form the group-normalized advantage $\hat{A}^{(i)} = (r^{(i)} - \mu_g)/(\sigma_g + \epsilon)$ with μ_g, σ_g the empirical mean/std over the G samples.*

Definition 0.274 (DPO; Rafailov et al. 2023)**.**

$$\mathcal{L}_{\text{DPO}}(\theta) = -\mathbb{E}_{(x,y_w,y_l)}\left[\log \sigma\left(\beta \log \frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta \log \frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)}\right)\right].$$

Theorems

Theorem 0.275 (DPO derivation — central result)**.** *Fix a reward r and a reference policy π_{ref} with full support. The unique maximizer of $\mathcal{J}_x(\pi) = \mathbb{E}_{y\sim\pi}[r(x,y)] - \beta\, D_{\text{KL}}(\pi\|\pi_{\text{ref}})$ over distributions $\pi(\cdot \mid x)$ on $\mathcal{V}^*$ is*

$$\pi^*(y \mid x) = \frac{1}{Z(x)}\,\pi_{\text{ref}}(y \mid x)\,\exp\bigl(r(x,y)/\beta\bigr), \qquad Z(x) = \sum_y \pi_{\text{ref}}(y \mid x)\exp(r(x,y)/\beta).$$

Inverting and substituting into the Bradley–Terry log-likelihood yields the DPO loss; the $\log Z(x)$ terms cancel.

Proof. Write the per-prompt objective as

$$\begin{aligned}\mathcal{J}_x(\pi) &= \sum_y \pi(y) r(x,y) - \beta \sum_y \pi(y)\log\frac{\pi(y)}{\pi_{\text{ref}}(y)} \\ &= -\beta \sum_y \pi(y)\log\frac{\pi(y)}{\pi_{\text{ref}}(y)\exp(r(x,y)/\beta)}.\end{aligned}$$

Define the unnormalized measure $q(y) := \pi_{\text{ref}}(y)\exp(r(x,y)/\beta)$ with $Z(x) := \sum_y q(y)$. Then

$$\mathcal{J}_x(\pi) = -\beta\, D_{\text{KL}}\bigl(\pi \,\|\, Z(x)^{-1}q\bigr) + \beta \log Z(x).$$

By Gibbs' inequality (Chapter 11), $D_{\text{KL}} \geq 0$ with equality iff $\pi = Z^{-1}q$, so the unique maximizer is $\pi^*(y \mid x) = Z(x)^{-1}\pi_{\text{ref}}(y \mid x)\exp(r(x,y)/\beta)$.

Solving for the reward gives $r(x,y) = \beta\log\dfrac{\pi^*(y \mid x)}{\pi_{\text{ref}}(y \mid x)} + \beta\log Z(x)$. For a preference triple (x, y_w, y_l), substitute into the Bradley–Terry log-likelihood:

$$\log\sigma\bigl(r(x,y_w) - r(x,y_l)\bigr) = \log\sigma\Biggl(\beta\log\frac{\pi^*(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta\log\frac{\pi^*(y_l|x)}{\pi_{\text{ref}}(y_l|x)} + \beta\log Z(x) - \beta\log Z(x)\Biggr).$$

The two $\beta\log Z(x)$ terms cancel — this is the cancellation that eliminates the intractable partition function. Replacing π^* with π_θ and negating the expectation yields $\mathcal{L}_{\text{DPO}}(\theta)$. □

Proposition 0.276 (PPO trust-region motivation)**.** *Let π_{old} be the policy before update and $\hat{A}$ an advantage estimator. The unclipped surrogate $L(\theta) = \mathbb{E}[\rho_t(\theta)\hat{A}_t]$ is a first-order approximation to the policy improvement $J(\pi_\theta) - J(\pi_{\text{old}})$ in a neighborhood of π_{old} (Kakade–Langford 2002). The clip operator caps $|\rho_t - 1| \leq \varepsilon$ on every step where the unclipped objective would push the policy out of the trust region, yielding a pessimistic lower bound on the surrogate.*

Remark 0.277. A full proof (Achiam et al. 2017) bounds the total-variation distance between successive policies. PPO's clip is a cheap surrogate for TRPO's KL constraint; empirically it preserves monotonic improvement in nearly all settings.

Theorem 0.278 (GRPO advantage is unbiased)**.** *Let $r^{(1)}, \ldots, r^{(G)}$ be rewards of G i.i.d. samples from $\pi_{\text{old}}(\cdot \mid x)$ and $\hat{A}^{(i)} = (r^{(i)} - \mu_g)/(\sigma_g + \epsilon)$. Then μ_g acts as a (nearly) unbiased baseline: subtracting it from $r^{(i)}$ does not change the expected REINFORCE gradient.*

Proof. The REINFORCE estimator with baseline b is $\hat{g} = \sum_i (r^{(i)} - b)\nabla_\theta \log \pi_\theta(y^{(i)} \mid x)$. The baseline lemma:

$$\mathbb{E}_{y\sim\pi}\big[b\,\nabla_\theta \log \pi(y)\big] = b\sum_y \nabla_\theta \pi(y) = b\,\nabla_\theta \sum_y \pi(y) = b\,\nabla_\theta 1 = 0$$

for any b independent of y. The empirical mean μ_g is not strictly independent of the samples, but conditional on a single $y^{(j)}$, the leave-one-out mean over the other $G-1$ samples is independent of $y^{(j)}$; standard control-variate arguments give bias $O(1/G)$, vanishing as $G \to \infty$. Division by σ_g rescales the loss by a data-dependent constant absorbed into the learning rate. □

Code sketch

We continue the tiny GPT of Chapter 27. The pipeline: (1) brief pre-train on a toy corpus to fix π_{base}; (2) SFT on a handful of (prompt, response) pairs to obtain π_{ref}; (3) train a Bradley–Terry reward head on ~ 5 preference triples and verify it ranks $y_w > y_l$; (4) compute one PPO clipped surrogate batch as a sanity check; (5) run DPO for a few hundred steps and verify $\pi_\theta(y_w \mid x)/\pi_\theta(y_l \mid x) > \pi_{\text{ref}}(y_w \mid x)/\pi_{\text{ref}}(y_l \mid x)$.

Connection to LLMs

InstructGPT (Ouyang et al. 2022) introduced the SFT → RM → PPO recipe; GPT-3.5/4, Claude 1/2/3, Llama-2-Chat, and Llama-3-Instruct all use a variant. DeepSeek-R1 (2024) replaced PPO with GRPO, dropping the critic entirely. Zephyr, Tülu, and most open-weight chat models from late 2023 onward use DPO (or its variants IPO, KTO, ORPO) because the closed-form derivation eliminates reward-model training, on-policy sampling, and credit assignment over long sequences. The KL leash to π_{ref} is what prevents post-training from destroying the capabilities laid down in pre-training (Chapter 27): post-training *reshapes* a small region of the policy manifold around the SFT initialization rather than learning a new model from scratch.

Block H — Reinforcement Learning

Chapter 29: MDP foundations: Bellman equations, value iteration, tabular Q-learning

Motivation

Chapter 28 framed RLHF as KL-regularized reward maximization over the entire response distribution and derived DPO and PPO at the *loss* level. To understand *why* those algorithms converge — and to make sense of value baselines, advantages, and the policy gradient itself — we need the formal object underneath every reinforcement-learning method: the *Markov Decision Process* (MDP). This chapter builds MDPs from scratch, derives the Bellman expectation and optimality equations as direct consequences of the definition of expected return (Chapter 10), proves the Bellman operator is a γ-contraction in sup norm, invokes the Banach fixed-point theorem of Chapter 7 to conclude that value iteration converges geometrically, sketches finite-step convergence of policy iteration, and quotes the Watkins–Tsitsiklis theorem connecting tabular Q-learning to the Robbins–Monro stochastic-approximation theory of Chapter 13. Token-by-token generation by an

LLM is itself an MDP — state = prefix, action = next token, reward = terminal preference signal — so every theorem here applies, modulo the combinatorial blow-up of $|\mathcal{S}|$, to the policies trained in Chapters 30–31.

Definitions

Definition 0.279 (Markov Decision Process)**.** *A tuple $(\mathcal{S}, \mathcal{A}, P, r, \gamma)$ with $\mathcal{S}$ a finite state space, $\mathcal{A}$ a finite action space, $P : \mathcal{S} \times \mathcal{A} \to \Delta(\mathcal{S})$ a transition kernel ($P(s' \mid s, a)$ is the conditional probability of s' given (s, a), in the sense of Chapter 8), $r : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$ a bounded reward, and $\gamma \in [0, 1)$ a discount factor.*

Definition 0.280 (Trajectory and return)**.** *A sample path is $\tau = (s_0, a_0, r_0, s_1, a_1, r_1, \dots)$ generated by $a_t \sim \pi(\cdot \mid s_t)$, $s_{t+1} \sim P(\cdot \mid s_t, a_t)$, $r_t = r(s_t, a_t)$. The* discounted return *is*

$$G_t \;:=\; \sum_{k=0}^{\infty} \gamma^k r_{t+k}.$$

Since $|r| \le R_{\max}$ and $\gamma < 1$, $|G_t| \le R_{\max}/(1-\gamma)$ and the series converges absolutely.

Definition 0.281 (Stationary policy and value functions)**.** *A stationary policy is $\pi : \mathcal{S} \to \Delta(\mathcal{A})$. Define*

$$V^\pi(s) := \mathbb{E}_\pi[G_t \mid s_t = s], \qquad Q^\pi(s, a) := \mathbb{E}_\pi[G_t \mid s_t = s, a_t = a],$$
$$V^*(s) := \sup_\pi V^\pi(s), \qquad Q^*(s, a) := \sup_\pi Q^\pi(s, a).$$

Definition 0.282 (Bellman operators)**.** *For $V \in \mathbb{R}^{|\mathcal{S}|}$,*

$$(\mathcal{T}^\pi V)(s) := \sum_a \pi(a \mid s) \left[r(s, a) + \gamma \sum_{s'} P(s' \mid s, a)\, V(s') \right],$$

$$(\mathcal{T}^* V)(s) := \max_a \left[r(s, a) + \gamma \sum_{s'} P(s' \mid s, a)\, V(s') \right].$$

Theorems and proofs

Theorem 0.283 (Bellman expectation equation)**.** $V^\pi = \mathcal{T}^\pi V^\pi$.

Proof. By definition $G_t = r_t + \gamma G_{t+1}$. Take $\mathbb{E}_\pi[\cdot \mid s_t = s]$ and apply linearity of expectation (Chapter 10):

$$V^\pi(s) = \mathbb{E}_\pi[r_t \mid s_t = s] + \gamma\, \mathbb{E}_\pi[G_{t+1} \mid s_t = s].$$

By the tower property,

$$\mathbb{E}_\pi[r_t \mid s_t = s] = \sum_a \pi(a \mid s)\, r(s, a),$$

and using the Markov property of P,

$$\mathbb{E}_\pi[G_{t+1} \mid s_t = s] = \sum_{a, s'} \pi(a \mid s)\, P(s' \mid s, a)\, V^\pi(s').$$

Combining gives $V^\pi(s) = (\mathcal{T}^\pi V^\pi)(s)$. □

Theorem 0.284 (Bellman optimality equation)**.** *$V^* = \mathcal{T}^* V^*$, and there exists a deterministic policy π^* attaining the maximum.*

Proof. Apply the same calculation at the optimal policy. The Bellman principle of optimality says that, conditional on being in state s, the optimal expected return decomposes as the maximum over the next action a of the immediate reward $r(s,a)$ plus the discounted optimal expected return from $s' \sim P(\cdot \mid s,a)$, giving $V^*(s) = \max_a [r(s,a) + \gamma \sum_{s'} P(s' \mid s,a) V^*(s')]$. Because $\mathcal{A}$ is finite, the arg max exists; defining $\pi^*(s) \in \arg\max$ produces a deterministic optimal policy. □

Theorem 0.285 (Contraction)**.** *$\mathcal{T}^\pi$ and $\mathcal{T}^*$ are γ-contractions in sup norm $\|V\|_\infty := \max_s |V(s)|$.*

Proof. For any $V, V' \in \mathbb{R}^{|\mathcal{S}|}$ and any state s,

$$|(\mathcal{T}^\pi V)(s) - (\mathcal{T}^\pi V')(s)| = \gamma \left| \sum_a \pi(a \mid s) \sum_{s'} P(s' \mid s,a) \, [V(s') - V'(s')] \right|.$$

Bounding the absolute value by the absolute sum and using $\sum_a \pi(a \mid s) = \sum_{s'} P(s' \mid s,a) = 1$ yields

$$|(\mathcal{T}^\pi V)(s) - (\mathcal{T}^\pi V')(s)| \le \gamma \|V - V'\|_\infty.$$

Taking the max over s gives the sup-norm bound. For $\mathcal{T}^*$, use the elementary inequality $|\max_a f(a) - \max_a g(a)| \le \max_a |f(a) - g(a)|$ to reduce to the previous case. □

Corollary 0.286 (Value iteration converges geometrically)**.** *Let $V_{k+1} := \mathcal{T}^* V_k$. By Banach's fixed-point theorem (Chapter 7) on the complete normed space $(\mathbb{R}^{|\mathcal{S}|}, \|\cdot\|_\infty)$,*

$$\|V_k - V^*\|_\infty \;\le\; \gamma^k \|V_0 - V^*\|_\infty.$$

In particular, $\log \|V_k - V^*\|_\infty$ has slope $\log \gamma$ as a function of k.

Theorem 0.287 (Policy iteration; finite termination)**.** *Alternate (i) policy evaluation $V^\pi = (I - \gamma P^\pi)^{-1} r^\pi$, where $P^\pi(s,s') = \sum_a \pi(a \mid s) P(s' \mid s,a)$; the matrix $I - \gamma P^\pi$ is invertible because $\rho(\gamma P^\pi) \le \gamma < 1$. (ii) Greedy improvement $\pi'(s) := \arg\max_a [r(s,a) + \gamma \sum_{s'} P(s' \mid s,a) V^\pi(s')]$. Then $V^{\pi'} \ge V^\pi$ pointwise, and the algorithm terminates in finitely many iterations at π^*.*

Sketch. By construction, $\mathcal{T}^{\pi'} V^\pi \ge V^\pi = \mathcal{T}^\pi V^\pi$. Applying $\mathcal{T}^{\pi'}$ repeatedly to both sides and using monotonicity ($V \le V' \Rightarrow \mathcal{T}^{\pi'} V \le \mathcal{T}^{\pi'} V'$) plus the contraction limit $(\mathcal{T}^{\pi'})^k V^\pi \to V^{\pi'}$ yields $V^{\pi'} \ge V^\pi$. The deterministic policy space has size $|\mathcal{A}|^{|\mathcal{S}|} < \infty$; each strict improvement visits a new policy; once $\pi' = \pi$, π is a fixed point of $\mathcal{T}^*$, hence equals π^*. □

Theorem 0.288 (Q-learning convergence; Watkins 1989, Tsitsiklis 1994)**.** *Consider the update*

$$Q_{t+1}(s_t, a_t) = (1 - \alpha_t) \, Q_t(s_t, a_t) + \alpha_t \left[r_t + \gamma \max_{a'} Q_t(s_{t+1}, a') \right].$$

If every (s,a) is visited infinitely often and the per-pair step sizes obey the Robbins–Monro conditions of Chapter 13, $\sum_t \alpha_t = \infty$, $\sum_t \alpha_t^2 < \infty$, then $Q_t \to Q^*$ almost surely.

Sketch. The update is precisely the Robbins–Monro stochastic-approximation recursion for the fixed point of $\mathcal{T}^*$, which is a γ-contraction in sup norm. The sample $s_{t+1} \sim P(\cdot \mid s_t, a_t)$ replaces the expectation $\sum_{s'} P(s' \mid s,a)$ by an unbiased one-sample estimator, producing a martingale-difference noise with bounded variance (since rewards and Q are bounded). Tsitsiklis' generalization of stochastic approximation to contraction operators gives almost-sure convergence under the stated step-size and visitation conditions. □

Code sketch

The notebook builds a 4×4 GridWorld with reward -1 per step and $+10$ at an absorbing goal corner, runs value iteration with $\gamma = 0.9$, and verifies that the slope of $\log \|V_k - V^*\|_\infty$ versus k matches $\log \gamma$. It extracts the greedy policy from Q^*, re-derives V^* via policy iteration in dramatically fewer outer iterations, then runs tabular Q-learning for 5000 episodes with $\alpha_t = 1/(1 + \text{visits}(s, a))$ and ε-greedy exploration ($\varepsilon = 0.1$), confirming $\|Q_t - Q^*\|_\infty \to 0$. A final cell runs REINFORCE on a 3-state, 2-action MDP with softmax policy parameterization and Monte-Carlo returns, previewing Chapter 30.

Connection to LLMs / RLHF

A language model is an MDP whose state is the prompt-plus-prefix $s_t = (x, y_{<t})$, action is the next token $a_t = y_t \in \mathcal{V}$, transition is deterministic ($s_{t+1} = s_t \oplus a_t$), and the reward is sparse and terminal: $r_t = 0$ for $t < T$ and $r_T = r_\phi(x, y)$ from a learned reward model. Under this MDP the KL-regularized objective of Chapter 28 is exactly expected discounted return with per-step reward shaped by $-\beta \log \frac{\pi_\theta(a_t|s_t)}{\pi_{\text{ref}}(a_t|s_t)}$. The Bellman optimality equation in this terminal-reward MDP collapses to the closed-form

$$\pi^*(y \mid x) \propto \pi_{\text{ref}}(y \mid x) \exp(r(x, y)/\beta),$$

which is precisely the central DPO identity (Chapter 28, Theorem 1). Value iteration and policy iteration are intractable because $|\mathcal{S}|$ is exponential in context length, forcing parametric methods — function approximation plus policy gradients — the subject of Chapter 30 (REINFORCE, advantage estimation, GAE) and Chapter 31 (actor-critic / PPO machinery used in InstructGPT, Claude, and Llama-3).

Chapter 30: Value-based deep RL: function approximation, DQN, max-entropy framework

Motivation

Chapter 29 solved the MDP problem under two unrealistic assumptions: the state space is finite and small enough to enumerate (so Q is a table), and learning is on-policy from full sweeps of $\mathcal{S} \times \mathcal{A}$. Real problems — Atari from raw pixels, continuous control, language models — break both. We need (a) a function approximator $Q_\theta(s, a)$ that generalizes across states and (b) a learning rule that is off-policy, reusing data collected under earlier behavior policies.

The naive combination of (a), (b), and the bootstrapping target of TD-learning is unstable: the *deadly triad* (Sutton & Barto, Ch 11). DQN (Mnih et al. 2015) cleared the triad with two simple tricks — a replay buffer and a target network — and won Atari. We then move to the maximum-entropy reformulation, where adding $\alpha H(\pi)$ to the reward turns the hard max into a log-sum-exp, gives the closed-form Boltzmann policy $\pi^* \propto \exp(Q/\alpha)$, and is the same derivation that produces the RLHF/DPO closed form $\pi^* \propto \pi_{\text{ref}} \exp(r/\beta)$ used in Chapter 28 and central to Chapter 31.

Definitions

Definition 0.289 (Function approximator)**.** *$Q_\theta(s, a) : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$ parameterized by $\theta \in \mathbb{R}^d$. Linear: $Q_\theta(s, a) = \phi(s, a)^\top \theta$ for a feature map ϕ. Neural: an MLP (Ch 15) with input s and $|\mathcal{A}|$*

outputs, trained by backprop (Ch 18).

Definition 0.290 (Mean-squared TD error — the DQN loss)**.** *For a transition (s, a, r, s') drawn from a replay buffer $\mathcal{D}$,*

$$\mathcal{L}(\theta) = \mathbb{E}_{(s,a,r,s')\sim\mathcal{D}}\left[\big(r + \gamma \max_{a'} Q_{\theta^-}(s', a') - Q_\theta(s, a)\big)^2\right],$$

where θ^- is a frozen target network*.*

Definition 0.291 (Replay buffer and target update)**.** *$\mathcal{D}$ is a FIFO of past transitions; minibatches are sampled iid for the gradient step. Target network update:* hard $\theta^- \leftarrow \theta$ *every K steps, or* Polyak $\theta^- \leftarrow \tau\theta + (1-\tau)\theta^-$ *with $\tau \ll 1$.*

Definition 0.292 (Maximum-entropy RL objective)**.** *For temperature $\alpha > 0$,*

$$J_\alpha(\pi) = \mathbb{E}_\pi\left[\sum_{t=0}^{\infty} \gamma^t \big(r_t + \alpha\, H(\pi(\cdot \mid s_t))\big)\right], \qquad H(p) = -\sum_a p(a) \log p(a).$$

Definition 0.293 (Soft value functions)**.**

$$V^\pi_{\text{soft}}(s) = \mathbb{E}_\pi[r + \gamma V^\pi_{\text{soft}}(s') + \alpha H(\pi(\cdot \mid s))], \qquad Q^\pi_{\text{soft}}(s, a) = r(s, a) + \gamma\, \mathbb{E}_{s'}[V^\pi_{\text{soft}}(s')].$$

Definition 0.294 (Soft Bellman optimality operator)**.**

$$(\mathcal{T}^*_\alpha Q)(s, a) = r(s, a) + \gamma\, \mathbb{E}_{s'}\Big[\alpha \log \sum_{a'} \exp\big(Q(s', a')/\alpha\big)\Big].$$

Theorems

Theorem 0.295 (The deadly triad; Baird 1995)**.** *Off-policy learning + bootstrapping + linear function approximation can diverge.*

Example 0.296 (Baird's 7-state counterexample)*.* Seven states $\{1, \dots, 7\}$ and 8-dimensional features

$$\phi(s) = 2e_s + e_8 \ (s = 1..6), \qquad \phi(7) = e_7 + 2e_8.$$

Two actions: *solid* deterministically goes to state 7; *dashed* goes uniformly to one of states 1..6. All rewards are 0, $\gamma = 0.99$. Target policy always picks *solid*; behavior policy picks solid with probability $1/7$, so the off-policy importance ratio is $\rho = 7$ on solid steps and 0 on dashed steps. Initialize $w = (1, 1, 1, 1, 1, 1, 10, 1)^\top$. The semi-gradient TD(0) update

$$w \leftarrow w + \alpha\, \rho\, \big(0 + \gamma \phi(7)^\top w - \phi(s)^\top w\big)\, \phi(s)$$

satisfies $\|w_t\| \to \infty$.

Why it diverges. Under the behavior state distribution d_b, the expected semi-gradient update is $w_{t+1} = w_t + \alpha A w_t$ with

$$A = \mathbb{E}_{s\sim d_b}\big[\rho(s)\, \phi(s)\, (\gamma\phi(7) - \phi(s))^\top\big].$$

Stability requires every eigenvalue of A to lie in the open left half-plane. In Baird's setup, only solid transitions contribute (dashed has $\rho = 0$); these contribute $\phi(s)(\gamma\phi(7) - \phi(s))^\top$ for each s. Note $\gamma\phi(7) - \phi(s) = -2e_s + \gamma e_7 + (2\gamma - 1)e_8$, so for $\gamma > 1/2$ the shared-feature direction e_8 accumulates

a positive contribution across states, producing a positive eigenvalue of A along e_8. Hence $\|w_t\|$ grows geometrically.

Role of each leg. Without function approximation (tabular: $\phi(s) = e_s$), A is diagonal-dominant and negative-definite, and TD(0) converges. Without bootstrapping (Monte Carlo), the loss is convex regression and SGD converges. Without off-policy ($\rho \equiv 1$), Tsitsiklis–Van Roy (1997) show the on-policy projection $\Pi\mathcal{T}^\pi$ is a γ-contraction in the d_π-weighted norm, so TD(0) converges to the projected fixed point. Removing any one leg restores stability; keeping all three permits divergence. The notebook reproduces $\|w\|$ growing from ≈ 10 to > 300 in 1000 steps. □

Theorem 0.297 (DQN's two tricks rescue stability — informal)**.** *Both the replay buffer and the target network are necessary for stable training of a neural-network Q-function.*

Argument. **Replay buffer.** SGD's stochastic-approximation theorem (Ch 13) requires gradient samples to be approximately iid. Sequential MDP transitions are highly correlated. Buffering $\sim 10^5$–10^6 past transitions and sampling minibatches uniformly (a) decorrelates the gradient signal, (b) reuses each transition, and (c) makes the data distribution close to stationary so the SGD analysis applies.

Target network. With a single network the bootstrap target $r + \gamma \max_{a'} Q_\theta(s', a')$ shifts every gradient step, coupling $Q_\theta(s, a)$ to itself and producing oscillation. Freezing θ^- for K steps decouples target from prediction; the loss is then a standard regression with stationary targets, which AdamW (Ch 14) handles. Mnih et al. 2015 verified both tricks are necessary in ablations. □

Theorem 0.298 (Max-entropy RL = constrained optimization; closed-form policy)**.** *For fixed s and $Q(s, \cdot)$, the maximizer of*

$$\sum_a \pi(a)Q(s, a) + \alpha H(\pi) \quad s.t. \quad \sum_a \pi(a) = 1,\ \pi(a) \geq 0$$

is

$$\pi^*(a \mid s) = \frac{\exp(Q(s, a)/\alpha)}{\sum_{a'} \exp(Q(s, a')/\alpha)}, \qquad \textit{value} = \alpha \log \sum_a \exp(Q(s, a)/\alpha).$$

Proof. Lagrangian

$$\mathcal{L} = \sum_a \pi(a)Q(s, a) - \alpha \sum_a \pi(a) \log \pi(a) - \lambda\Big(\sum_a \pi(a) - 1\Big).$$

Stationarity $\partial\mathcal{L}/\partial\pi(a) = Q(s, a) - \alpha(\log \pi(a) + 1) - \lambda = 0$ gives $\pi(a) = \exp((Q(s, a) - \lambda - \alpha)/\alpha)$. Imposing $\sum_a \pi(a) = 1$ pins λ and produces the softmax. Substituting back yields $\sum_a \pi^*(a)Q(s, a) + \alpha H(\pi^*) = \alpha \log \sum_a \exp(Q(s, a)/\alpha)$. The objective is strictly concave (entropy is strictly concave; reward is linear), so the KKT point is the unique global maximum; positivity is automatic. □

Remark 0.299 (RLHF correspondence). Replace $H(\pi)$ by $-D_{\mathrm{KL}}(\pi \| \pi_{\mathrm{ref}})$: the same Lagrangian gives $\pi^* \propto \pi_{\mathrm{ref}} \exp(r/\beta)$, the starting point of the DPO derivation in Chapter 28.

Theorem 0.300 (Soft Bellman is a γ-contraction)**.** *$\mathcal{T}^*_\alpha$ is a γ-contraction on $(\mathbb{R}^{|\mathcal{S}|\times|\mathcal{A}|}, \|\cdot\|_\infty)$, hence has a unique fixed point Q^*_{soft} which value iteration $Q_{k+1} = \mathcal{T}^*_\alpha Q_k$ reaches at rate γ^k.*

Proof. The log-sum-exp $L_\alpha(x) := \alpha \log \sum_i \exp(x_i/\alpha)$ is 1-Lipschitz in $\|\cdot\|_\infty$. Indeed $\nabla L_\alpha(x) = \mathrm{softmax}(x/\alpha) =: p$ is a probability vector ($p_i \geq 0,\ \sum p_i = 1$), so $\|p\|_1 = 1$. By the mean-value

theorem $L_\alpha(x) - L_\alpha(y) = p_\xi^\top(x-y)$ for some ξ on the segment between x and y; Hölder gives $|p_\xi^\top(x-y)| \le \|p_\xi\|_1 \|x-y\|_\infty = \|x-y\|_\infty$. Now for any Q_1, Q_2 and any (s,a),

$$|\mathcal{T}_\alpha^* Q_1(s,a) - \mathcal{T}_\alpha^* Q_2(s,a)| = \gamma \left|\mathbb{E}_{s'}[L_\alpha(Q_1(s',\cdot)) - L_\alpha(Q_2(s',\cdot))]\right| \le \gamma\, \mathbb{E}_{s'} \|Q_1(s',\cdot) - Q_2(s',\cdot)\|_\infty \le \gamma \|Q_1 - Q_2\|_\infty.$$

Taking sup over (s,a) yields $\|\mathcal{T}_\alpha^* Q_1 - \mathcal{T}_\alpha^* Q_2\|_\infty \le \gamma \|Q_1 - Q_2\|_\infty$. Banach gives existence, uniqueness of Q^*_{soft}, and geometric convergence $\|Q_k - Q^*_{\text{soft}}\|_\infty \le \gamma^k \|Q_0 - Q^*_{\text{soft}}\|_\infty$. □

Theorem 0.301 (Soft → hard as $\alpha \to 0$)**.** *$\alpha \log \sum_a \exp(Q(s,a)/\alpha) \to \max_a Q(s,a)$ as $\alpha \to 0^+$, and the soft policy π^*_α collapses to the greedy policy (uniform over the argmax set).*

Proof. Let $M = \max_a Q(s,a)$, $\mathcal{A}^* = \{a : Q(s,a) = M\}$, $k = |\mathcal{A}^*|$. Then

$$\alpha \log \sum_a \exp(Q(s,a)/\alpha) = M + \alpha \log\Big(k + \sum_{a \notin \mathcal{A}^*} \exp\big((Q(s,a) - M)/\alpha\big)\Big).$$

Each suboptimal exponent is strictly negative, so its exponential vanishes as $\alpha \to 0$, leaving $M + \alpha \log k \to M$. The softmax mass concentrates on $\mathcal{A}^*$ at $1/k$ each. This is the Ch 16 softmax-temperature limit applied to Q/α. □

Code sketch

The notebook implements (i) linear TD(0) on a 5-state chain, recovering the closed-form V; (ii) Baird's counterexample, with $\|w\|$ exploding; (iii) a numpy DQN on a hand-coded CartPole, showing average episode length growing from ~ 22 to > 100 without any deep-learning library; (iv) soft Q-learning on a 4×4 grid for $\alpha \in \{0.01, 0.5, 5\}$, verifying $Q_{\text{soft}} \to Q^*$ as $\alpha \to 0$; (v) the soft Bellman fixed point on a 3-state MDP, with zero residual, alongside the analogous RLHF closed form.

Connection to LLMs

Maximum-entropy RL is the bridge to alignment. Compare the two per-state objectives:

- **Soft RL (this chapter):** $\max_\pi \sum_a \pi(a) Q(s,a) + \alpha H(\pi) \;\Rightarrow\; \pi^*(a \mid s) \propto \exp(Q(s,a)/\alpha)$.
- **RLHF (Ch 28):** $\max_\pi \mathbb{E}_{y\sim\pi}[r(x,y)] - \beta D_{\text{KL}}(\pi \| \pi_{\text{ref}}) \;\Rightarrow\; \pi^*(y \mid x) \propto \pi_{\text{ref}}(y \mid x) \exp(r(x,y)/\beta)$.

The two derivations are the same Lagrangian: replace the entropy bonus $-\alpha \sum_a \pi(a) \log \pi(a)$ with the KL leash $-\beta \sum_a \pi(a) \log(\pi(a)/\pi_{\text{ref}}(a))$, which adds a constant shift inside the log. Stationarity then yields $\pi(a) \propto \pi_{\text{ref}}(a) \exp(Q(s,a)/\beta)$, the RLHF closed form. DPO (Ch 28) inverts this expression and substitutes into the Bradley–Terry preference likelihood, where the partition function $Z(x)$ cancels.

Architecturally, DQN's target network plays the same role as RLHF's π_{ref}: a slowly-moving anchor that prevents the bootstrap (or the KL leash) from chasing its own tail. In DQN the anchor is refreshed every K gradient steps; in RLHF it is held fixed for the entire fine-tuning run. In both cases the anchor turns a non-stationary self-referential objective into a sequence of stationary regression problems that AdamW (Ch 14) can reliably minimize. Chapter 31 closes the loop, fusing policy gradient (REINFORCE/GRPO) with the soft-RL view to derive the alignment loss actually used by modern LLMs.

Chapter 31: Policy gradient, GRPO, and the RLHF/DPO bridge: tiny-GPT alignment loop

Motivation

Chapters 29 and 30 established the MDP machinery and the maximum-entropy framework; Chapter 28 derived the closed-form DPO solution to the KL-regularized RLHF objective. What remains is the *online* alignment loop: an algorithm that takes a pre-trained language model (Chapter 27), a reward signal, and produces a better policy by gradient ascent on $\mathbb{E}_{y\sim\pi_\theta}[r(x,y)]$. We derive that loop from first principles, starting with the policy gradient theorem (Sutton, 1999), reducing variance with baselines and GAE, enforcing a trust region with PPO, dropping the value function with GRPO (DeepSeek, 2024), and ending by running GRPO on the Chapter 27 tiny GPT — closing the 31-chapter loop with a measurable reward improvement on generated text.

Definitions

Definition 0.302 (Stochastic policy and trajectory distribution)**.** *A parameterized stochastic policy is $\pi_\theta(a \mid s)$. A trajectory $\tau = (s_0, a_0, r_0, \ldots, s_T)$ has return $R(\tau) = \sum_{t=0}^{T-1} \gamma^t r_t$ and distribution*

$$p_\theta(\tau) = p_0(s_0) \prod_{t=0}^{T-1} \pi_\theta(a_t \mid s_t)\, P(s_{t+1} \mid s_t, a_t).$$

The objective is $J(\theta) = \mathbb{E}_{\tau\sim p_\theta}[R(\tau)]$. The discounted state distribution $d^\pi(s) = (1-\gamma)\sum_{t\geq 0}\gamma^t \Pr_\pi[s_t = s]$ and value functions V^π, Q^π are inherited from Chapter 29; the advantage is $A^\pi(s,a) = Q^\pi(s,a) - V^\pi(s)$.

Definition 0.303 (Score function)**.** *The likelihood-ratio identity for the trajectory distribution is*

$$\nabla_\theta \log p_\theta(\tau) = \sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t \mid s_t),$$

since p_0 and P are θ-independent.

Definition 0.304 (Generalized Advantage Estimation, Schulman 2016)**.** *With TD residual $\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$,*

$$\hat{A}_t^{\mathrm{GAE}(\gamma,\lambda)} = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l}, \qquad \lambda \in [0,1].$$

Definition 0.305 (TRPO and PPO surrogates)**.** *Let $r_t(\theta) = \pi_\theta(a_t \mid s_t)/\pi_{\mathrm{old}}(a_t \mid s_t)$. TRPO maximizes $\mathbb{E}[r_t(\theta)\hat{A}_t]$ subject to $\mathbb{E}_s D_{\mathrm{KL}}(\pi_{\mathrm{old}} \| \pi_\theta) \leq \delta$. PPO replaces the constraint by a clipped objective:*

$$\mathcal{L}^{\mathrm{PPO}}(\theta) = \mathbb{E}\Big[\min\big(r_t(\theta)\hat{A}_t,\ \mathrm{clip}(r_t(\theta), 1-\varepsilon, 1+\varepsilon)\hat{A}_t\big)\Big].$$

Definition 0.306 (GRPO, DeepSeek 2024)**.** *For each prompt x sample G completions $\{y_1, \ldots, y_G\}$ from $\pi_{\mathrm{old}}(\cdot \mid x)$ with rewards $r_1, \ldots, r_G$. The group-relative advantage is*

$$\hat{A}_i \;=\; \frac{r_i - \mathrm{mean}(r_{1:G})}{\mathrm{std}(r_{1:G}) + \epsilon},$$

broadcast to every token of y_i. The PPO clipped surrogate is then applied per token, with a KL penalty $\beta\, \mathbb{E}\, D_{\mathrm{KL}}(\pi_\theta \| \pi_{\mathrm{ref}})$ to the SFT reference.

Theorems

Theorem 0.307 (Policy gradient theorem; Sutton 1999)**.** *Under conditions allowing exchange of* ∇_θ *and* $\int$*,*

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau\sim p_\theta}\left[\sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t \mid s_t)\, Q^{\pi_\theta}(s_t, a_t)\right] = \frac{1}{1-\gamma}\, \mathbb{E}_{(s,a)\sim d^{\pi_\theta}}\left[\nabla_\theta \log \pi_\theta(a \mid s)\, Q^{\pi_\theta}(s,a)\right].$$

Proof. Begin with $J(\theta) = \int p_\theta(\tau) R(\tau)\, d\tau$ and apply the log-derivative identity $\nabla p_\theta = p_\theta \nabla \log p_\theta$:

$$\nabla_\theta J(\theta) = \int p_\theta(\tau)\, \nabla_\theta \log p_\theta(\tau)\, R(\tau)\, d\tau = \mathbb{E}[\nabla_\theta \log p_\theta(\tau)\, R(\tau)].$$

Substitute the factorized score: $\nabla_\theta \log p_\theta(\tau) = \sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)$ since p_0 and P do not depend on θ. Causality of rewards: $r_{t'}$ for $t' < t$ is uncorrelated with a_t given history, so $\mathbb{E}[\nabla \log \pi_\theta(a_t \mid s_t)\, r_{t'}] = 0$ for $t' < t$. Therefore

$$\nabla_\theta J(\theta) = \mathbb{E}\left[\sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t) \sum_{t'\geq t} \gamma^{t'} r_{t'}\right] = \mathbb{E}\left[\sum_t \gamma^t \nabla_\theta \log \pi_\theta(a_t \mid s_t)\, Q^{\pi_\theta}(s_t, a_t)\right],$$

using $\mathbb{E}[\sum_{t'\geq t} \gamma^{t'-t} r_{t'} \mid s_t, a_t] = Q^\pi(s_t, a_t)$. Folding γ^t into the discounted state distribution gives the second form. □

Lemma 0.308 (Baselines are unbiased)**.** *For any function* $b : S \to \mathbb{R}$*,*

$$\mathbb{E}_{a\sim\pi_\theta(\cdot|s)}[\nabla_\theta \log \pi_\theta(a \mid s)\, b(s)] = 0.$$

Proof. Pull $b(s)$ outside the action expectation and use the log-trick in reverse:

$$\sum_a \pi_\theta(a \mid s) \nabla_\theta \log \pi_\theta(a \mid s) = \sum_a \nabla_\theta \pi_\theta(a \mid s) = \nabla_\theta \sum_a \pi_\theta(a \mid s) = \nabla_\theta 1 = 0. \qquad \square$$

Substituting $b = V^\pi$ yields the advantage form $\nabla_\theta J = \mathbb{E}[\nabla_\theta \log \pi_\theta(a \mid s)\, A^\pi(s,a)]$.

Lemma 0.309 (GAE recursion and bias-variance)**.** $\hat{A}_t^{\mathrm{GAE}(\gamma,\lambda)}$ *satisfies* $\hat{A}_t = \delta_t + \gamma\lambda\, \hat{A}_{t+1}$*. With* $\lambda = 0$*,* $\hat{A}_t = \delta_t$ *(one-step TD; low variance, biased by bootstrap). With* $\lambda = 1$*, telescoping gives* $\hat{A}_t = \sum_{l\geq 0} \gamma^l r_{t+l} - V(s_t)$ *(Monte Carlo minus baseline; unbiased, high variance).*

Proof. Direct from $\sum_{l\geq 0} (\gamma\lambda)^l \delta_{t+l} = \delta_t + \gamma\lambda \sum_{l\geq 0} (\gamma\lambda)^l \delta_{t+1+l}$. □

Theorem 0.310 (TRPO surrogate-improvement inequality; Schulman 2015)**.** *For policies* π, π'*,*

$$J(\pi') \geq J(\pi) + \mathbb{E}_{(s,a)\sim d^\pi, \pi'(\cdot|s)}[A^\pi(s,a)] - \frac{2\varepsilon\gamma}{(1-\gamma)^2}\, D_{\mathrm{KL}}^{\max}(\pi, \pi'),$$

with $\varepsilon = \max_{s,a} |A^\pi(s,a)|$ *and* $D_{\mathrm{KL}}^{\max} = \max_s D_{\mathrm{KL}}(\pi(\cdot \mid s) \| \pi'(\cdot \mid s))$*.*

Sketch. The Kakade–Langford performance-difference lemma reads $J(\pi') - J(\pi) = \frac{1}{1-\gamma} \mathbb{E}_{(s,a)\sim d^{\pi'}, \pi'}[A^\pi(s,a)]$. Replacing $d^{\pi'}$ by d^π introduces an error bounded by the total-variation distance $\|d^{\pi'} - d^\pi\|_{TV}$. Pinsker's inequality (Chapter 11) bounds this by $\sqrt{\frac{1}{2} D_{\mathrm{KL}}^{\max}}$, and a discounted-chain argument supplies the $(1-\gamma)^{-2}$ factor. See Kakade–Langford (2002) and Schulman (2015) for the full proof. □

Proposition 0.311 (PPO is a first-order trust region)**.** *Around* θ_{old}*,* $r_t(\theta) = 1 + \nabla_\theta \log \pi_\theta(a_t \mid s_t)\big|_{\theta_{\text{old}}}^\top (\theta - \theta_{\text{old}}) + O(\|\theta - \theta_{\text{old}}\|^2)$*. Bounding* $r_t \in [1-\varepsilon, 1+\varepsilon]$ *thus enforces a per-sample first-order surrogate of the KL trust region that TRPO solves with a Fisher-vector product. Empirically the clip is loose enough to be cheap and tight enough to mimic the trust region.*

Theorem 0.312 (GRPO group baseline is asymptotically unbiased)**.** *Sample* $y_i \sim \pi_{\text{old}}(\cdot \mid x)$ *i.i.d., set* $\bar{r} = \frac{1}{G}\sum_j r_j$ *and* $\hat{A}_i = (r_i - \bar{r})/(\hat{\sigma} + \epsilon)$*. As* $G \to \infty$*,* $\bar{r} \to \mu(x) := \mathbb{E}_{y\sim\pi_{\text{old}}}[r(x,y)]$ *and* $\hat{\sigma} \to \sigma(x)$*, both independent of any single* y_i*. Then*

$$\mathbb{E}_{y_i}[\hat{A}_i \nabla \log \pi_\theta(y_i \mid x)] = \frac{1}{\sigma(x)}\big(\mathbb{E}[r_i \nabla \log \pi_\theta] - \mu(x)\,\mathbb{E}[\nabla \log \pi_\theta]\big) = \frac{1}{\sigma(x)} \nabla J(x),$$

since the second term vanishes by Lemma 0.308.

Remark on finite G*.* For finite G, $\bar{r}$ depends on y_i through its own term, so an $O(1/G)$ bias appears. DeepSeek (2024) shows empirically that this bias is negligible for $G \geq 4$ and that the variance reduction more than compensates. The engineering payoff is dramatic: *no value network.* □

Theorem 0.313 (DPO–RLHF equivalence; recap of Chapter 28)**.** *The KL-regularized RLHF objective* $\max_\pi \mathbb{E}_{x,y\sim\pi}[r(x,y)] - \beta D_{\text{KL}}(\pi \| \pi_{\text{ref}})$ *has closed-form maximizer*

$$\pi^*(y \mid x) = \frac{1}{Z(x)} \pi_{\text{ref}}(y \mid x) \exp\big(r(x,y)/\beta\big).$$

Inverting yields $r(x,y) = \beta \log \frac{\pi^*(y|x)}{\pi_{\text{ref}}(y|x)} + \beta \log Z(x)$*. Substituted into the Bradley–Terry preference likelihood,* $Z(x)$ *cancels in the difference* $r(x, y_w) - r(x, y_l)$*, yielding the DPO loss directly on* π_θ *with no reward model.*

Code sketch

The notebook walks the algorithmic ladder. (1) REINFORCE on a 3-state MDP with softmax policy and Monte Carlo returns. (2) REINFORCE with a learned tabular state-value baseline; gradient variance drops by an order of magnitude. (3) GAE on a 5-state chain with $\lambda \in \{0, 0.5, 0.95, 1\}$, displaying the bias-variance trade-off. (4) PPO with a 2-layer MLP policy and hand backprop on a CartPole-style env from Chapter 30. (5) GRPO warm-up on a synthetic 5-arm contextual bandit. (6) The climax: GRPO on the Chapter 27 tiny GPT, where the reward fires when the generated continuation contains a target substring; the KL to π_{ref} is monitored to confirm reward improvement without policy collapse.

Connection to LLMs

Modern post-training is three coordinates on the same KL-regularized landscape:

- **RLHF (PPO + RM).** SFT → reward model → PPO with KL penalty to the SFT reference. Used by InstructGPT, GPT-3.5/4, Claude, Llama-2-Chat. Pros: arbitrary, learned reward signals. Cons: four models in memory (policy, value, reward, reference); reward hacking; brittle infrastructure.
- **DPO.** Skip the RM; use the closed-form solution of the KL-regularized objective to write a direct preference loss on π_θ vs π_{ref}. Used by Zephyr, Tülu, most open-weight chat models since late 2023. Pros: one model, one loss, no online sampling. Cons: limited to pairwise preferences; bounded by offline-dataset diversity.

- **GRPO.** PPO without the value network. Sample G completions per prompt; baseline with the group mean; normalize by group std. Used by DeepSeek-Math and DeepSeek-R1. Pros: scales to verifiable rewards (math, code, format); no reward-model bottleneck. Cons: G samples per prompt at training time.

The unifying mental model: all three are surrogate optimizers of $J(\theta) - \beta D_{\mathrm{KL}}(\pi_\theta \| \pi_{\mathrm{ref}})$. RLHF estimates the gradient with PPO and a value function; DPO eliminates online sampling by inverting the closed-form solution; GRPO eliminates the value function via the group baseline. Pre-training (Chapter 27) is the floor — it sets what is sayable — and post-training reshapes only a small region around it. Pre-training is calorie intake; alignment is digestion. Every chapter in this 31-chapter chain composes to produce that final sentence.

Block I — Inference and Serving

Chapter 32: KV cache mechanics: memory analysis and tokens-per-second benchmarks

Motivation

A trained transformer is a function; an *inference engine* is a schedule for evaluating that function token by token. The schedule chosen by every modern LLM serving stack — vLLM, TensorRT-LLM, llama.cpp, MLX, `transformers` — pivots on one data structure: the **KV cache**. It is the difference between $O(T^2)$ generation and $O(T^3)$ generation, between 200 tokens/sec and 5 tokens/sec on the same hardware, and between fitting a 32k-context Llama on one A100 and not. This chapter derives the KV cache from the autoregressive factorization, proves it preserves the model's distribution exactly, characterizes its memory cost, and benchmarks the speedup on the tiny GPT trained in Chapter 27.

Throughout, T denotes current sequence length, d the model dimension, N_ℓ the number of decoder layers, N_h the number of attention heads, and $d_k = d/N_h$.

Definitions

Definition 0.314 (Autoregressive generation)**.** *A decoder-only transformer parameterizes $p_\theta(x_t \mid x_{<t})$ for each position t. To sample a continuation of length L given a prompt of length T_0, one repeats: compute $p_\theta(\cdot \mid x_{<t})$, draw x_t, append, advance t. The naive implementation runs the full forward pass over all of $x_{<t}$ at every step.*

Remark 0.315 (Self-attention recap, Ch. 21–22). A single head at layer ℓ maps the input $H^{(\ell-1)} \in \mathbb{R}^{T \times d}$ to

$$Q = H^{(\ell-1)} W_Q, \quad K = H^{(\ell-1)} W_K, \quad V = H^{(\ell-1)} W_V,$$

and outputs

$$\mathrm{Attn}(Q, K, V) = \mathrm{softmax}\Big(\tfrac{QK^\top}{\sqrt{d_k}} + M_{\mathrm{causal}} \Big) V.$$

Multi-head attention concatenates N_h such heads.

Definition 0.316 (KV cache)**.** *For each layer ℓ, head h, and previously processed token position j, store the row vectors*

$$K_j^{(\ell,h)} \in \mathbb{R}^{d_k}, \qquad V_j^{(\ell,h)} \in \mathbb{R}^{d_v}.$$

At step $t+1$, given the new token embedding $h_t^{(\ell-1)}$, compute only Q_t, K_t, V_t for that single position, append K_t, V_t to the cache, and compute attention as

$$\mathrm{Attn}_t = \mathrm{softmax}\Big(\tfrac{Q_t[K_{0:t+1}]^\top}{\sqrt{d_k}}\Big) V_{0:t+1}.$$

There is no causal mask term because the cache contains only past keys by construction.

Proposition 0.317 (Cache size in bytes)**.**

$$\boxed{\mathrm{bytes}(T) \;=\; 2 \cdot N_\ell \cdot T \cdot N_h \cdot d_k \cdot b}$$

where b is the byte width of the storage dtype ($b = 2$ for fp16/bf16, $b = 4$ for fp32) and the leading 2 accounts for storing both K and V.

Theorems and proofs

Theorem 0.318 (Functional equivalence)**.** *Let f_θ denote the deterministic forward map of a causally-masked transformer. For any prompt $x_{0:T_0}$ and sampling seed s, the token sequence produced by cached generation equals the sequence produced by uncached generation.*

Proof. By induction on t. **Base** ($t = T_0$): no cache exists; both implementations run the same forward pass on $x_{0:T_0}$ and produce identical logits, hence (with shared seed s) identical x_{T_0}. **Inductive step.** Assume identical token sequences and identical layerwise activations $H_{0:t}^{(\ell)}$ through step t. At step $t+1$ the uncached path computes

$$Q_t^{(\ell)} = h_t^{(\ell-1)} W_Q, \;\; K_{0:t+1}^{(\ell)} = H_{0:t+1}^{(\ell-1)} W_K, \;\; V_{0:t+1}^{(\ell)} = H_{0:t+1}^{(\ell-1)} W_V,$$

then $\mathrm{Attn}(Q_t, K_{0:t+1}, V_{0:t+1})$. The cached path stored $K_{0:t}^{(\ell)}, V_{0:t}^{(\ell)}$ from prior steps; by the inductive hypothesis these stored values are exactly the rows that the uncached path recomputes (each $K_j^{(\ell)}$ depends only on $h_j^{(\ell-1)}$, fixed by the hypothesis). The cached path computes K_t, V_t from the same $h_t^{(\ell-1)}$, concatenates, and applies the same softmax–matmul. Composition of deterministic operations on equal inputs yields equal outputs. The shared sampler seed s then yields identical x_{t+1}. □

Remark 0.319. The proof relies critically on the *causal mask*: K_j, V_j for $j \le t$ never depend on later activations, so caching them is lossless. Without causality (e.g., bidirectional encoders), no analogous KV cache exists.

Theorem 0.320 (Compute saved per generation step)**.** *Generating L tokens after a prompt of length T_0 costs $\Theta((T_0+L)^2\, d)$ FLOPs with KV cache and $\Theta((T_0+L)^3\, d)$ without.*

Proof. At step t (current length $T_0 + t$), the per-layer attention cost decomposes into the $QK^\top$ matmul and the softmax $\cdot\, V$ matmul.

Cached. $Q_t \in \mathbb{R}^{1\times d_k}$ and $K \in \mathbb{R}^{(T_0+t)\times d_k}$, so $QK^\top$ costs $\Theta((T_0+t)d_k)$ per head, $\Theta((T_0+t)d)$ across heads. The V-multiply is the same order. Total per layer: $\Theta((T_0+t)d)$.

Uncached. The model recomputes all $T_0 + t$ rows of K, V and the full $(T_0+t)\times(T_0+t)$ attention matrix, giving $\Theta((T_0+t)^2 d)$ per layer.

Multiplying by N_ℓ (constant) and summing $t = 1, \ldots, L$:

$$\sum_{t=1}^{L} (T_0+t)\, d = \Theta((T_0+L)^2 d), \quad \sum_{t=1}^{L} (T_0+t)^2 d = \Theta((T_0+L)^3 d). \qquad \square$$

Theorem 0.321 (Memory cost)**.** *The KV cache for a fixed model grows linearly in context length T, with slope $2\,N_\ell\,N_h\,d_k\,b$ bytes per token.*

Proof. Direct from the bytes formula; differentiate with respect to T. □

Corollary 0.322 (Llama-7B at 8k)**.** *Take $N_\ell = 32$, $N_h = 32$, $d_k = 128$, $b = 2$ (fp16), $T = 8192$:*

$$\text{bytes} = 2 \cdot 32 \cdot 8192 \cdot 32 \cdot 128 \cdot 2 = 4.29 \times 10^9 \;\approx\; 4.0\,\text{GiB}.$$

This exceeds the size of the model's W_K, W_V projection weights themselves, and motivates the architectural responses of Ch. 22 (GQA, MQA) and the system-level response of PagedAttention.

Code sketch and benchmarks

The accompanying notebook implements a ~ 50-line PyTorch decoder with an optional `kv_cache` argument. Each block holds a (K, V) pair of shape `(B, n_heads, T_cache, d_k)` that is `torch.cat`-extended at every step. Device auto-detection picks `mps` on the M4 Pro, falling back to CUDA or CPU. The benchmark loop generates 256 tokens for prompt lengths $T_0 \in \{32, 64, 128, 256\}$ and reports the wall-clock ratio. With cache, throughput is roughly constant in T_0; without cache, it falls as $1/T_0^2$, giving observed speedups of $\sim 5\times$ at $T_0 = 32$ and $\sim 40\times$ at $T_0 = 256$ — consistent with Theorem 0.320.

Example 0.323 (Cache budget table). A table cell evaluates the bytes formula for: the Ch. 27 tiny GPT ($N_\ell = 4, N_h = 4, d_k = 16$), GPT-2 small $(12, 12, 64)$, Llama-7B $(32, 32, 128)$, and Llama-70B with 8-way GQA (effective $N_h = 8$ for the cache). At $T = 8192$ in fp16, the four caches are $\approx 16\,\text{MiB}$, $\approx 288\,\text{MiB}$, $\approx 4.0\,\text{GiB}$, and $\approx 2.5\,\text{GiB}$ respectively — the GQA model holds a longer context on the same accelerator than its denser sibling.

Connection to LLMs

Every production inference stack is, fundamentally, KV-cache management. **PagedAttention** (vLLM) treats the cache as virtual memory with block-level paging, eliminating fragmentation. **GQA / MQA** (Ch. 22) shrink N_h in the cache while keeping it for queries. **Continuous batching** packs many requests' caches side by side and schedules them at token granularity. **Speculative decoding** amortizes a verifier's cache across draft tokens. **Quantization** to int8 or int4 halves or quarters the per-token slope. The lesson of this chapter is that all of these techniques are reactions to two facts proved here: cached generation is exact (Thm. 0.318) and memory is linear in T (Thm. 0.321). Once both hold, the engineering question is just how to fit the line under the accelerator's HBM ceiling.

Chapter 33: Speculative decoding: draft + target model with rejection sampling correctness proof

Chapter 33: Speculative Decoding

1. Motivation

Autoregressive sampling from a transformer (Chapter 25) is latency-bound: each new token requires one full forward pass over $O(L)$ layers. Even with a KV cache (Chapter 32) eliminating recomputation over the prefix, the next token still costs one sequential pass through the target π.

On modern GPUs this is *memory-bandwidth bound* – the matmuls underutilize the compute units because we load the entire weight matrix to produce a single token's logits.

Speculative decoding (Leviathan et al. 2022; Chen et al. 2023) exploits a beautiful observation: a single forward pass of π over $K+1$ tokens costs roughly the same wall-clock time as a pass over 1 token, because the bottleneck is weight-loading, not arithmetic. So if we can *guess* the next K tokens cheaply, we can verify all K in parallel with one pass of π – and provably get samples from π exactly.

2. Definitions

Definition 0.324 (Draft model)**.** *A small, fast autoregressive model π_d (typically 5–10× smaller than the target) generating K candidate tokens $\tilde{x}_1, \ldots, \tilde{x}_K$ sequentially.*

Definition 0.325 (Target model)**.** *The model π whose distribution we actually want to sample from. After the draft proposes $\tilde{x}_{1:K}$, we run one forward pass of π on the prefix concatenated with $\tilde{x}_{1:K}$, producing $\pi(\cdot \mid x_{<i})$ at every position $i \in \{1, \ldots, K+1\}$ in parallel.*

Definition 0.326 (Rejection-sampling acceptance rule)**.** *Process draft tokens left-to-right. Accept $\tilde{x}_i$ with probability*

$$a_i \;=\; \min\Bigl(1,\; \tfrac{\pi(\tilde{x}_i \mid x_{<i})}{\pi_d(\tilde{x}_i \mid x_{<i})}\Bigr).$$

On the first *rejection at position i, discard $\tilde{x}_i, \tilde{x}_{i+1}, \ldots$ and resample from the residual distribution*

$$r_i(y) \;=\; \frac{\max\bigl(0,\; \pi(y \mid x_{<i}) - \pi_d(y \mid x_{<i})\bigr)}{Z_i}, \qquad Z_i \;=\; \sum_y \max(0, \pi(y) - \pi_d(y)).$$

If all K drafts accept, sample one bonus token from the target's free $(K+1)^{th}$ logit.

Remark 0.327 (Wall-clock speedup). Let $\bar{n} \in [1, K+1]$ be the mean tokens accepted per round and $\alpha = c_\pi / c_{\pi_d}$ the cost ratio. Then

$$\text{speedup} \;\approx\; \frac{\bar{n}}{1 + K/\alpha}.$$

For $\alpha \gg K$, speedup $\to \bar{n}$, capped at $K+1$.

3. Theorems and Proofs

Theorem 0.328 (Correctness: exact equivalence to sampling from π)**.** *For any draft distribution π_d with $\operatorname{supp}(\pi_d) \supseteq \operatorname{supp}(\pi)$, the speculative-decoding output at any single position is distributed exactly as π.*

Proof. Fix a position and condition on $x_{<i}$; write $\pi(y) := \pi(y \mid x_{<i})$ and $\pi_d(y) := \pi_d(y \mid x_{<i})$. The output equals a token x via exactly one of two disjoint events.

Case 1 – accept the draft. The draft proposes $\tilde{x} = x$ (probability $\pi_d(x)$) and we accept (probability $\min(1, \pi(x)/\pi_d(x))$):

$$\mathbb{P}_1(x) \;=\; \pi_d(x) \cdot \min\Bigl(1, \tfrac{\pi(x)}{\pi_d(x)}\Bigr) \;=\; \min(\pi_d(x), \pi(x)).$$

Case 2 – reject and resample from r. Total rejection probability:

$$P_{\text{rej}} \;=\; \sum_{\tilde{x}} \pi_d(\tilde{x}) \Bigl(1 - \min\bigl(1, \tfrac{\pi(\tilde{x})}{\pi_d(\tilde{x})}\bigr)\Bigr) \;=\; \sum_{\tilde{x}} \max(0, \pi_d(\tilde{x}) - \pi(\tilde{x})).$$

Since $\sum_y \pi = \sum_y \pi_d = 1$,

$$\sum_y \max(0, \pi_d(y) - \pi(y)) \;=\; \sum_y \max(0, \pi(y) - \pi_d(y)) \;=\; Z,$$

so $P_{\text{rej}} = Z$. After rejection we sample from r, contributing

$$\mathbb{P}_2(x) \;=\; Z \cdot r(x) \;=\; \max(0, \pi(x) - \pi_d(x)).$$

Sum. If $\pi(x) \le \pi_d(x)$: $\mathbb{P}_1 + \mathbb{P}_2 = \pi(x) + 0 = \pi(x)$. If $\pi(x) > \pi_d(x)$: $\mathbb{P}_1 + \mathbb{P}_2 = \pi_d(x) + (\pi(x) - \pi_d(x)) = \pi(x)$. Hence $\mathbb{P}(\text{output} = x) = \pi(x)$ for all x. □

The argument extends position-by-position: once $\tilde{x}_i$ is accepted, x_i has the marginal $\pi(\cdot \mid x_{<i})$, and the chain rule gives joint sampling from π over the full $K{+}1$ block.

Proposition 0.329 (Expected accepted prefix length)**.** *Let $\alpha_j = \mathbb{E}_{\tilde{x}_j \sim \pi_d}[\min(1, \pi(\tilde{x}_j)/\pi_d(\tilde{x}_j))]$ be the marginal acceptance probability at position j. The expected number of accepted draft tokens in a K-token round is*

$$\bar{n}_K \;=\; \sum_{i=1}^{K} \prod_{j=1}^{i} \alpha_j.$$

Sketch. Let $A_j = \mathbb{1}[\text{first } j \text{ drafts all accepted}]$. Conditional on the prefix, acceptance is independent across j, so $\mathbb{E}[A_j] = \prod_{j' \le j} \alpha_{j'}$. Sum by linearity of expectation. □

Remark 0.330 (TV-distance form). Using $\alpha = \sum_y \min(\pi(y), \pi_d(y)) = 1 - \frac{1}{2}\|\pi - \pi_d\|_1$, acceptance is governed by total variation distance.

Corollary 0.331 (Optimal draft is the target)**.** *If $\pi_d = \pi$, then $\alpha_j = 1$ and $\bar{n}_K = K$ – but no compute is saved because the draft costs as much as the target. The interesting regime is a cheap draft slightly worse than the target.*

4. Code Sketch and Benchmarks

The companion notebook (`cells.json`) implements `speculative_decode(draft, target, prompt, K)` using toy categorical distributions over a small vocabulary – no GPT inference is required, since the math *is* the chapter (Chapter 32 covers GPT-on-MPS benchmarking).

Example 0.332 (Experiments). (i) Draw 10,000 tokens via the speculative protocol and compare against direct samples from π; a χ^2 goodness-of-fit test fails to reject. (ii) Sweep noise q where $\pi_d = (1-q)\pi + q\,\text{Unif}$; plot $\bar{n}$ and predicted wall-clock speedup at $K = 4,\ \alpha = 7$. (iii) Identify the breakeven q at which speculative decoding loses to plain decoding.

5. Connection to LLMs

Production inference engines – **vLLM**, **llama.cpp**, **MLX-lm**, **TensorRT-LLM** – all ship speculative decoding. Reported wall-clock speedups: 1.5–3× for code/translation (high draft-target agreement) and 1.1–1.5× for open-ended chat. Variants:

- **Self-speculation / Medusa heads.** Attach extra prediction heads to the target itself, eliminating the separate draft model.

- **Tree attention / SpecInfer.** Verify multiple candidate continuations in one forward pass via causal-mask trees, raising $\bar{n}$.
- **EAGLE / Lookahead.** Train a tiny draft head conditioned on the target's hidden state for higher acceptance.

Theorem 0.328 transfers verbatim to all these schemes: as long as verification uses the rejection rule with the residual-distribution fallback, the output marginal is *exactly* π. No quality is sacrificed for the speedup – a rare free lunch in deep learning.

Chapter 34: Quantization: int8 and int4 post-training quantization with measured perplexity / throughput tradeoffs

Motivation

Llama-3 70B in `bfloat16` is 140 GB of weights – larger than any consumer GPU's VRAM and most workstation RAM. The same model in `int4` is roughly 40 GB and fits on a single 48 GB card with room for the KV-cache. The accuracy cost? For well-calibrated `int8` quantization, perplexity typically rises by less than 0.5%; for `int4`, by 1–3%. Quantization is the single technique most responsible for moving frontier-class models out of the data center and onto laptops.

We restrict attention to *post-training quantization* (PTQ): take an already-trained fp16/bf16 checkpoint and replace its weights with low-precision approximations without further gradient updates. The mathematical object of study is a map $Q : \mathbb{R}^{m\times n} \to \mathbb{Z}^{m\times n}$ together with a dequantizer D that approximates the identity, $D(Q(W)) \approx W$. Two questions follow: (i) what is the worst-case reconstruction error $\|W - D(Q(W))\|$ (Ch. 6 norm theory), and (ii) how does that error propagate to the cross-entropy and hence the perplexity e^{CE} (Ch. 17)?

Definitions

Definition 0.333 (Affine quantizer)**.** *Fix bit-width $b \in \{8, 4, 2\}$ and let $N = 2^b$. An* affine *quantizer with scale $s > 0$ and zero-point $z \in \mathbb{Z}$ is the function*

$$Q_{s,z}(w) \;=\; \mathrm{clip}\big(\mathrm{round}(w/s) - z,\; q_{\min},\; q_{\max}\big),$$

where the integer range is $[q_{\min}, q_{\max}] = [-N/2,\, N/2 - 1]$ (signed) or $[0,\, N - 1]$ (unsigned). The dequantizer is $D_{s,z}(q) = (q + z) \cdot s$. The map is symmetric *when $z = 0$.*

Definition 0.334 (Calibration)**.** *Given a calibration set $\{x_1, \ldots, x_K\} \subset \mathbb{R}^n$ and a layer with weight W, the scale s is chosen to cover the dynamic range observed during a forward pass – typically $s = \max_{ij} |w_{ij}|/(N/2 - 1)$ (max-abs) or the 99.99-percentile absolute value (percentile clipping, robust to outliers).*

Definition 0.335 (Granularity)**.** Per-tensor *quantization uses one scalar s for the entire matrix.* Per-row *(per-output-channel) gives each row $W_{i,:}$ its own s_i.* Per-channel-group *breaks each row into groups of* 128 *contiguous elements, each with its own scale. The metadata cost in bits per weight is, respectively, $O(1/mn)$, $O(1/n)$, $O(1/g)$, traded against tighter dynamic range.*

Definition 0.336 (Weight-only quantization, WbA16)**.** *Only the weight tensors are stored in* ***int****b; activations stay in fp16/bf16. The matmul dequantizes weights on-the-fly. This avoids the catastrophic effect of* activation outliers *identified by Dettmers et al. (2022, LLM.int8()).*

Definition 0.337 (Layer-local objective)**.** *For a linear layer $y = Wx$ and calibration matrix $X \in \mathbb{R}^{n\times K}$, the* layer-local quantization problem *is*

$$\widehat{W} \;\in\; \arg\min_{\widehat{W}\in\mathcal{Q}} \; \|WX - \widehat{W}X\|_F^2,$$

where $\mathcal{Q}$ is the discrete set of representable quantized matrices. This is the objective minimized by GPTQ (Frantar et al., 2023).

Theorems

Theorem 0.338 (Round-to-nearest error bound)**.** *Let Q_s be the symmetric affine quantizer with scale s and let $\widehat{w} = D_s(Q_s(w)) = s \cdot \mathrm{round}(w/s)$. For every w in the representable range,*

$$|\, w - \widehat{w}\,| \;\le\; s/2.$$

Proof. By definition of round-to-nearest, for any real u, $|u - \mathrm{round}(u)| \le 1/2$. Set $u = w/s$: $|w/s - \mathrm{round}(w/s)| \le 1/2$. Multiply by $s > 0$ to get $|w - s\cdot\mathrm{round}(w/s)| = |w - \widehat{w}| \le s/2$. □

Corollary 0.339 (Frobenius bound)**.** *For an $m \times n$ matrix W quantized symmetrically per-tensor with scale s, $\|W - \widehat{W}\|_F \le (s/2)\sqrt{mn}$.*

Theorem 0.340 (Per-row beats per-tensor on heterogeneous matrices)**.** *Let $W \in \mathbb{R}^{m\times n}$ have rows with ℓ_∞ norms $r_i = \|W_{i,:}\|_\infty$. With b bits and $N = 2^b$:*

- Per-tensor *scale $s_\star = (\max_i r_i)/(N/2-1)$ gives entry error $\le s_\star/2$, so*

$$\|W - \widehat{W}\|_F^2 \;\le\; \frac{mn}{4}\cdot\frac{(\max_i r_i)^2}{(N/2-1)^2}.$$

- Per-row *scale $s_i = r_i/(N/2-1)$ gives*

$$\|W - \widehat{W}\|_F^2 \;\le\; \frac{n}{4(N/2-1)^2}\cdot\sum_{i=1}^{m} r_i^2.$$

The ratio is

$$\frac{\textit{per-row bound}}{\textit{per-tensor bound}} \;=\; \frac{1}{m}\cdot\frac{\sum_i r_i^2}{(\max_i r_i)^2} \;\le\; 1,$$

with equality iff every row has the same ℓ_∞ norm. The gap grows with the variance of $\{r_i\}$.

Proof. Both inequalities follow from Theorem 0.338 applied entrywise; the ratio identity is algebra. The bound is tight when one row dominates (max $\gg$ mean), wasting most of the integer grid on small rows. □

Theorem 0.341 (GPTQ row decomposition)**.** *For a single layer with weight $W \in \mathbb{R}^{m\times n}$ and calibration Hessian $H = XX^\top \in \mathbb{R}^{n\times n}$ (PSD by construction), the layer-local objective decouples row-wise:*

$$\|WX - \widehat{W}X\|_F^2 \;=\; \sum_{i=1}^{m} (W_{i,:} - \widehat{W}_{i,:})\, H\, (W_{i,:} - \widehat{W}_{i,:})^\top.$$

Proof. $\|WX - \widehat{W}X\|_F^2 = \mathrm{tr}\big((W-\widehat{W})XX^\top(W-\widehat{W})^\top\big) = \sum_i \mathrm{row}_i(W-\widehat{W})\, H\, \mathrm{row}_i(W-\widehat{W})^\top$. □

Remark 0.342 (GPTQ algorithm sketch). Process columns $j = 1, \ldots, n$ in order. At step j, for each row i quantize w_{ij} to $\widehat{w}_{ij}$, compute $\delta = w_{ij} - \widehat{w}_{ij}$, and update unquantized columns $k > j$ by the Hessian-correcting term

$$w_{ik} \;\leftarrow\; w_{ik} \;-\; \delta \cdot \frac{(H^{-1})_{jk}}{(H^{-1})_{jj}}.$$

The Cholesky factor of H^{-1} allows this in $O(n^3)$ per layer. The correction absorbs the rounding error of column j into the not-yet-quantized columns, partially cancelling subsequent rounding. The naive RTN baseline is the special case where the correction is omitted.

Code sketch and benchmarks

The accompanying notebook contains six experiments. (1) A bytes-math table for GPT-2 small (124M), Llama-7B, Llama-70B at fp16/int8/int4. (2) A round-to-nearest implementation `quantize(w, bits)` with measured Frobenius error on a Gaussian matrix. (3) A 64×128 matrix with row norms $r_i = 10^{i/64}$ comparing per-tensor and per-row scales at int4: per-row reduces error by an order of magnitude. (4) A GPTQ implementation on a $d_{\text{in}} = 16, d_{\text{out}} = 8$ linear layer with 100-sample calibration: GPTQ beats RTN on the layer-output reconstruction error. (5) End-to-end perplexity: a miniature Ch. 27-style numpy GPT is trained briefly, then quantized to int8 and int4 by RTN; perplexity on the training corpus is reported before and after.

Connection to LLMs

Quantization closes the *deployment gap*. Training requires fp32 master weights (Ch. 14 AdamW state, Ch. 27 mixed precision); inference needs only the forward pass, where weight precision dominates memory bandwidth on modern GPUs. The empirical rule – $< 0.5\%$ perplexity loss for int8, 1–3% for int4 with GPTQ – means a Llama-3 70B user trades roughly 2% perplexity for a $3.5\times$ memory reduction and $2\times$ inference throughput. AWQ, SmoothQuant, and HQQ extend this picture: all minimize variants of Theorem 0.341's layer-local objective, differing in how they reweight by activation statistics. The thread that ties them all to first principles is Theorem 0.338: every byte saved costs at most $s/2$ of error per weight, and the entire research program is about choosing s wisely.